TFLite to Verilog
streaming CNN compiler
for embedded AI applications

andromeda.py
+ conv2d.v
+ concatenate.v
+ replicate.v

Replicate(), Concatenate(), Add()

https://vision.middlebury.edu/stereo/data/scenes2006/

Applications

LAD-RCNN:A Powerful Tool for Livestock Face Detection and Normalization
EvConv: Fast CNN Inference on Event Camera Inputs For High-Speed Robot Perception
    Sony Event-based Vision Sensor
CNN-based Methods for Object Recognition with High-Resolution Tactile Sensors
FusionRCNN: LiDAR-Camera Fusion for Two-stage 3D Object Detection
Low-complexity CNNs for Acoustic Scene Classification
Counting Varying Density Crowds Through Density Guided Adaptive Selection CNN and Transformer Estimation
Agricultural Plantation Classification using Transfer Learning Approach based on CNN
Predictive Modeling of Charge Levels for Battery Electric Vehicles using CNN EfficientNet and IGTD Algorithm
A CNN Approach for 5G mmWave Positioning Using Beamformed CSI Measurements

FPGA as compute platform

2.1. Intel Agilex® 7 FPGAs and SoCs F-Series

Table 4. F-Series FPGA Family Plan—Core Features

The values in this table are maximum resources or performance.

supports bfloat16

Device	Logic Element	Adaptive Logic Module	eSRAM		M20K		MLAB		DSP		Crypto Block
			Count	Size (Mb)	Count	Size (Mb)	Count	Size (Mb)	Count	18×19 Multiplier	
AGF 006	573,480	194,400	—	—	2,844	56	9,720	6	1,640	3,280	—
AGF 008	764,640	259,200	—	—	3,792	74	12,960	8	2,296	4,592	—
AGF 012	1,178,525	399,500	2	36	5,900	115	19,975	12	3,743	7,486	—
AGF 014	1,437,240	487,200	2	36	7,110	139	24,360	15	4,510	9,020	—
AGF 019	1,918,975	650,500	1	18	8,500	166	35,525	20	1,354	2,708	2
AGF 023	2,308,080	782,400	1	18	10,464	204	39,120	24	1,640	3,280	2
AGF 022	2,208,075	748,500	—	—	10,900	212	37,425	23	6,250	12,500	—
AGF 027	2,692,760	912,800	—	—	13,272	259	45,640	28	8,528	17,056	—

10000 DSP @500MHz = **5 TFLOP**
10000 M20K @ 500MHz = **10 TB/s**

example: 18-layer CNN
in: 3840x2160 RGB video @100fps
out: feature map / prediction @100fps

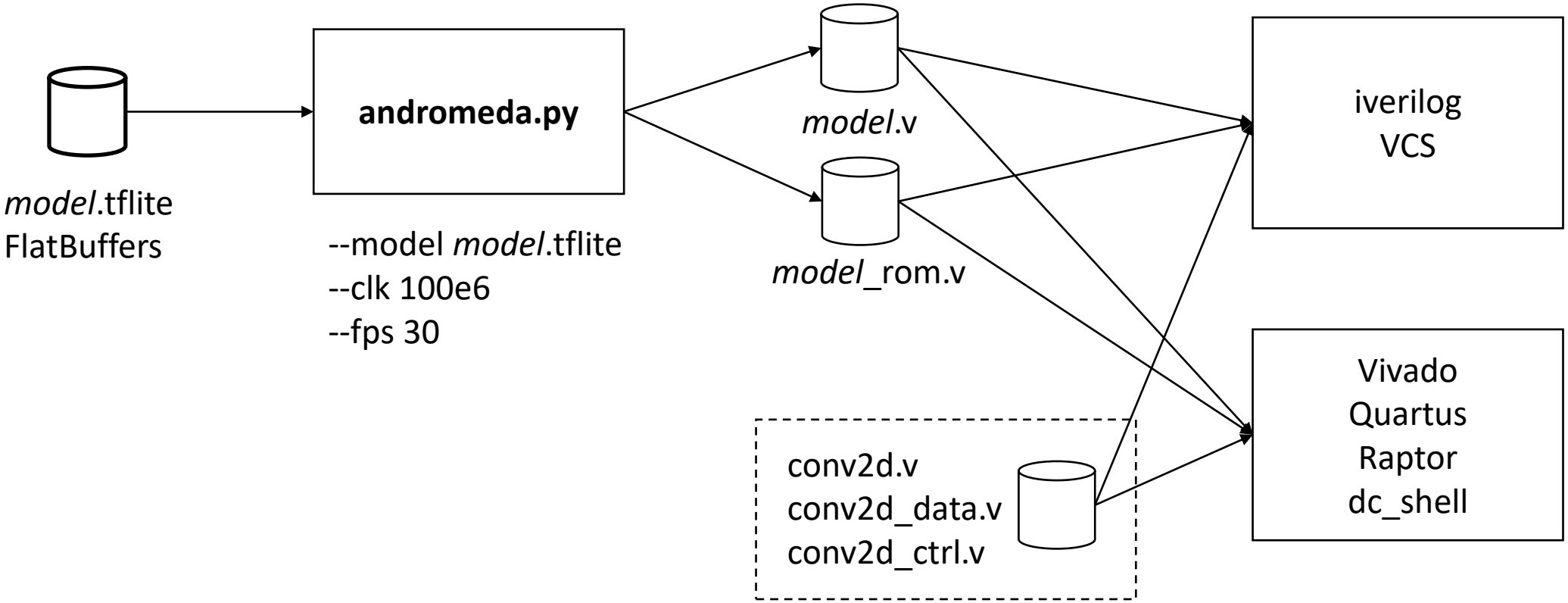
Nvidia Jetson Orin:
205 GB/s memory bandwidth
5.3 FP32 TFLOP (<60% utilization, V100)

Target applications

1. Prediction / parameter estimation from joint sensor data at ~100K samples/sec
   - e.g. MLX90640 thermal array, piezo, Hall effect
   - app: thermal screening , motor control, equipment monitoring
2. AI audio processing using multiple microphones/speakers
3. Advanced multi-camera vision tasks at 100M+ pixels/s
   - 300+ FPS, 20M+ uncompressed pixel resolution, 10+ cameras, hyperspectral
4. AI radio using RF baseband IQ samples, MIMO antennas

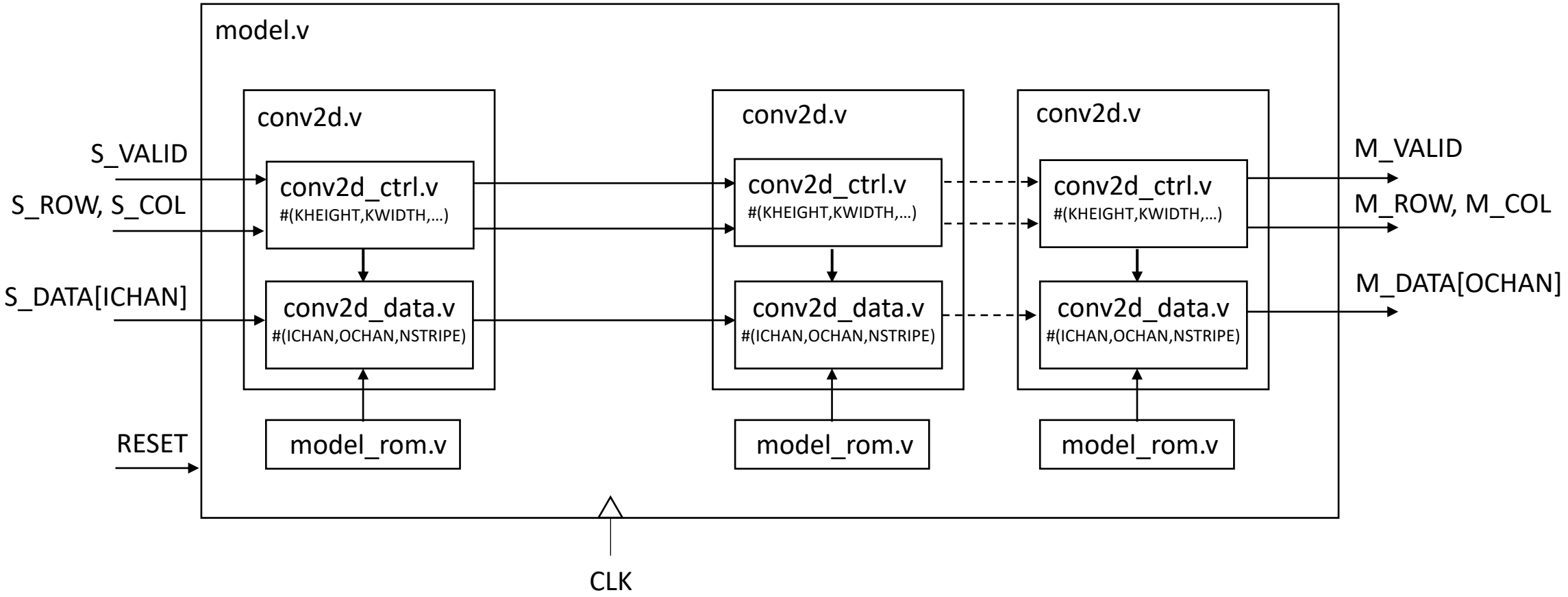
Replace traditional DSP algorithms with trained deep models
- Performance improvement from using deep model instead of heuristics
- Can learn over time and adapt
- Can automate the implementation of e.g. MATLAB model using imitation learning

Streaming CNN compiler

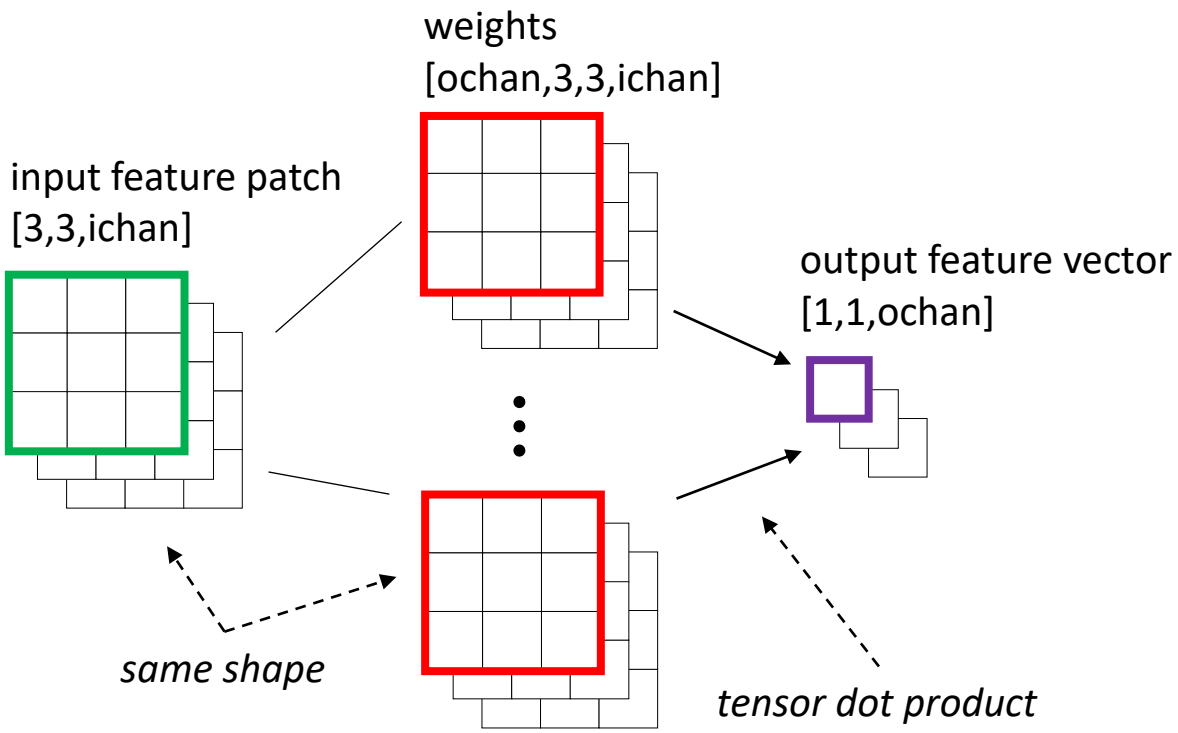


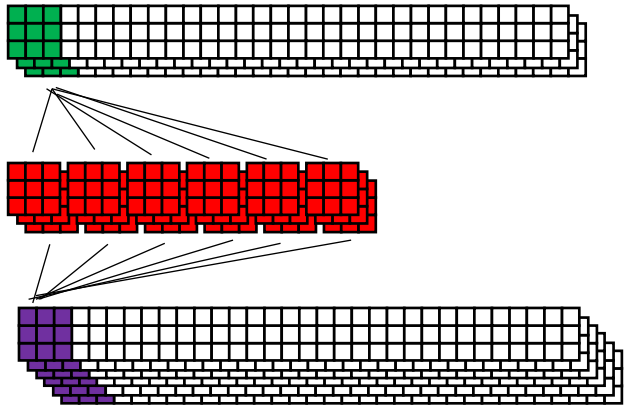
automated tool to compile TFLite model into synthesizable Verilog, using conv2d.v parameterized module
compiler generates parallel code for OCHAN*NSTRIPE MAC operations per layer

Generated Verilog

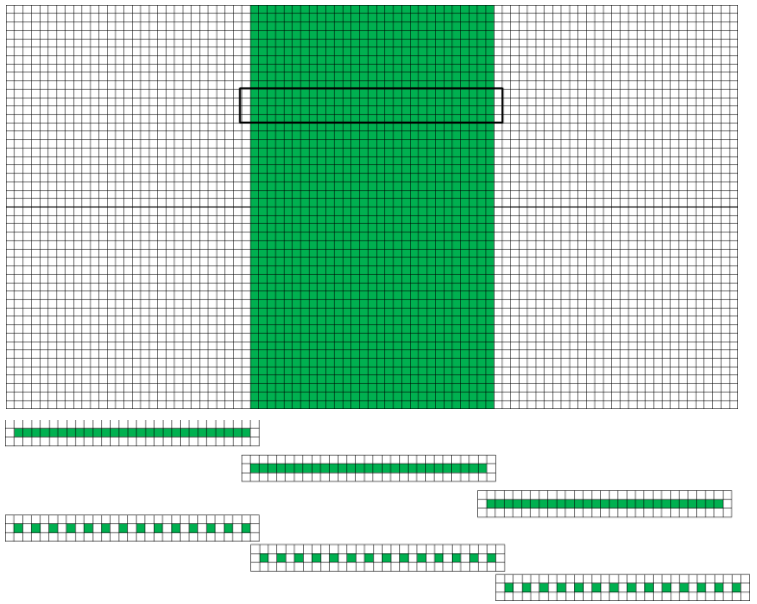


Parallel computation of CNN dot products

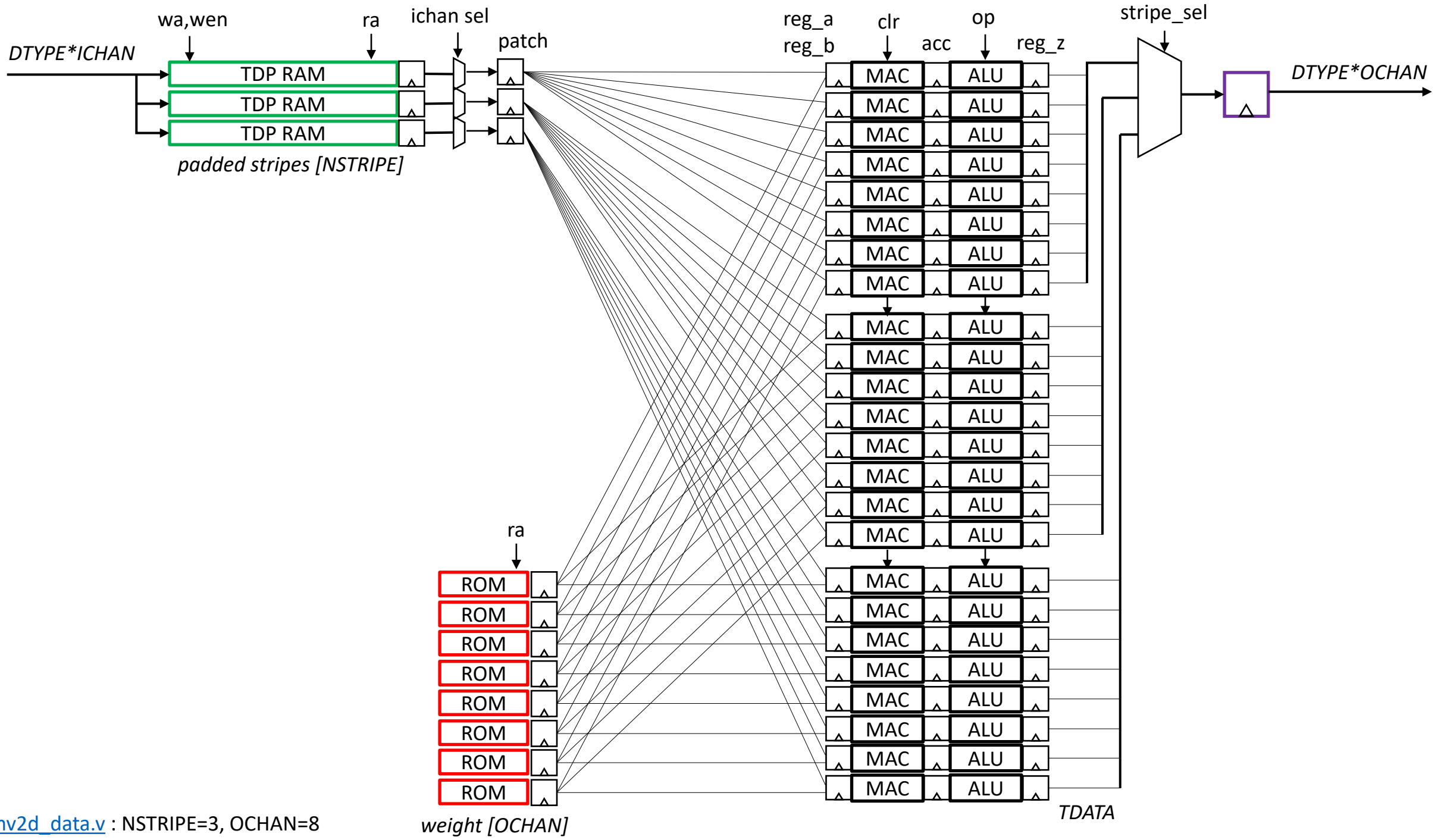




parallel ochan



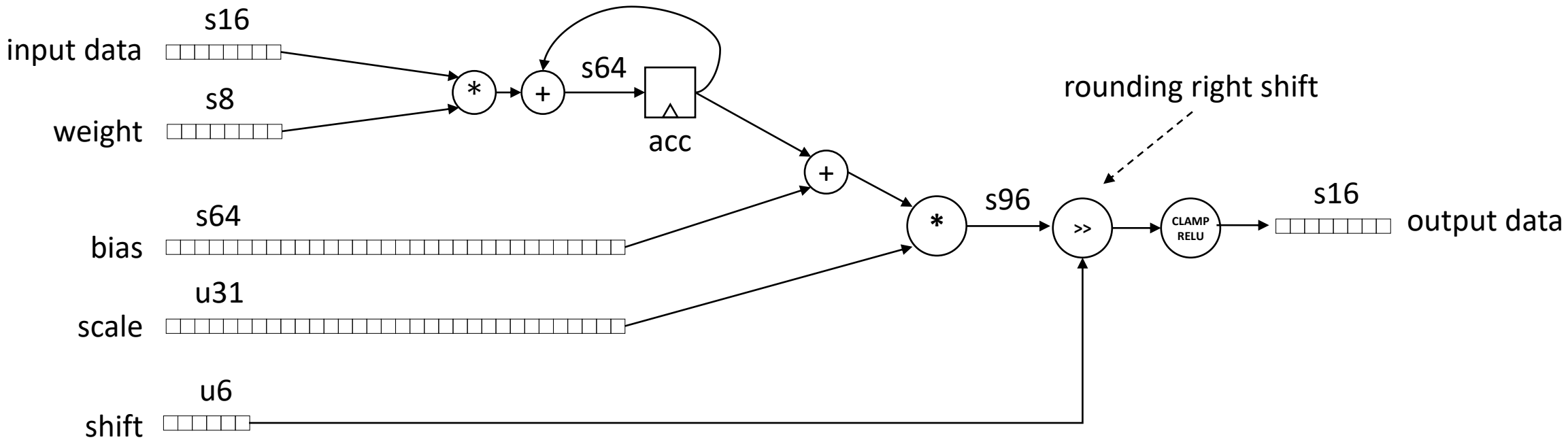
parallel nstripe



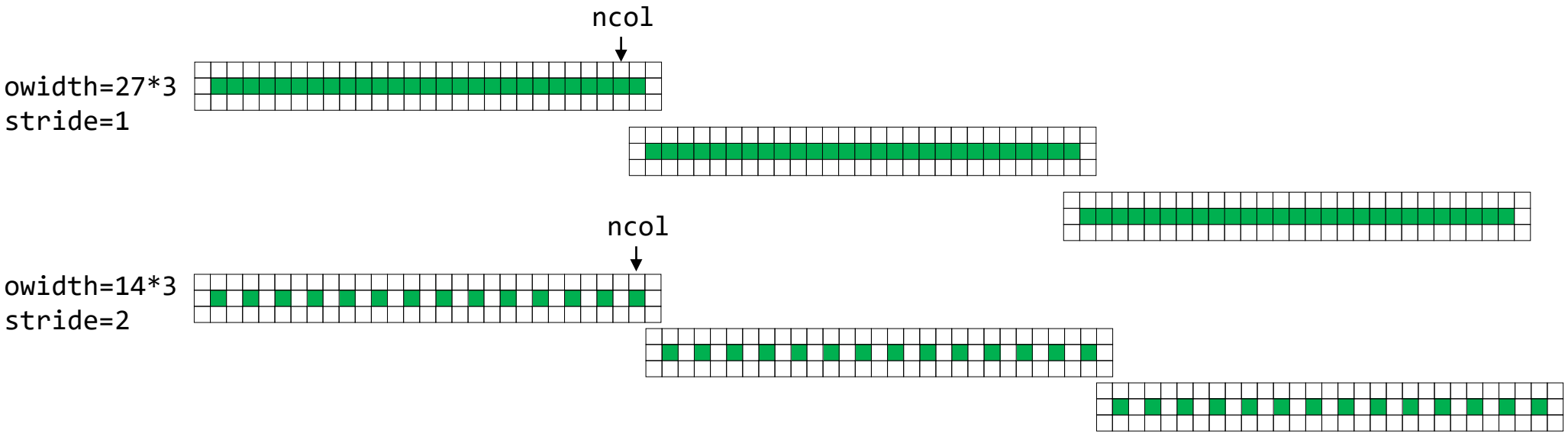
conv2d_data.v : NSTRIPE=3, OCHAN=8

TFLite 16x8 quantization

1: compute dot product
2: add bias
3. multiply by scale
4. shift right with rounding
5. clamp and RELU

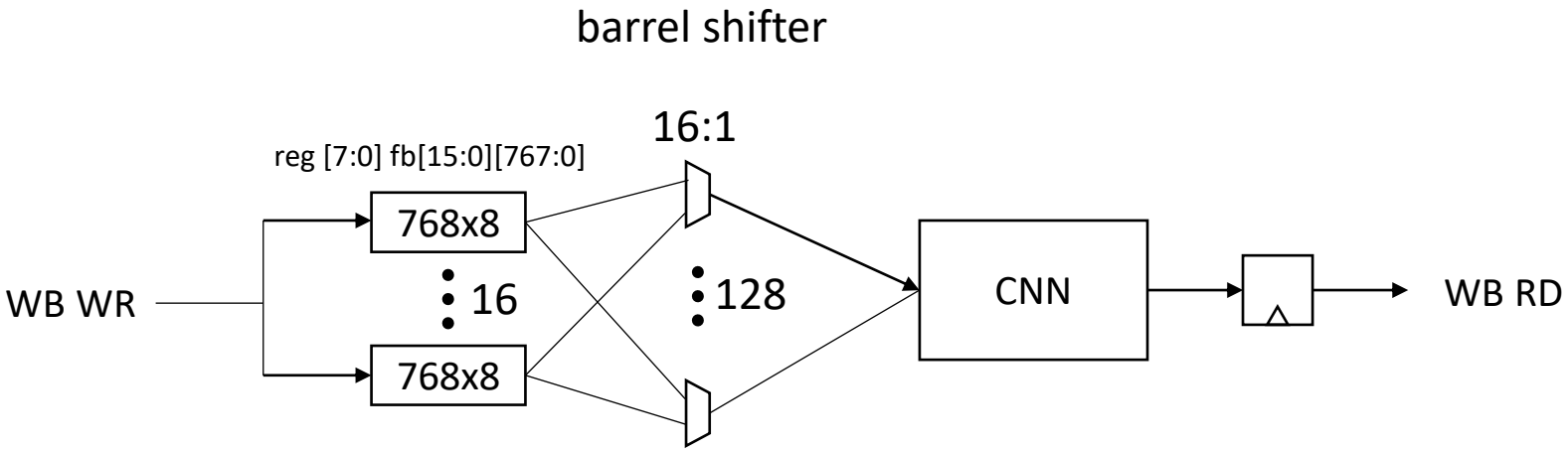


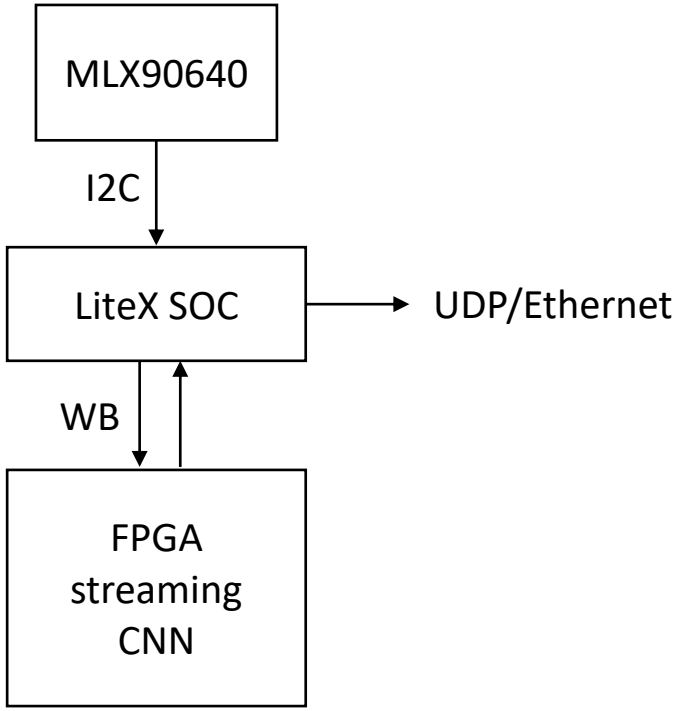
CNN striped parallel execution




Waves
1 ns
2 ns
3 ns
4 ns
5 ns
6 ns
7 ns
8 ns
9 ns
10 ns

Demo





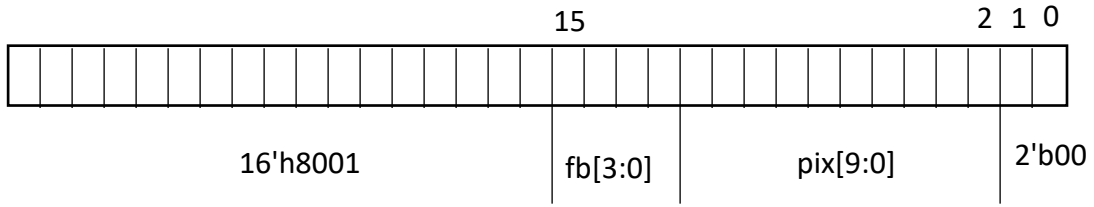
1. read MLX90640 frame using I2C
2. read previous prediction and assert seq++
3. assert reset
4. copy frame to next fb[15:0]
5. increment and write chan[3:0]
6. deassert reset
7. goto 1

Demo

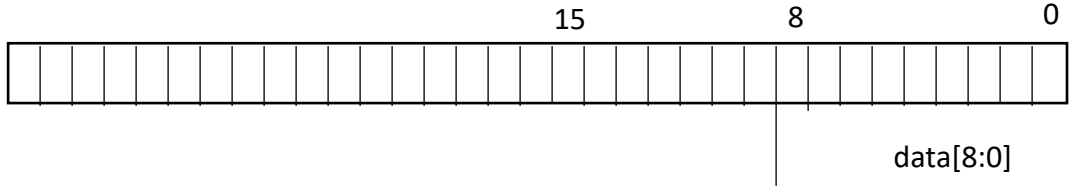
chan0[3:0] selects which physical fb is fed to the CNN input as channel 0

pred seq[5:0] increments every m_tdata_valid

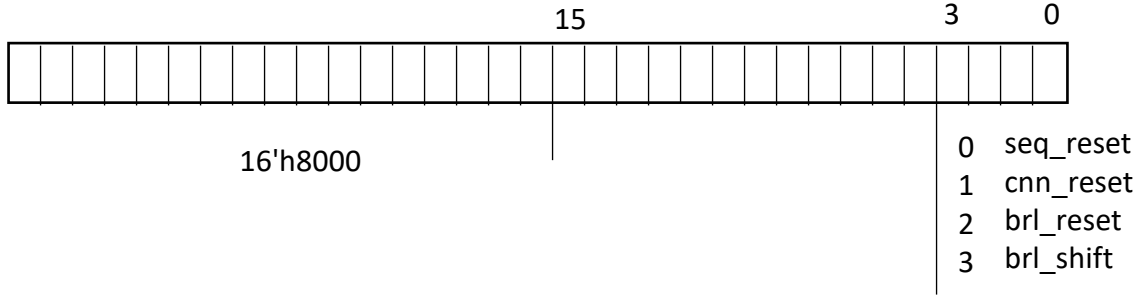
FB addr



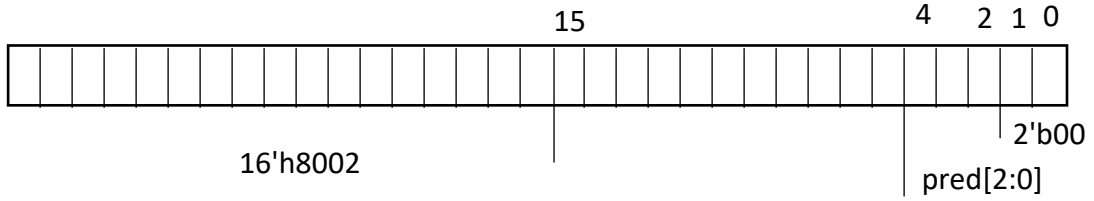
FB data



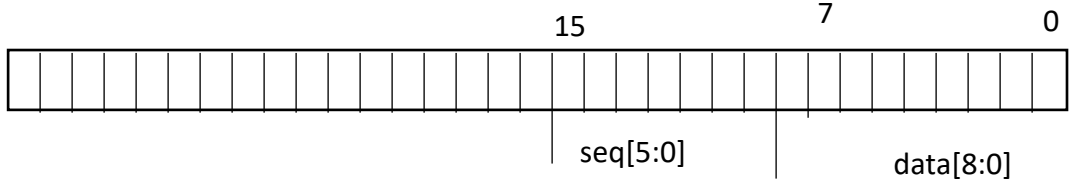
CSR 0x80000000
CSR data

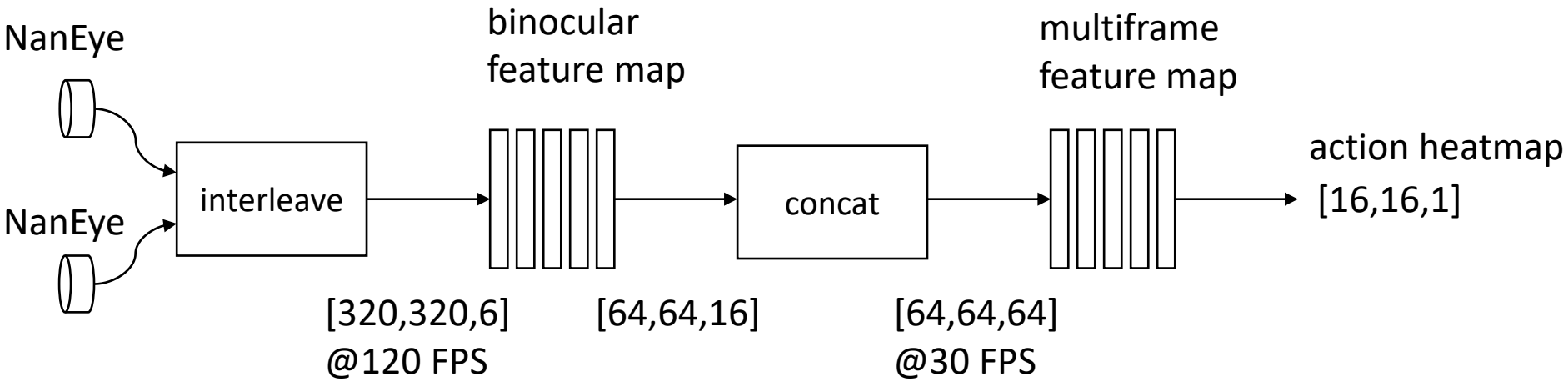


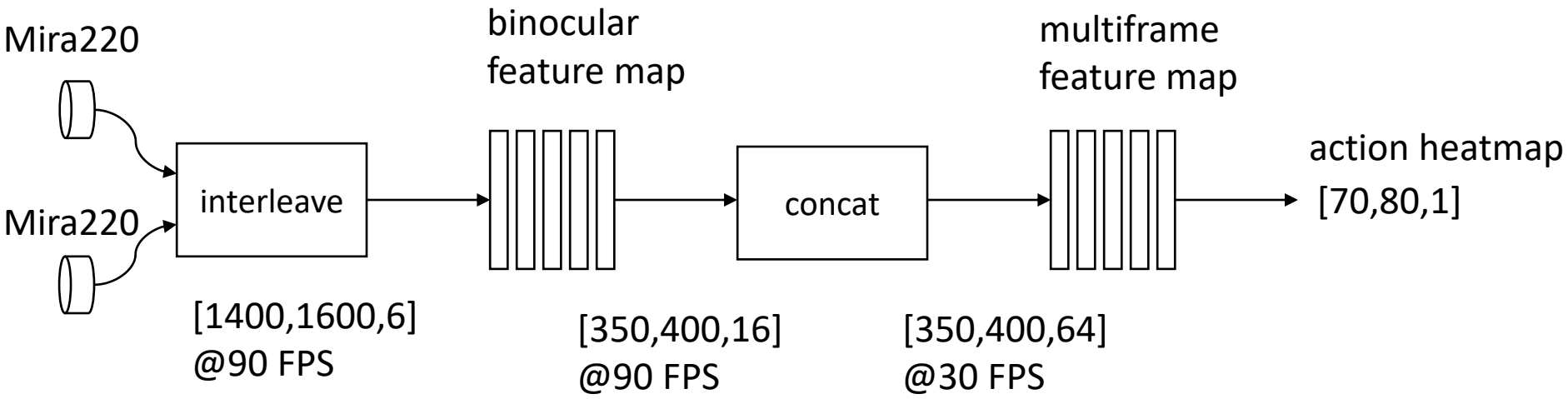
PRED addr

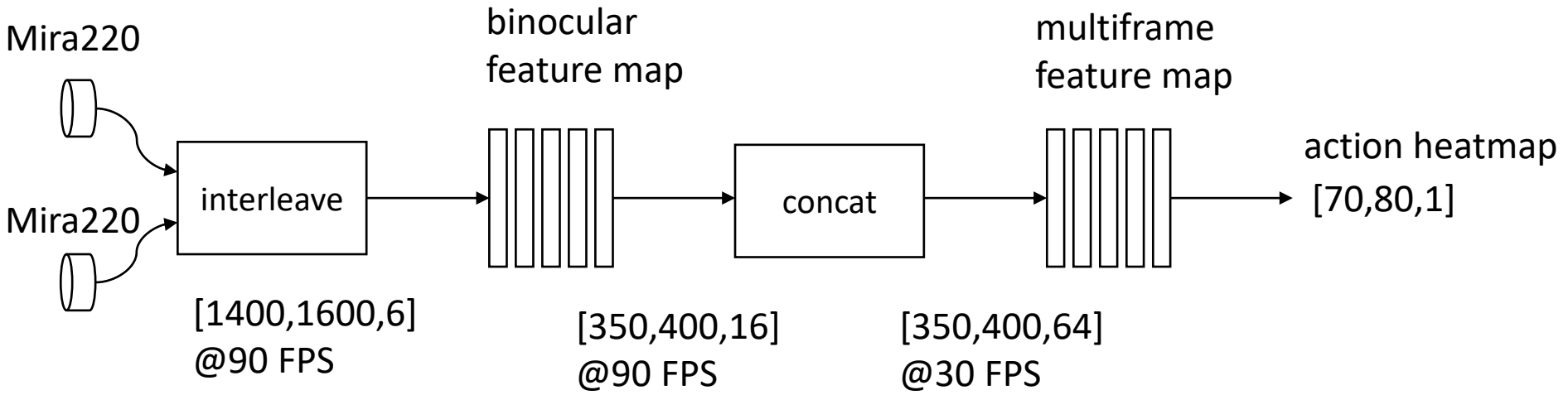


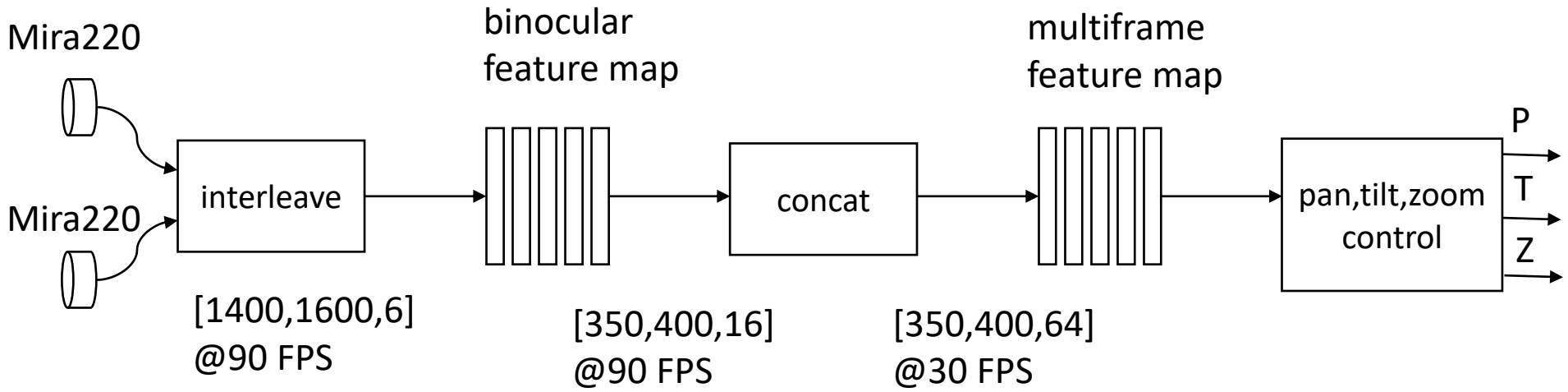
PRED data



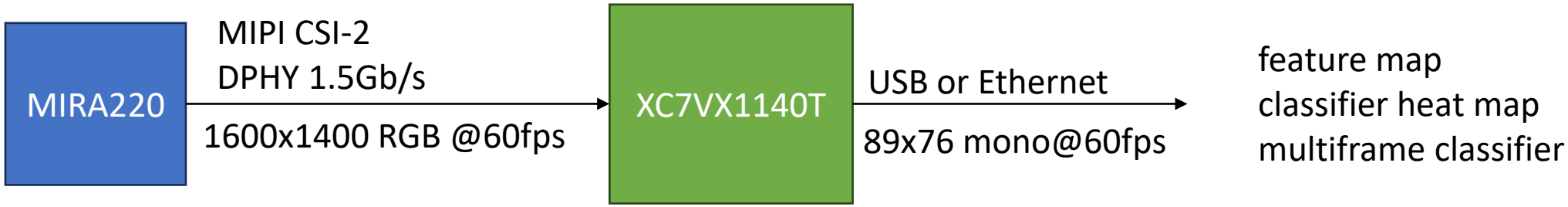

NanEye
NanEye
interleave
binocular
feature map
concat
multiframe
feature map
action heatmap
[16,16,1]
[320,320,6]
@120 FPS
[64,64,16]
[64,64,64]
@30 FPS


Mira220
Mira220
interleave
binocular
feature map
concat
multiframe
feature map
action heatmap
[70,80,1]
[1400,1600,6]
@90 FPS
[350,400,16]
@90 FPS
[350,400,64]
@30 FPS


Mira220
Mira220
interleave
binocular
feature map
concat
multiframe
feature map
action heatmap
[70,80,1]
[1400,1600,6]
@90 FPS
[350,400,16]
@90 FPS
[350,400,64]
@30 FPS

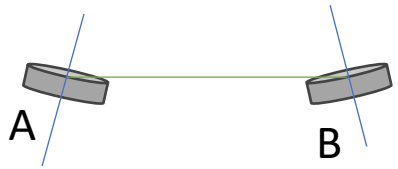

Mira220
Mira220
interleave
binocular
feature map
concat
multiframe
feature map
pan,tilt,zoom
control
P
T
Z
[1400,1600,6]
@90 FPS
[350,400,16]
@90 FPS
[350,400,64]
@30 FPS

High performance demo



classifier heat map (auto PTZ)
multiframe classifier (action recognition)
stereo vision (RGBD)
hyperspectral imaging, feature map based on sensor fusion
line scan image sensor with infinite scrolling
audio analysis using spectrograms https://en.wikipedia.org/wiki/Spectrogram
self supervised visual feature learning https://arxiv.org/pdf/1902.06162.pdf

binocular vision demo - self supervised binocular feature map training
https://en.wikipedia.org/wiki/Binocular_vision



~10 deg vergence angle
~10 cm distance between cameras

predict A given B, B given A, sharing weights
self supervised "pixels to feature map" learning with binocular inputs, multi-frame features

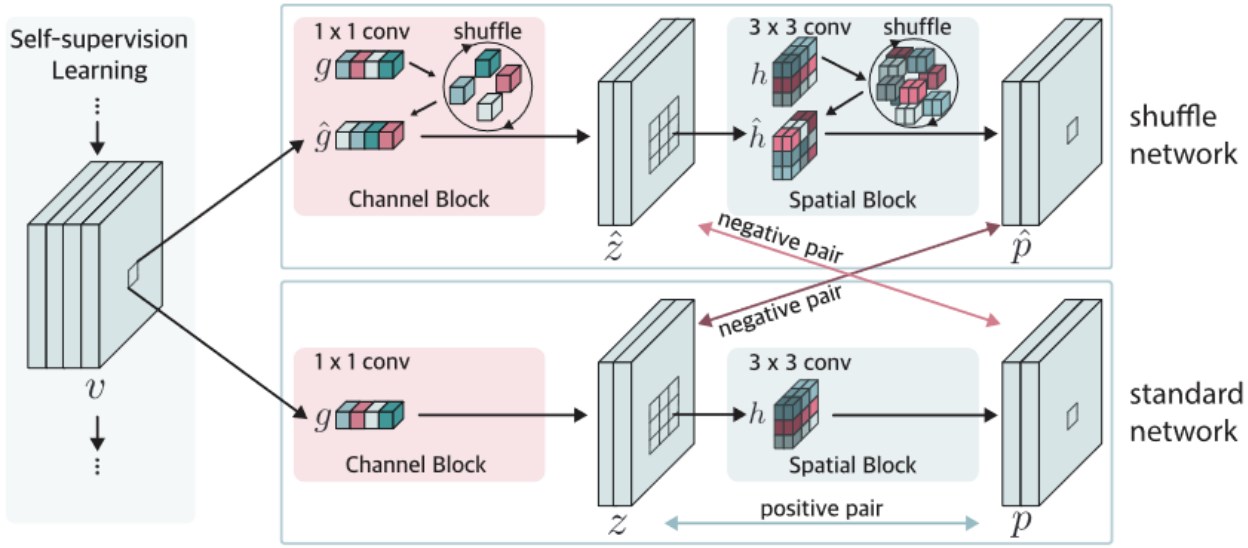


Fig. 2. Overall architecture. Our framework shuffles the convolution kernels to make the network learn spatial information within a small region. We use two networks: standard network and shuffle network. Each network contains two blocks called channel block and spatial block. The channel blocks are attached to the output v which is an arbitrary output of the backbone network. The output v is fed into both channel blocks g and $\hat{g}$ where $\hat{g}$ is randomly shuffled kernel of g. The output of each channel block is then fed into the spatial block h and $\hat{h}$. The outputs of each block of standard network z and p are considered as a positive pair. The outputs between the shuffle network and standard network are considered as negative pairs. To make the pairs asymmetric, $\hat{z}$ and p are considered as one negative pair and z and $\hat{p}$ are considered as the other negative pair for our module.

https://www.sciencedirect.com/science/article/pii/S0020025522013032
https://www.robots.ox.ac.uk/~vgg/publications/2021/Ge21/ge21.pdf

self supervised egocentric multicamera representations

- predict camera A feature map given camera B feature map
- predict camera B given camera A, share weights
- can extend to more cameras, multiframe feature maps at high frame rate

References

Research
AoCStream: All-on-Chip CNN Accelerator With Stream-Based Line-Buffer Architecture
https://github.com/Xilinx/finn
A Study on the Intersection of GPU Utilization and CNN Inference

Commercial Edge AI
https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/
https://perceive.io/

TFLite
https://www.tensorflow.org/lite/api_docs
https://arxiv.org/pdf/1712.05877.pdf
https://www.tensorflow.org/lite/performance/quantization_spec
https://www.tensorflow.org/lite/performance/post_training_quantization
https://github.com/tensorflow/tensorflow/blob/92f84571edb98bdb31f1e7b1e23f17bf2c1438e9/tensorflow/lite/kernels/internal/reference/conv.h#L101
https://pypi.org/project/tflite/

Image Encoder Concept

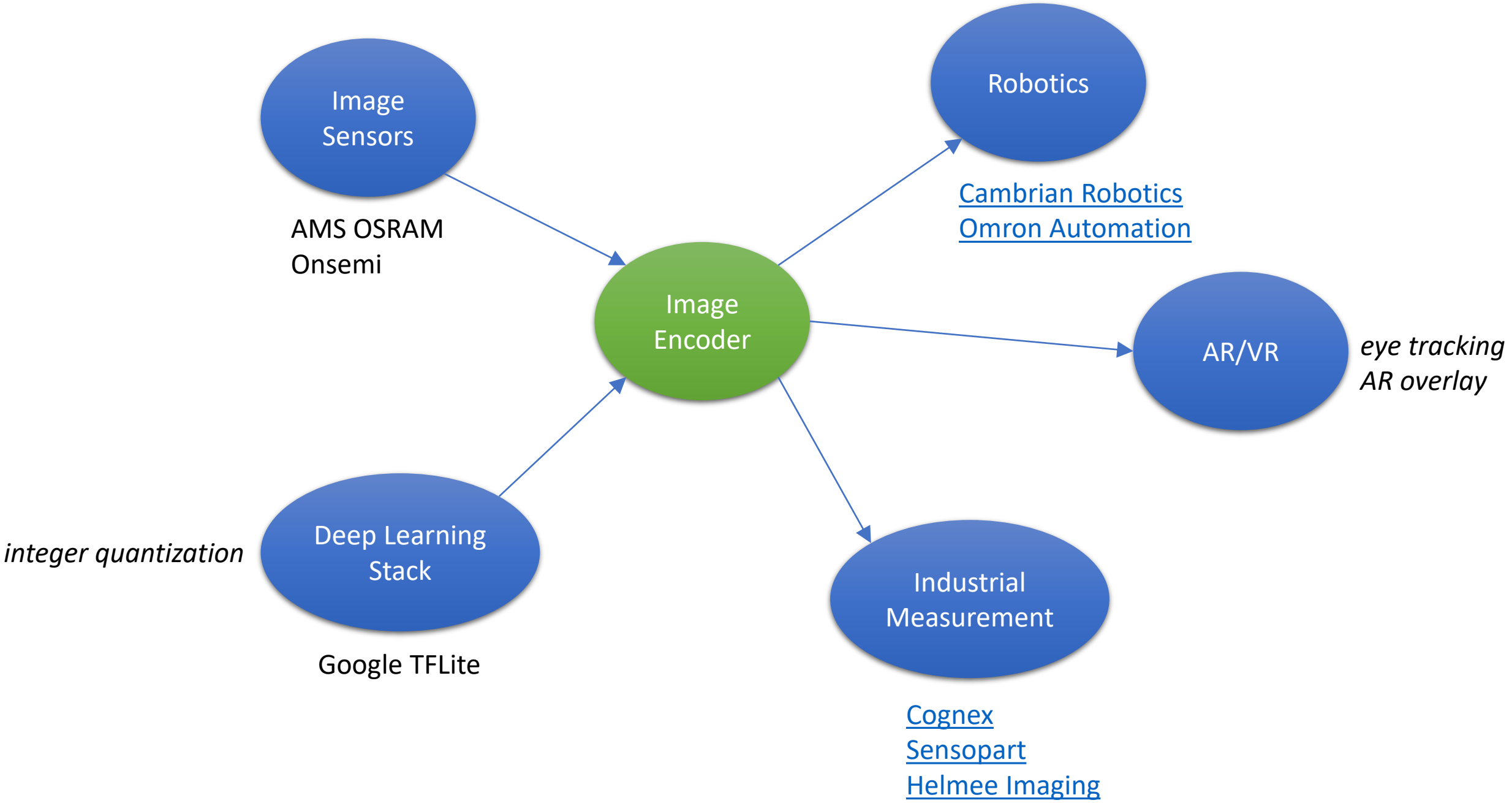
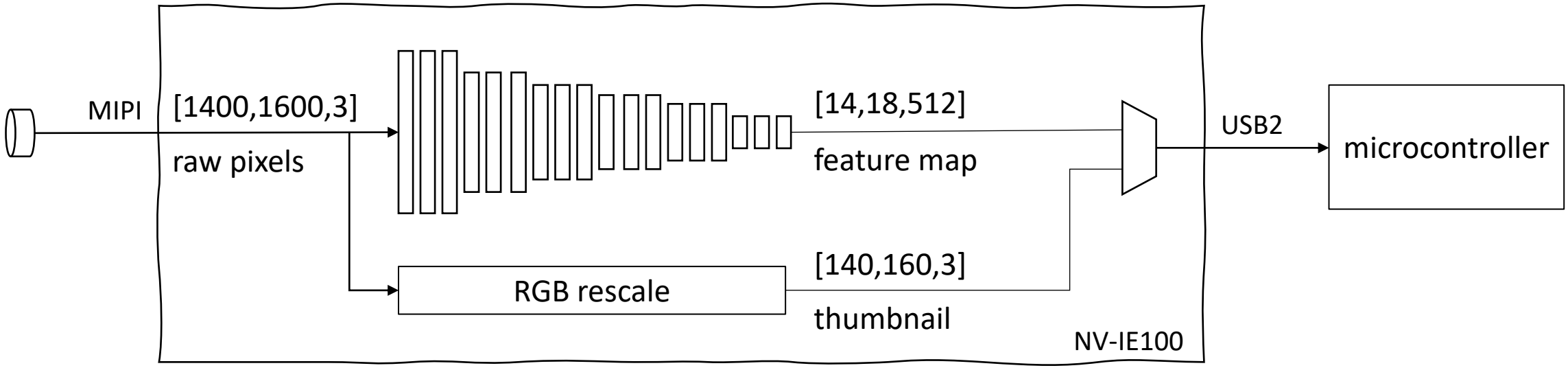


Image Sensors
AMS OSRAM
Onsemi
Robotics
Cambrian Robotics
Omron Automation
Image Encoder
AR/VR
eye tracking
AR overlay
integer quantization
Deep Learning Stack
Google TFLite
Industrial Measurement
Cognex
Sensopart
Helmee Imaging

Image Encoder Concept



scalable foundation vision CNN with 8M parameters
reconfigurable SRAM-based weights

MIPI
1600x1400 RGB pixels
12 bits/pixel
30 frames/s
= 2.42 Gb/s

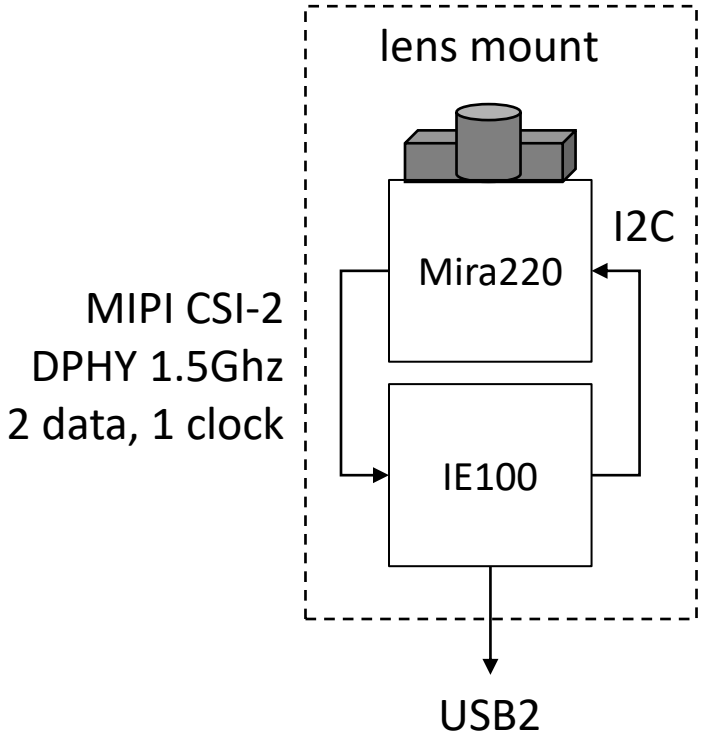
30X compression

USB
[14,18,512] features
16 bits/feature
30 frames/s
= 62 Mb/s

USB
[140,160,3] thumbnail
8 bits/pixel
30 frames/s
= 16 Mb/s

NV-IE100
500MHz clock
3840 integer MAC units
64Mb SRAM (weights)
20Mb SRAM (rowbuf)

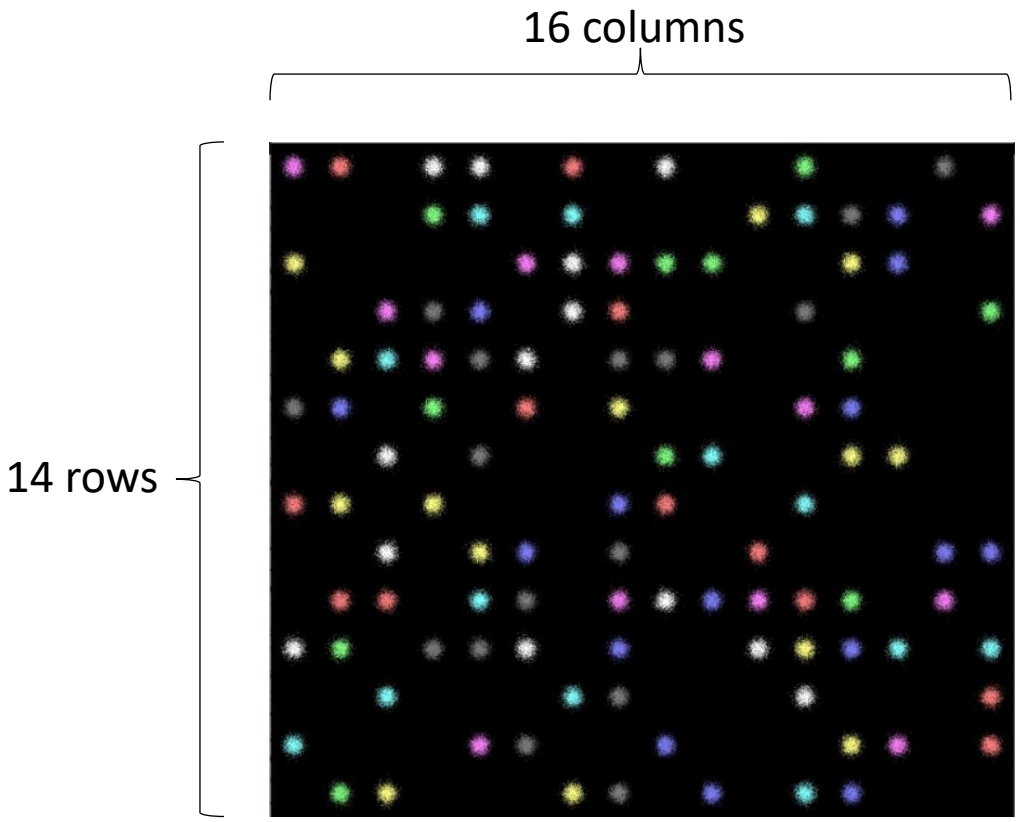
Reference Board Concept



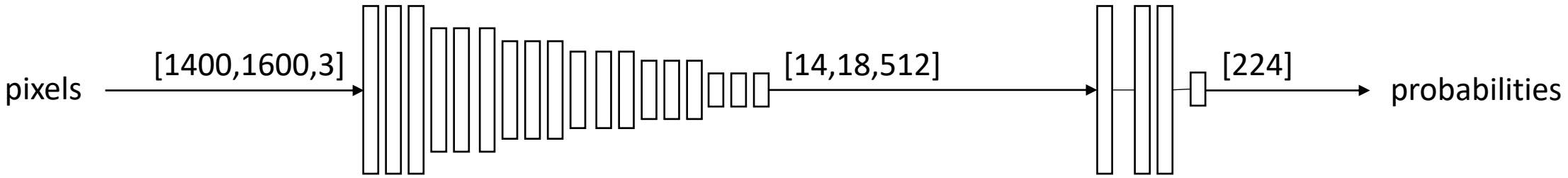
- real time image encoder
- compact form factor, USB powered
- supports a broad range of applications
- low pin count interfaces
- includes USB to I2C for configuration
- memory map provides access to weights

NV-IE USB endpoint	M/S
feature pipe	M
thumbnail pipe	M
I2C bridge	S
NV-IE memory map	S

Receptive field unit test



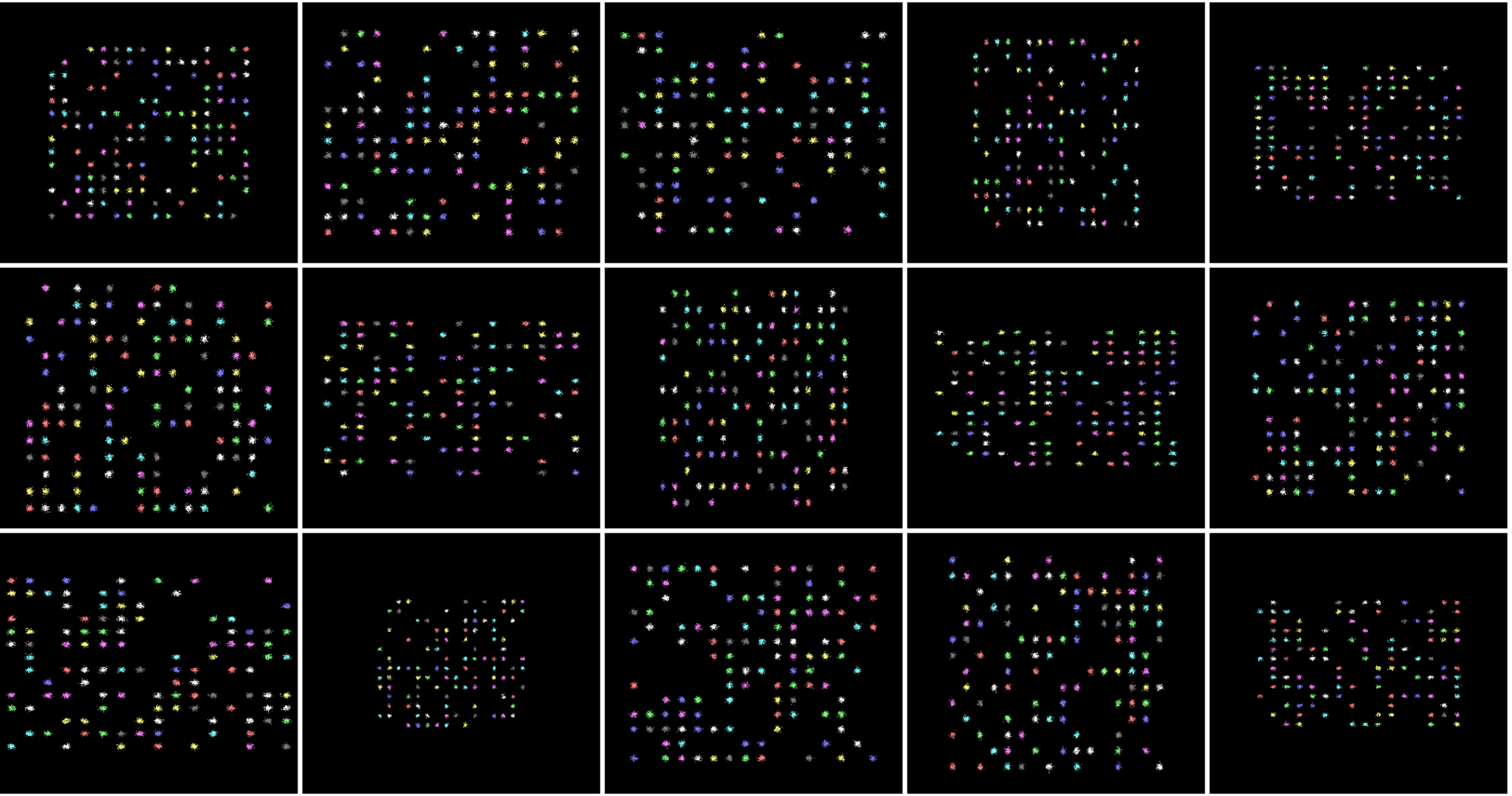
randomly populate 16x14 grid locations
label is '0' if empty, '1' if colored ball
224 binary labels



IE100 CNN

1x1 conv, 2x1000 dense, 224 dense
binary crossentropy loss

thumbnail gallery of 1600x1400 RGB input images

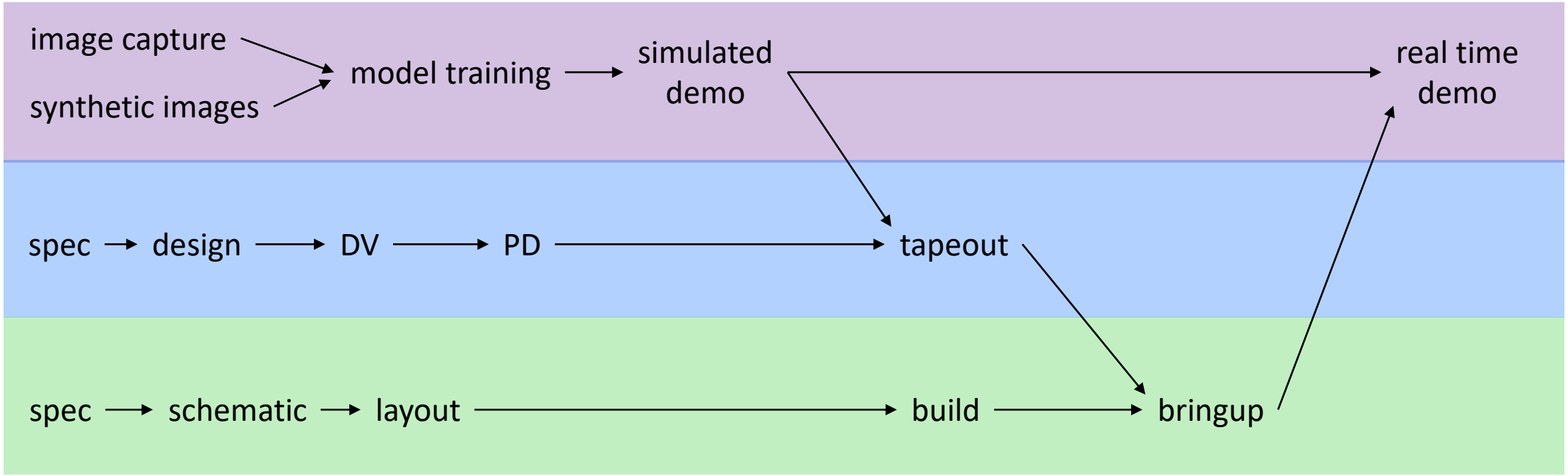


Product Planning

Milestone	Dependencies	Status
Concept Commit	PRD/datasheet die size/power estimate preliminary budget/schedule	needed for joint development proposal
Execution Commit	complete program plan to FCS	
FCS	standard SoC development process	

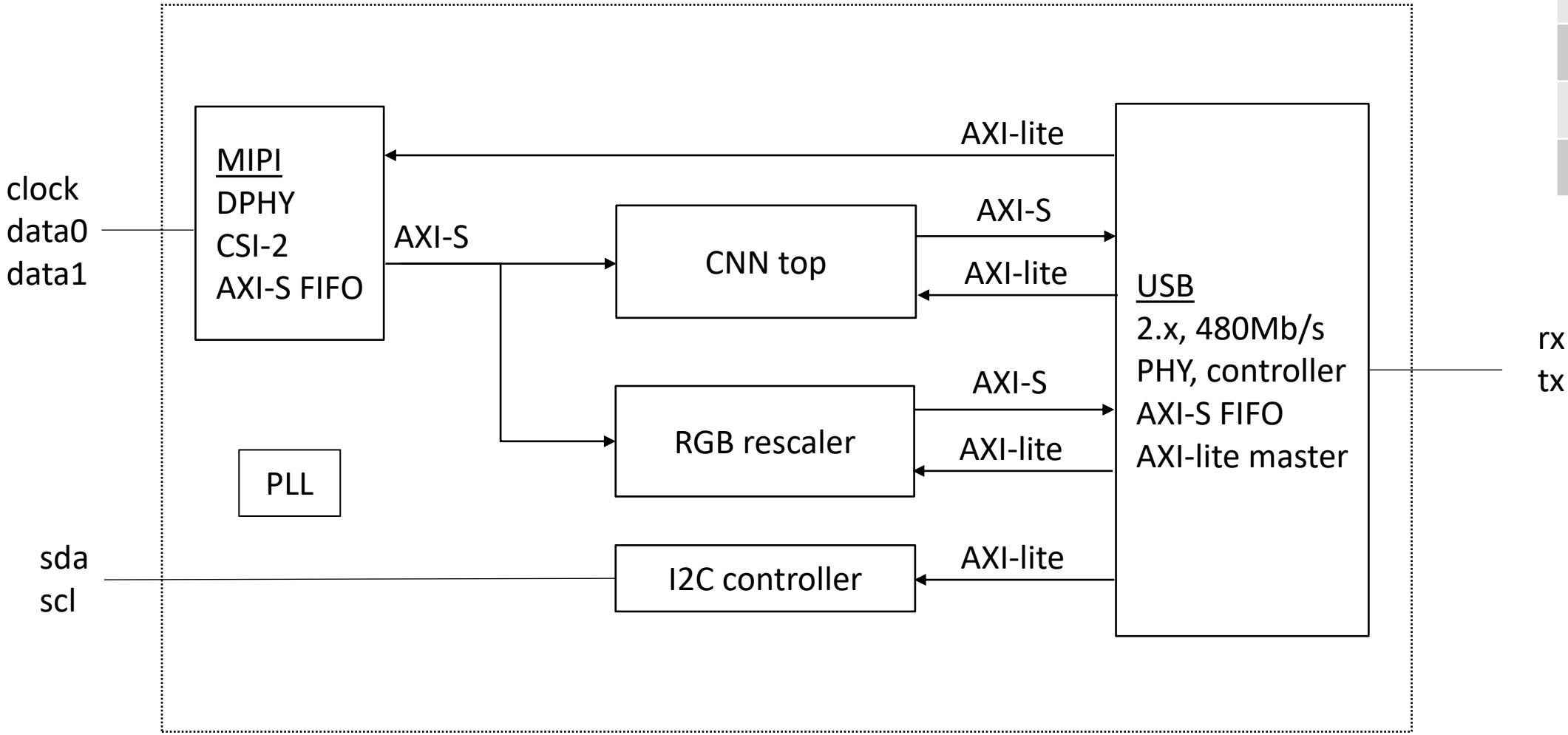
PRD Feature	Priority	Schedule
MIPI CSI-2, DPHY RX, 1C 2D, 1.5GHz	P1	MVP, fastest TTM
[1400,1600,3] input shape (frame) 12 bits/pixel 30 frames/s	P1	Mira220 RGB
[14,18,512] output shape (feature map)	P1	
[140,160,3] output shape (thumbnail)	P1	
USB 2.0 PHY, controller	P1	

Execution Plan



NV-IE100 top level diagram

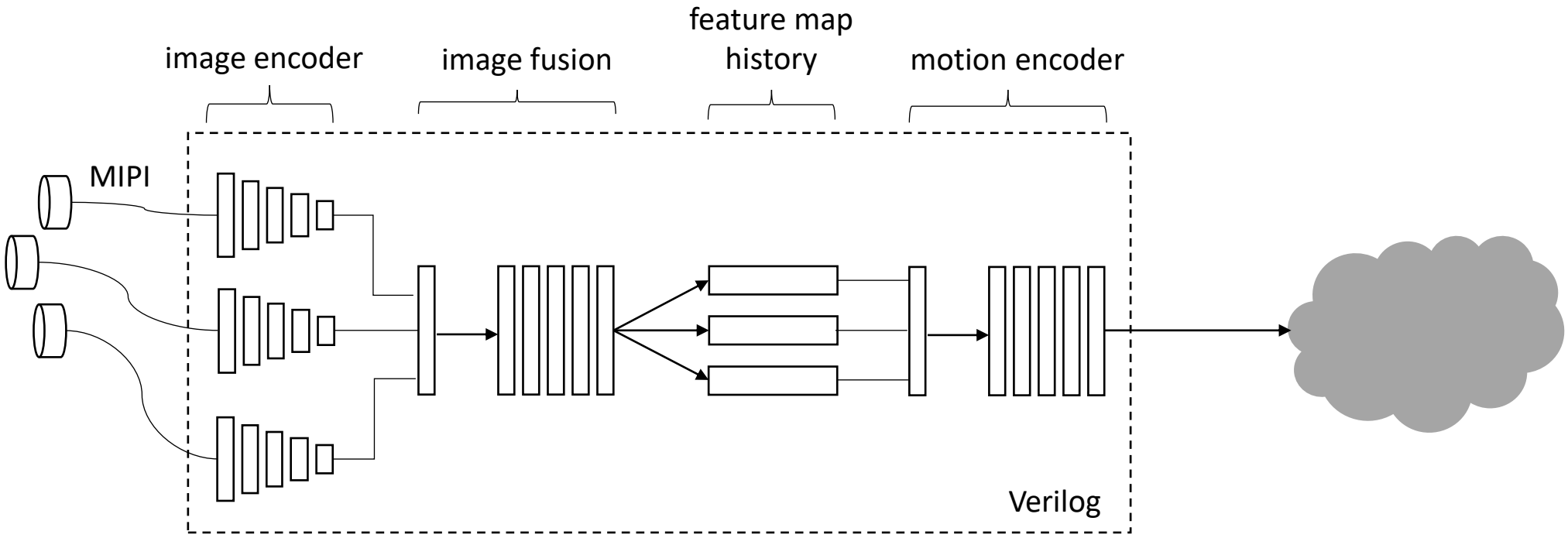
Block	Owner
MIPI	
USB	
RGB	
CNN	
I2C	



Execution Planning

Function	Tasks	Lead
CNN compiler	add level of hierarchy for inferred SRAM add SRAM option to replace weight ROM (SPI config) enhancements and fixes as needed (DAG,TFLite metadata,quantization)	
RTL design	MIPI CSI-2 RX with DPHY USB PHY and controller CNN top input/output interfaces and FIFO RGB image rescaler and FIFO I2C controller for image sensor configuration	
Verification (simulation)	End to end unit tests, pixels to predictions Also correlate predictions with TFLite reference interpreter	
Physical implementation	Standard ASIC flow with 500MHz core clock, 16nm libraries Run initial flat P&R experiment on compiled CNN with 18 layers	
Tools / Automation	Automate SRAM physical memory compiler for arbitrary size CNN instances	
Verification (emulation)	Prototype full chip at 200MHz, partition across multiple FPGAs	
Reference board	Mira220 RGB + NV-IE100, compact fanless form factor, USB powered	
Applications/Models Documentation FAE Support	Mira220+ Opal Kelly Artix US+ data capture and demo Build reference apps/models e.g facial keypoint detection, image super resolution, support HuggingFace model community	

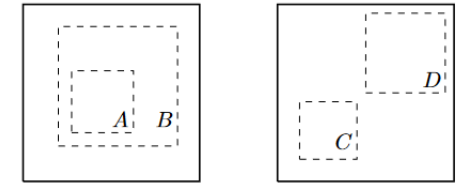
Roadmap



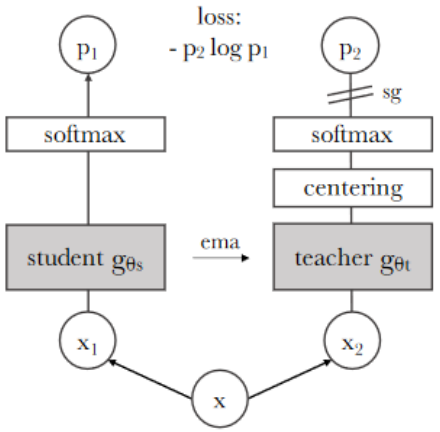
- integrate image fusion and motion encoder into the chip
- output feature maps will contain motion information which reduces the frame rate
- 100M IQ baseband samples/s for wireless sensing and communication e.g. MIMO OFDMA

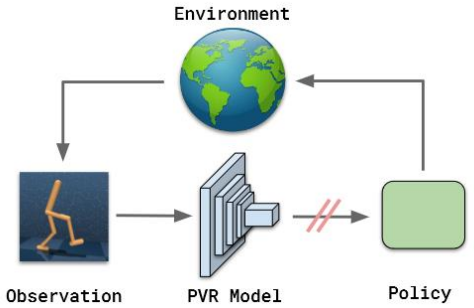
NeuVision Robotics Demo Application

Paper	Year	
A Simple Framework for Contrastive Learning of Visual Representations	2020	random crop and rescale
Bootstrap Your Own Latent A New Approach to Self-Supervised Learning	2020	positive pairs only
Emerging Properties in Self-Supervised Vision Transformers	2021	our baseline method (DINO)
The (Un)Surprising Effectiveness of Pre-Trained Vision Models for Control	2022	robot demo



(a) Global and local views. (b) Adjacent views.





APPENDIX

Risks

Risk	Sev	Mitigation
negative effects of integer quantization		POR use TFLite signed integer 8b weights, 16b activations Straightforward to switch to 8b/16b floating point as a fallback
no batch normalization or skip connections		POR use RELU activations with small learning rate Straightforward to support SELU using 64Kx16 SRAM LUT as a fallback
MIPI burstiness due to vertical blanking		CNN throughput is specified as a row time, not frame time

MIPI SoC platforms exceed the cost of the image sensor

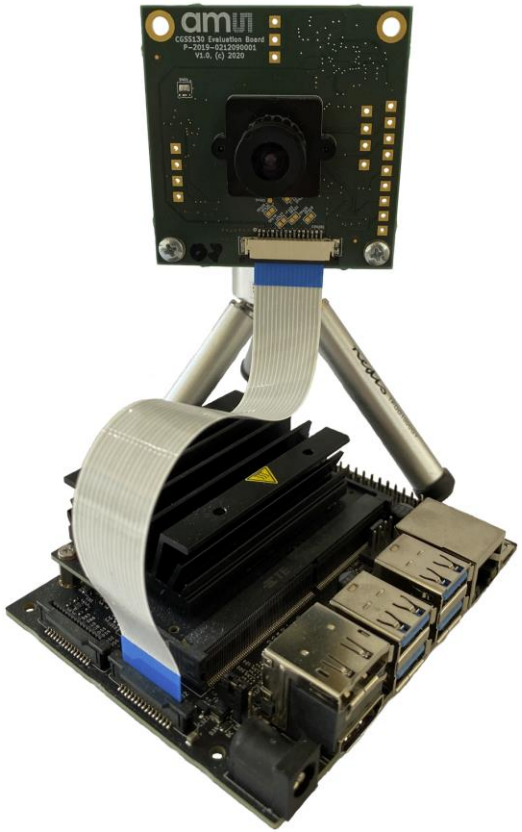


Image Encoder Product Options

NeuVision Product	Model Architecture	Model Weights	Scalability
IE-FPGA	reconfigurable	reconfigurable	<1M weights, <1 TOP, chiplets?
IE-100-RAM	**fixed VGG18**	**reconfigurable (RAM)**	**8M weights, 2 TOP**
IE-100-ROM	fixed VGG18	fixed (ROM)	8M weights, 2 TOP

- Image Encoder performs the initial convolutional pyramid using a VGG-like architecture.
- The proposed 8M weight VGG18 architecture is a strong foundation vision model with broad application.
- The image encoder also performs a valuable compression function.
- Weights are 8 bits, so 64Mb SRAM (weights) + 20Mb SRAM (row buffers) + standard cell logic

proposed VGG-like CNN

uses conv2d layer only

downsample using stride=2

padding=valid

activation nonlinearity=RELU, last layer=None

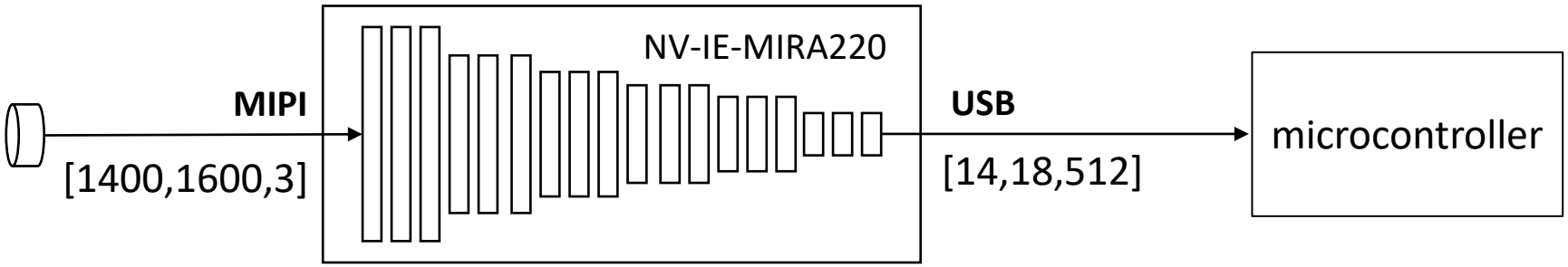
8 bit integer weights, 16 bit integer activations

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 1400, 1600, 3)]	0
conv2d (Conv2D)	(None, 1398, 1598, 16)	448
conv2d_1 (Conv2D)	(None, 698, 798, 16)	2320
conv2d_2 (Conv2D)	(None, 696, 796, 16)	2320
conv2d_3 (Conv2D)	(None, 694, 794, 32)	4640
conv2d_4 (Conv2D)	(None, 346, 396, 32)	9248
conv2d_5 (Conv2D)	(None, 344, 394, 32)	9248
conv2d_6 (Conv2D)	(None, 342, 392, 64)	18496
conv2d_7 (Conv2D)	(None, 170, 195, 64)	36928
conv2d_8 (Conv2D)	(None, 168, 193, 64)	36928
conv2d_9 (Conv2D)	(None, 166, 191, 128)	73856
conv2d_10 (Conv2D)	(None, 82, 95, 128)	147584
conv2d_11 (Conv2D)	(None, 80, 93, 128)	147584
conv2d_12 (Conv2D)	(None, 78, 91, 256)	295168
conv2d_13 (Conv2D)	(None, 38, 45, 256)	590080
conv2d_14 (Conv2D)	(None, 36, 43, 256)	590080
conv2d_15 (Conv2D)	(None, 34, 41, 512)	1180160
conv2d_16 (Conv2D)	(None, 16, 20, 512)	2359808
conv2d_17 (Conv2D)	(None, 14, 18, 512)	2359808

Total params: 7,864,704
Trainable params: 7,864,704
Non-trainable params: 0

Proposed Image Encoder

NV-IE-MIRA220
500MHz clock
7500 integer MAC units
64Mb SRAM (weights)
20Mb SRAM (rowbuf)



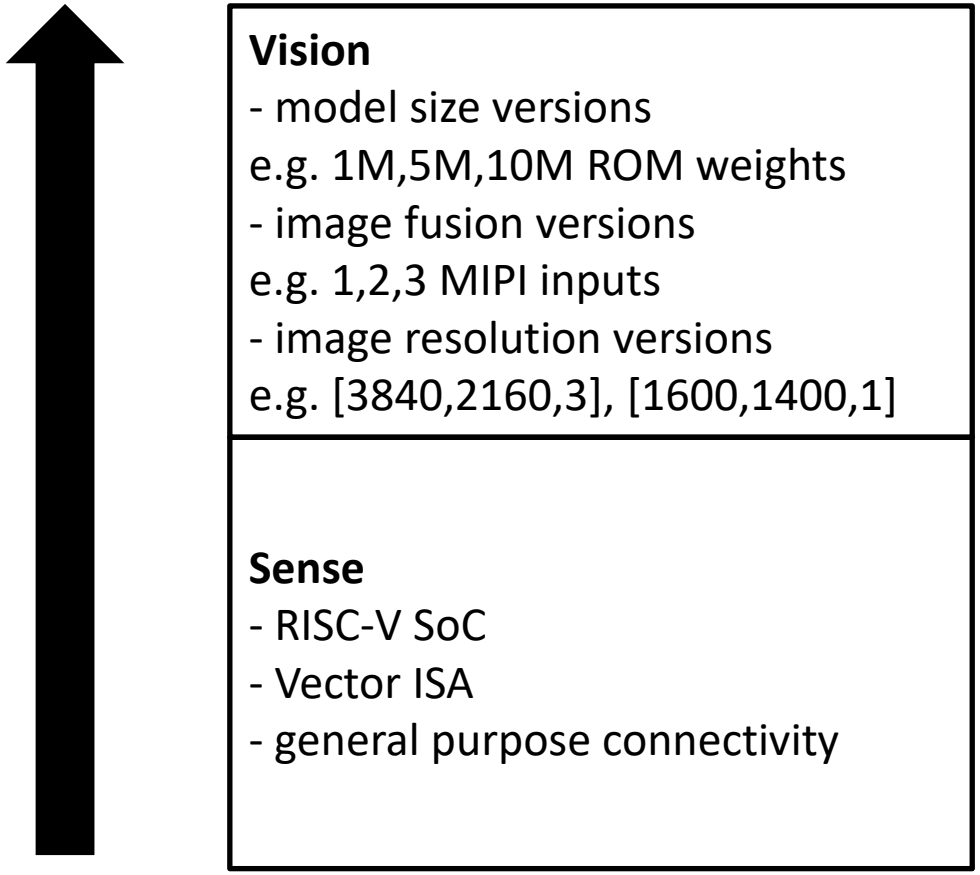
scalable foundation vision CNN with 8M parameters
reconfigurable SRAM-based weights

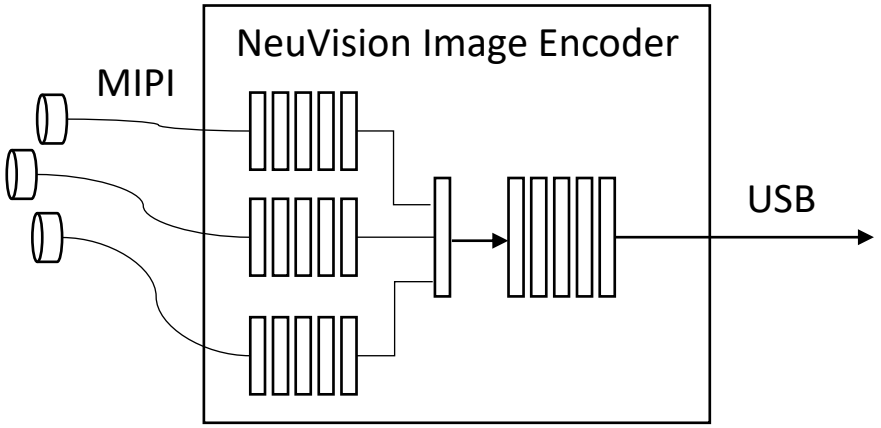
MIPI
1600x1400 RGB pixels
12 bits/pixel
110 frames/s
= 8.87 Gb/s

39X compression

USB
[14,18,512] features
16 bits/feature
110 frames/s
= 0.227 Gb/s

Smart Sensor Silicon Segmentation

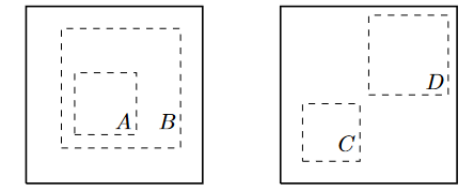




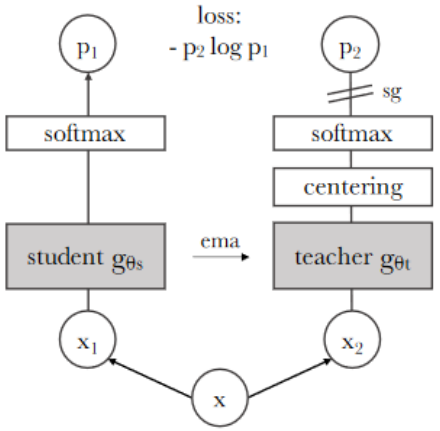
1. collect unlabeled dataset
    a. synchronized frame capture from multiple image sensors
    b. synchronized metadata e.g. robot servo angles
2. self supervised pretraining using unlabeled dataset
    a. BYOL, DINO
3. few shot learning using pretrained vision model
    a. imitation learning for robotics
    b. classifier for object detection
    c. image decoder for object segmentation
    d. patch embeddings for multimodal LLM

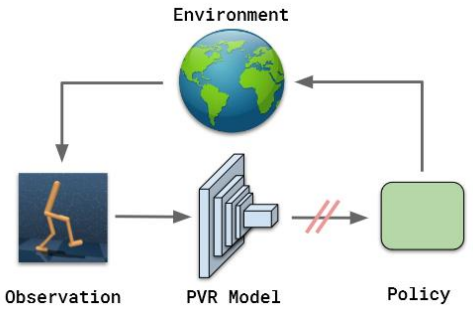
4. data sheet for {100K,1M,5M,20M parameter VGG-like CNN}, {FPGA, ASIC}
    a. area, ROM area, clock frequency

Paper	Year	
A Simple Framework for Contrastive Learning of Visual Representations	2020	random crop and rescale
Bootstrap Your Own Latent A New Approach to Self-Supervised Learning	2020	positive pairs only
Emerging Properties in Self-Supervised Vision Transformers	2021	our baseline method (DINO)
DINOv2: Learning Robust Visual Features without Supervision	2023	
Masked Autoencoders As Spatiotemporal Learners	2022	
The (Un)Surprising Effectiveness of Pre-Trained Vision Models for Control	2022	robot demo



(a) Global and local views. (b) Adjacent views.

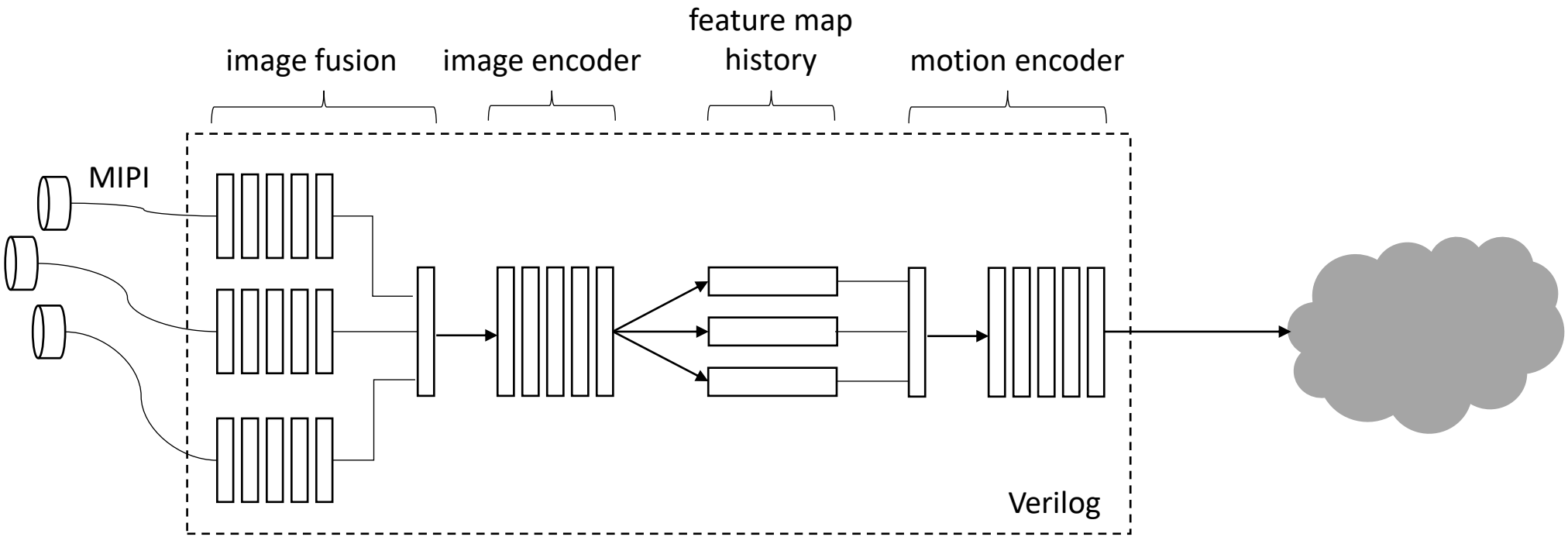


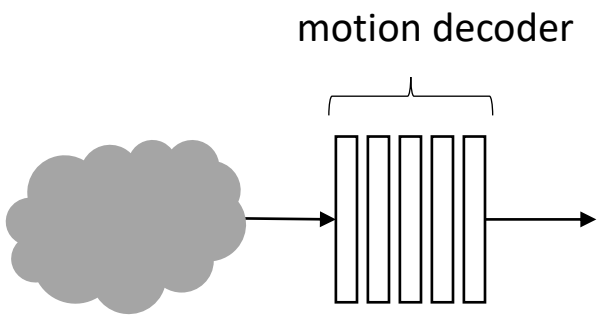


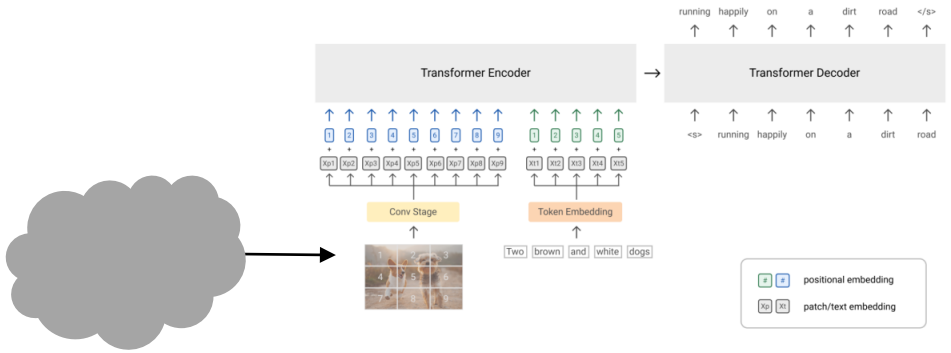
[1120,1280,3]
@110 FPS

1000X compression

[16,19,256]
@10 FPS







patch embeddings for multimodal LLM, e.g. SIMVLM

image sensors

image fusion

image encoder

MIPI

[1120,1280,3]
@110 FPS

Verilog
CNN

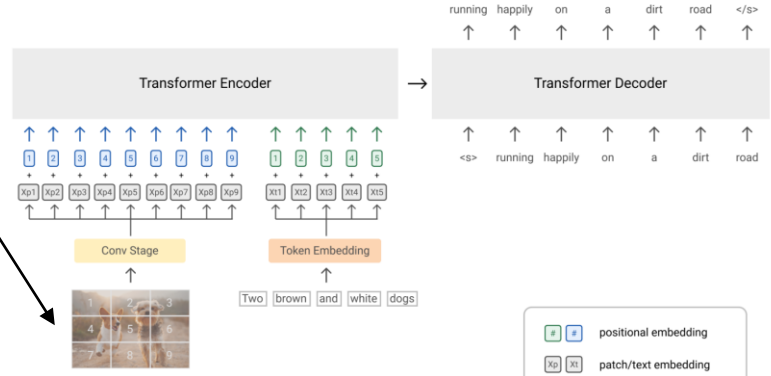
[16,19,256]
@110 FPS

100X compression

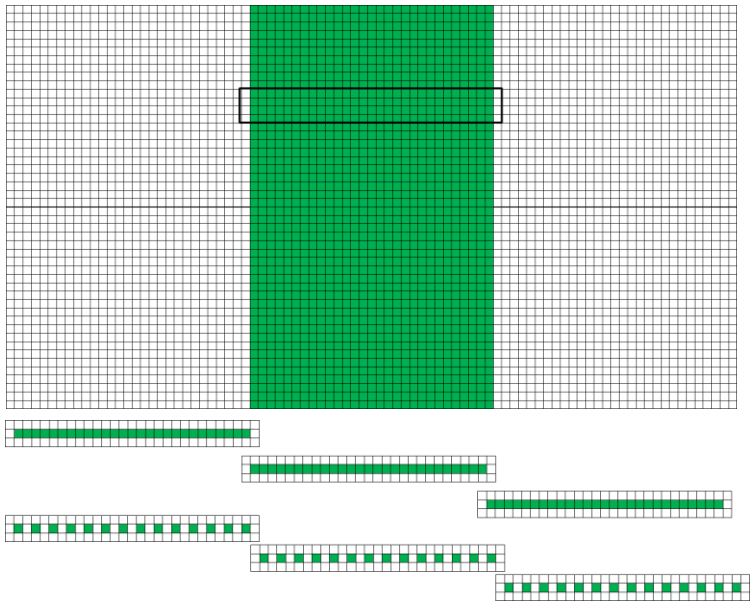
image decoder

depth map
object detection
semantic heatmap

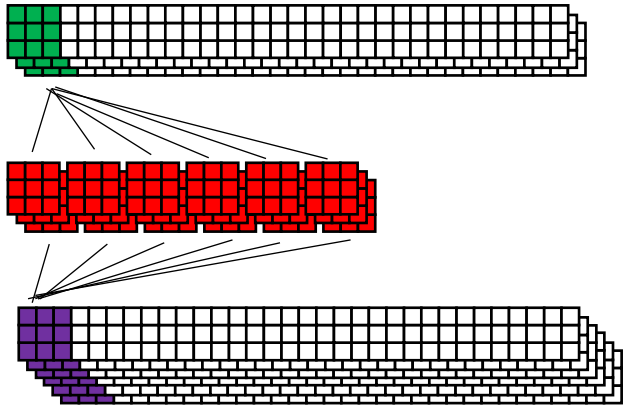
multimodal LLM, e.g. SIMVLM



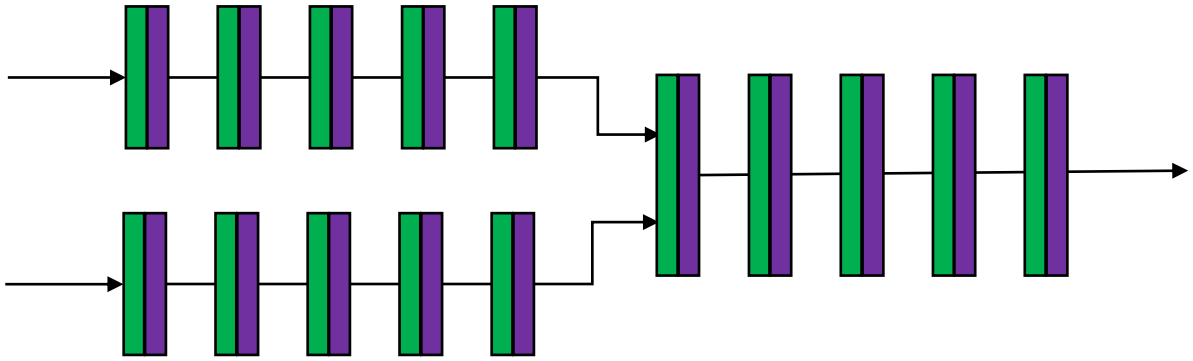
patch embeddings



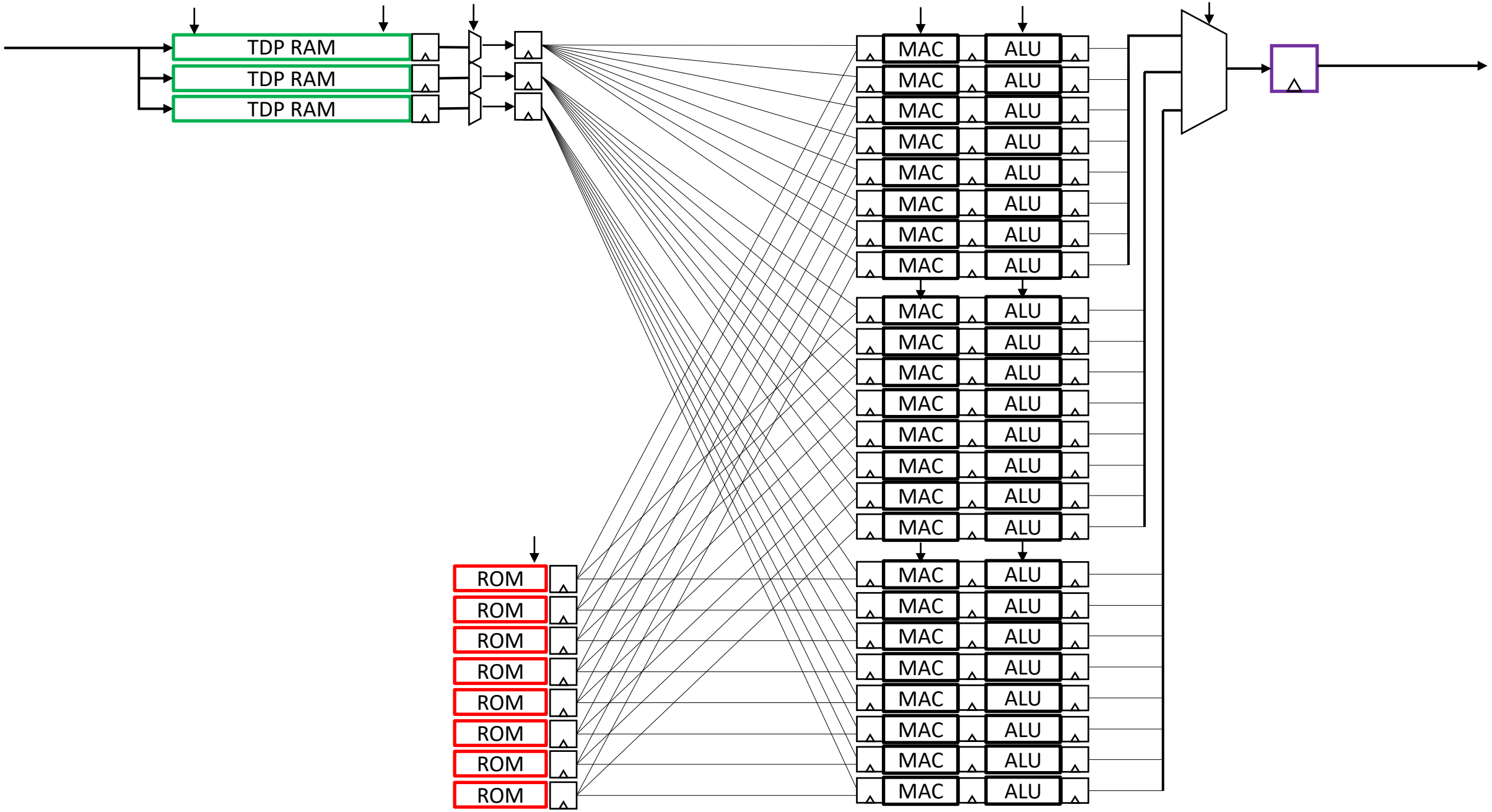
A) parallel vertical stripes

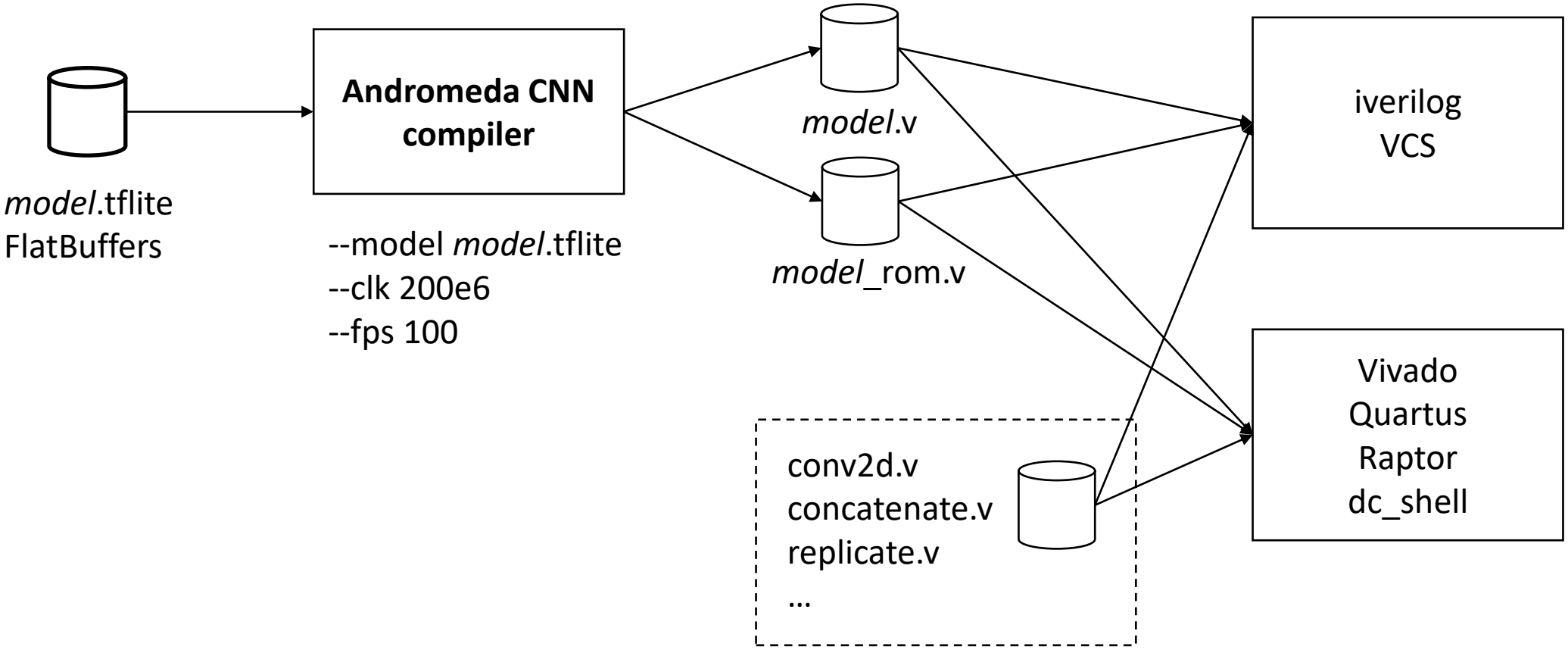


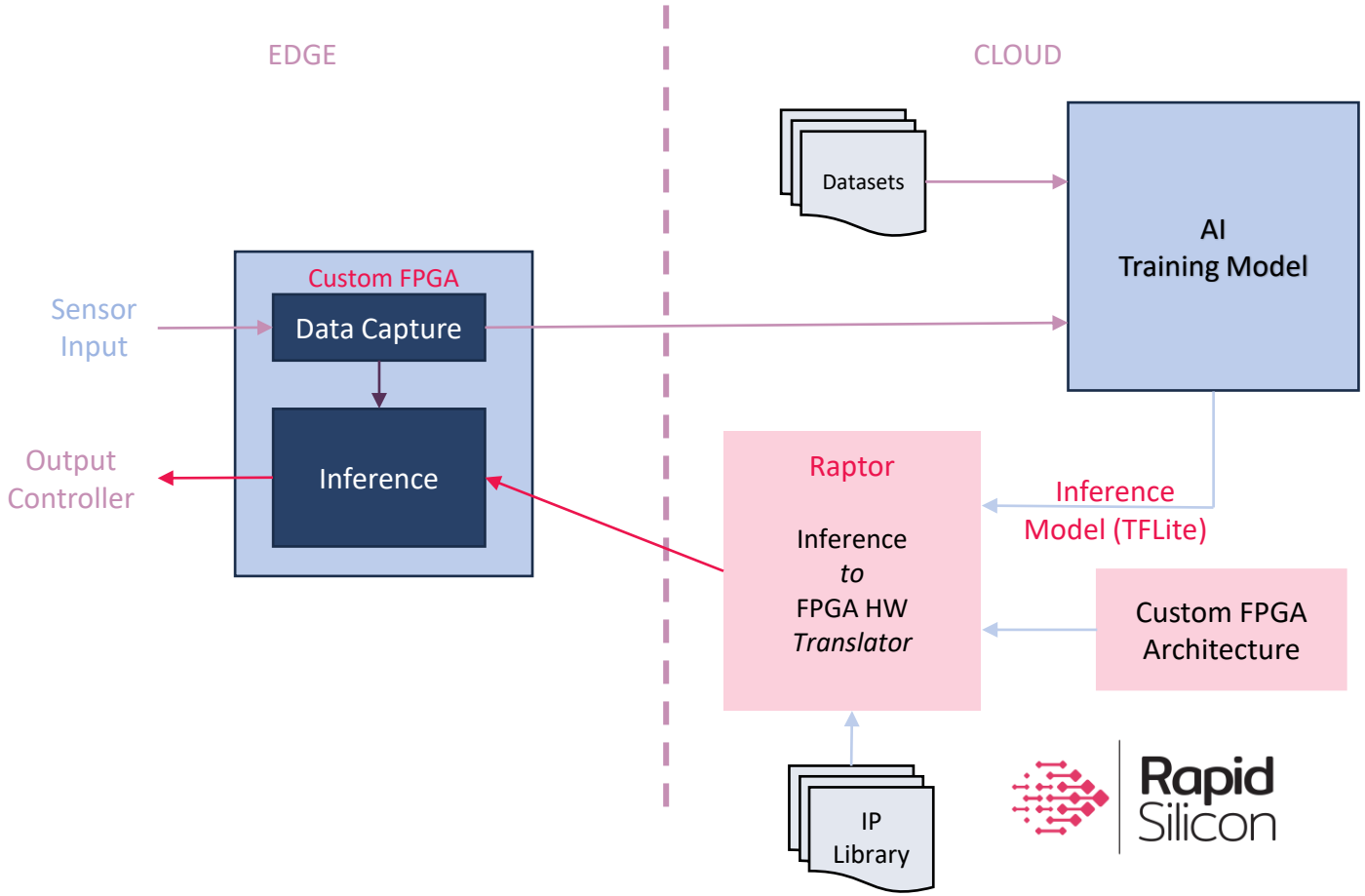
B) parallel output channels



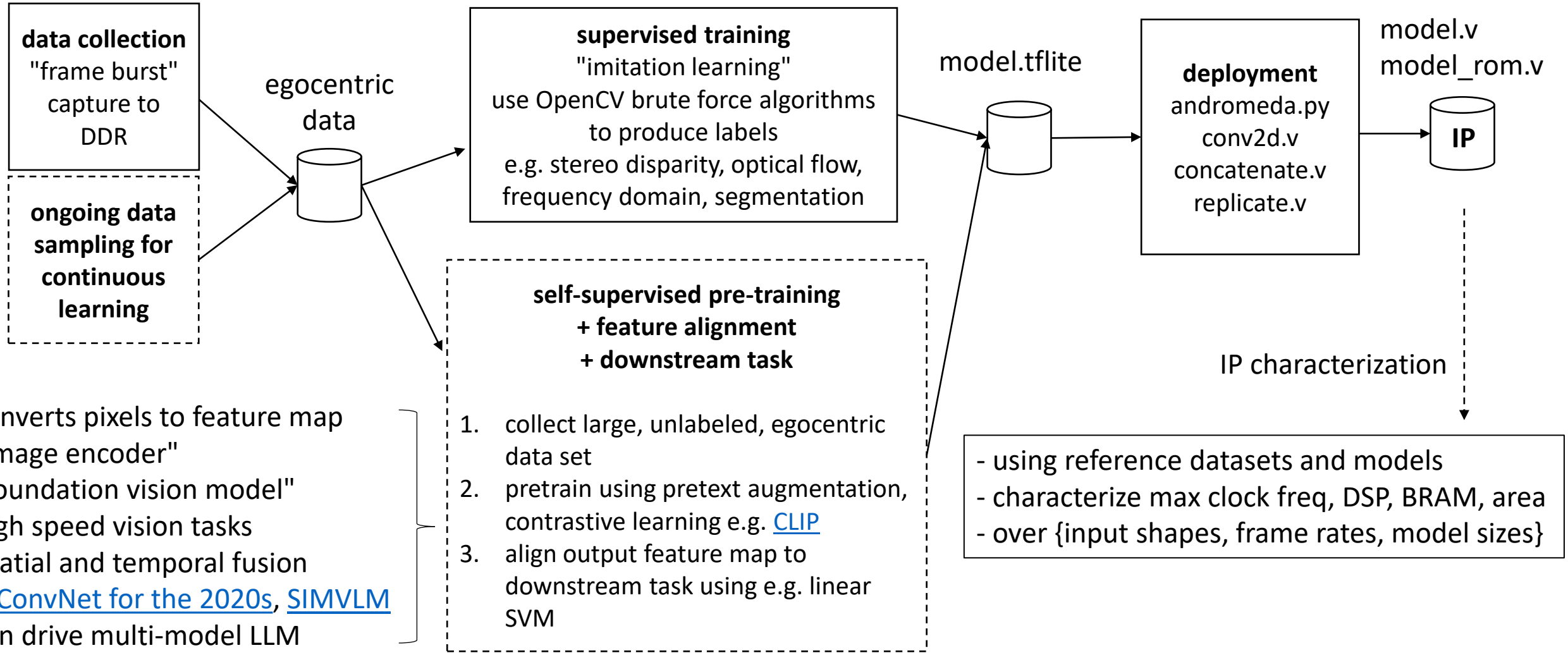
C) parallel layers


TDP RAM
TDP RAM
TDP RAM
ROM
ROM
ROM
ROM
ROM
ROM
ROM
ROM
MAC
ALU

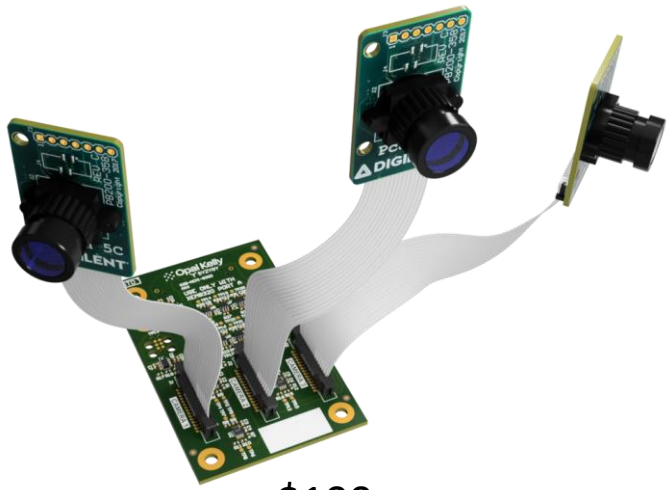

model.tflite
FlatBuffers
Andromeda CNN compiler
--model *model*.tflite
--clk 200e6
--fps 100
model.v
*model*_rom.v
conv2d.v
concatenate.v
replicate.v
…
iverilog
VCS
Vivado
Quartus
Raptor
dc_shell


EDGE
CLOUD
Datasets
AI
Training Model
Custom FPGA
Sensor
Input
Data Capture
Inference
Output
Controller
Raptor
Inference
to
FPGA HW
Translator
Inference
Model (TFLite)
Custom FPGA
Architecture
IP
Library
Rapid
Silicon

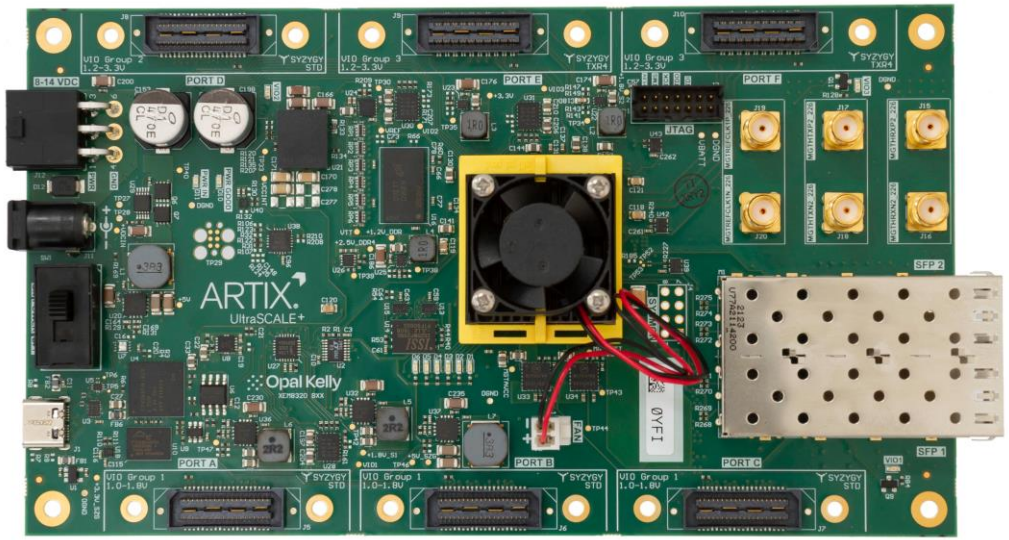
Real time vision IP flow

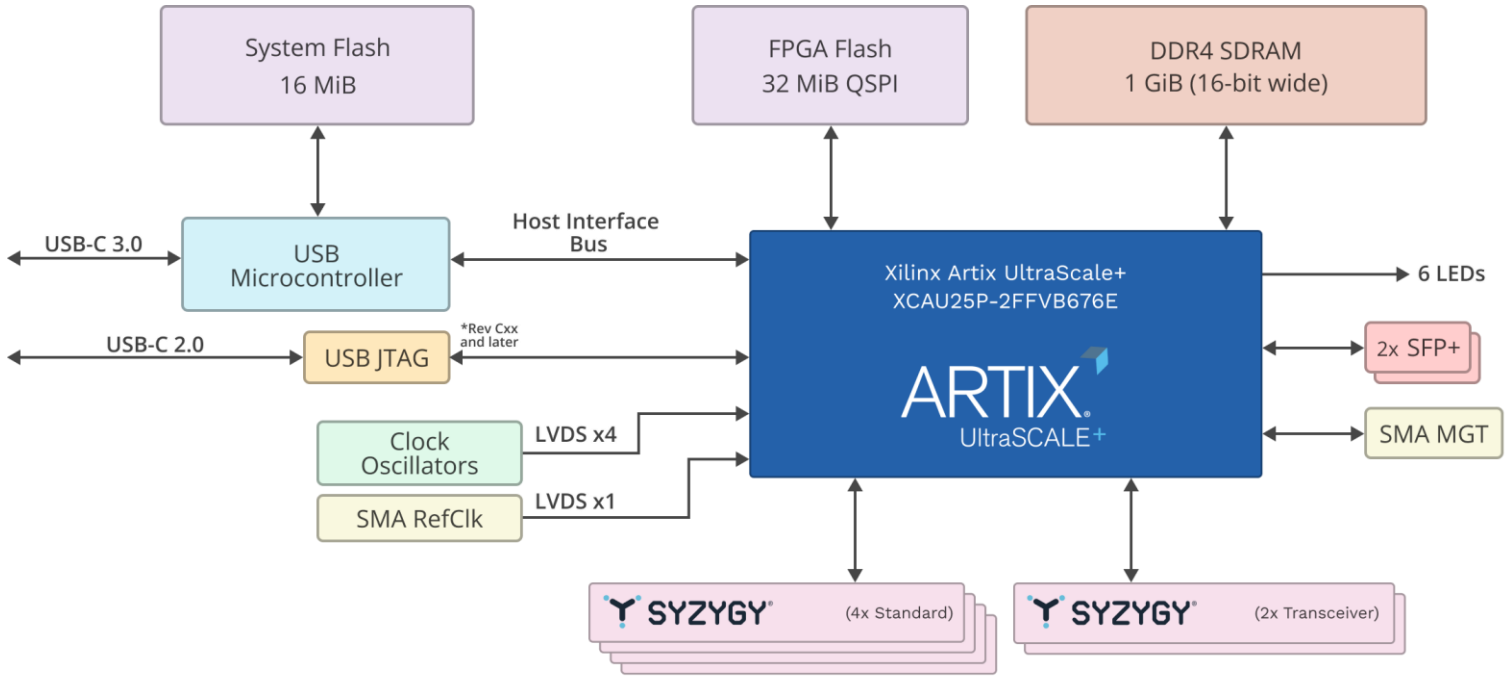


$1400

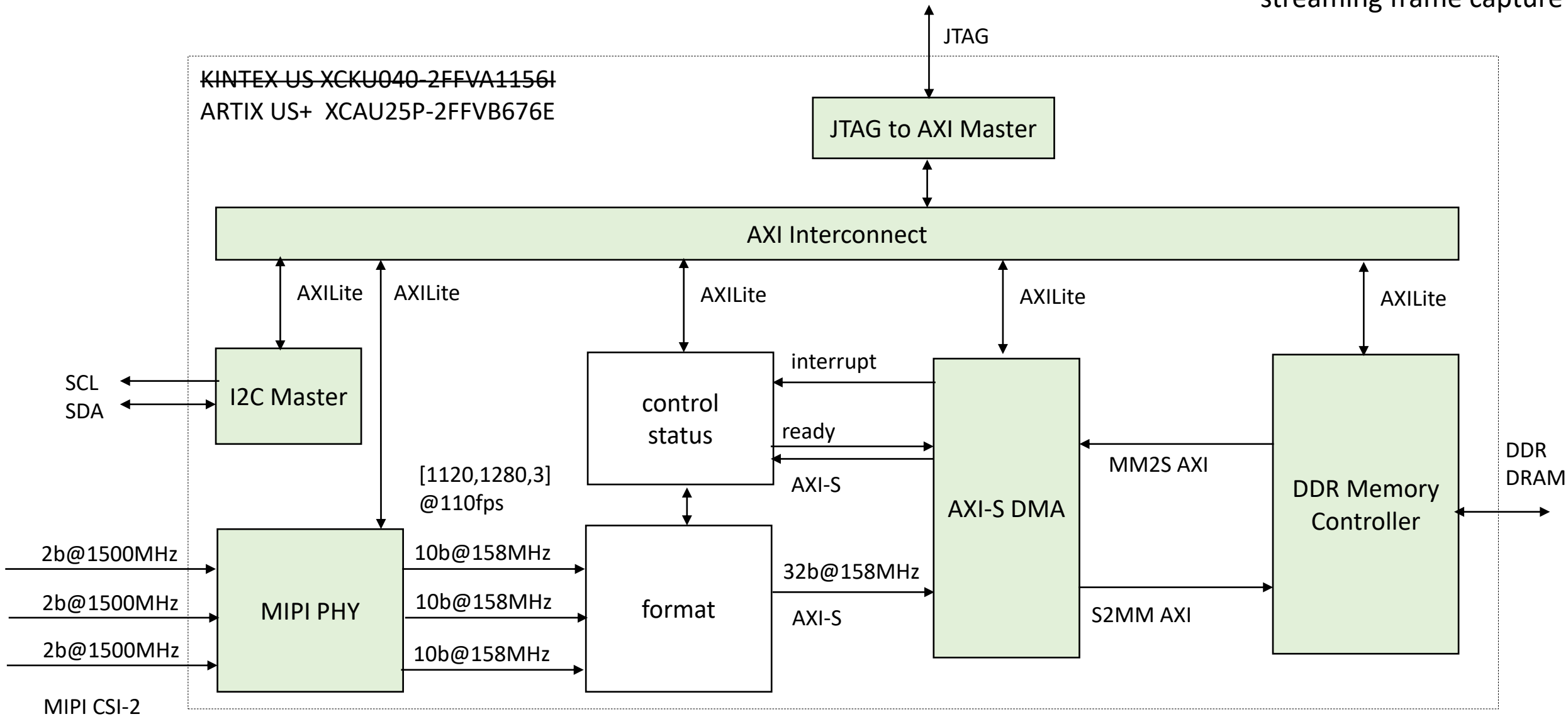


$100


ARTIX
UltraSCALE+
Opal Kelly


System Flash
16 MiB
FPGA Flash
32 MiB QSPI
DDR4 SDRAM
1 GiB (16-bit wide)
USB-C 3.0
USB Microcontroller
Host Interface Bus
USB-C 2.0
USB JTAG
*Rev Cxx and later
Clock Oscillators
LVDS x4
SMA RefClk
LVDS x1
Xilinx Artix UltraScale+
XCAU25P-2FFVB676E
ARTIX
UltraSCALE+
6 LEDs
2x SFP+
SMA MGT
SYZYGY (4x Standard)
SYZYGY (2x Transceiver)

streaming frame capture

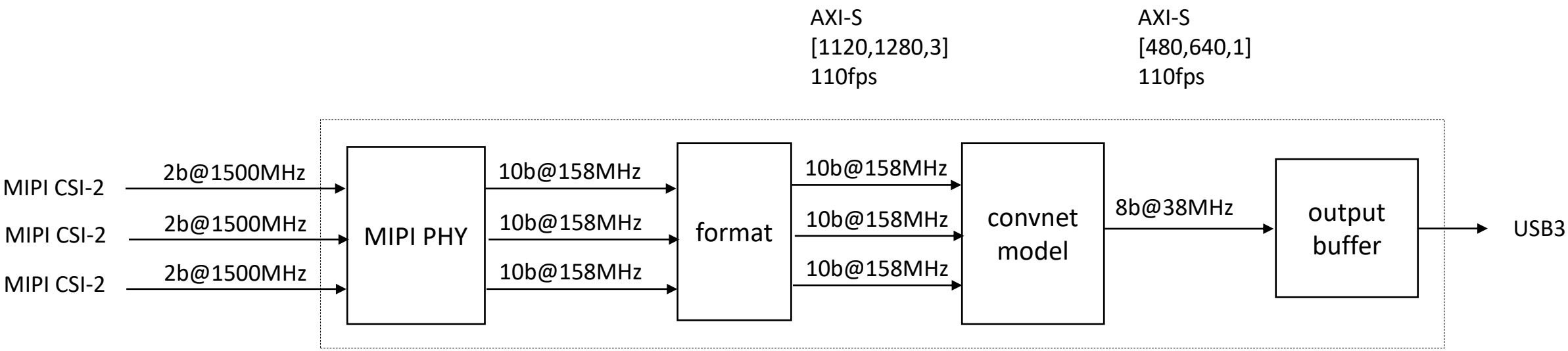


https://www.xilinx.com/products/intellectual-property/jtag_to_axi_master.html
https://docs.xilinx.com/r/en-US/pg232-mipi-csi2-rx/MIPI-CSI-2-Receiver-Subsystem-Product-Guide
https://docs.xilinx.com/v/u/en-US/pg150-ultrascale-memory-ip
https://docs.xilinx.com/r/en-US/pg021_axi_dma/AXI-DMA-Register-Address-Map?section=XREF_38966_AXI_DMA_Simple_DMA
https://docs.xilinx.com/v/u/en-US/ds768_axi_interconnect

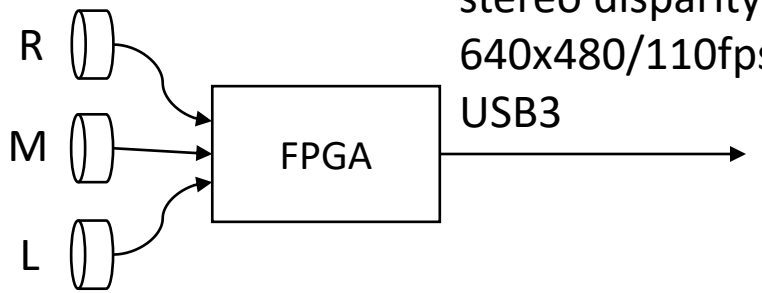
EP address	Type	Bits	Description
0x00	wire_in	31:0	AWADDR
0x01		31:0	WDATA
0x02		31:0	ARADDR
0x03		7:0	AWLEN
0x03		10:8	AWSIZE
0x03		12:11	AWBURST
0x03		13	AWLOCK
0x03		17:14	AWCACHE
0x03		20:18	AWPROT
0x03		24:21	AWQOS
0x04		3:0	WSTRB
0x04		4	WLAST
0x05		7:0	ARLEN
0x05		10:8	ARSIZE
0x05		12:11	ARBURST
0x05		13	ARLOCK
0x05		17:14	ARCACHE
0x05		20:18	ARPROT
0x05		24:21	ARQOS

EP address	Type	Bits	Description
0x40	trigger_in	0	AWVALID
0x40		1	WVALID
0x40		2	BREADY
0x40		3	ARVALID
0x40		4	RREADY
0x40		5	global_resetn

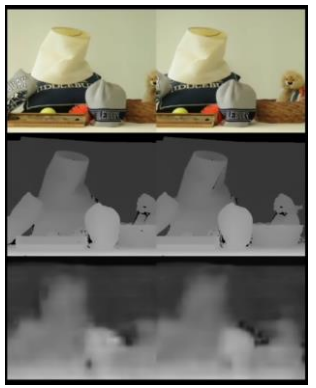
EP address	Type	Bits	Description
0x20	wire_out	0	AWVALID
0x20		1	WVALID
0x20		2	BVALID
0x20		3	ARVALID
0x20		4	RVALID
0x21		31:0	RDATA
0x22		1:0	BRESP
0x23		1:0	RRESP
0x23		2	RLAST


AXI-S
[1120,1280,3]
110fps
AXI-S
[480,640,1]
110fps
MIPI CSI-2
MIPI CSI-2
MIPI CSI-2
2b@1500MHz
2b@1500MHz
2b@1500MHz
MIPI PHY
10b@158MHz
10b@158MHz
10b@158MHz
format
10b@158MHz
10b@158MHz
10b@158MHz
convnet
model
8b@38MHz
output
buffer
USB3

Mira220, IMG.2/SPEED.1/ADC.2
1280x1120/110fps/10b
MIPI CSI-2



stereo disparity
640x480/110fps/8b
USB3



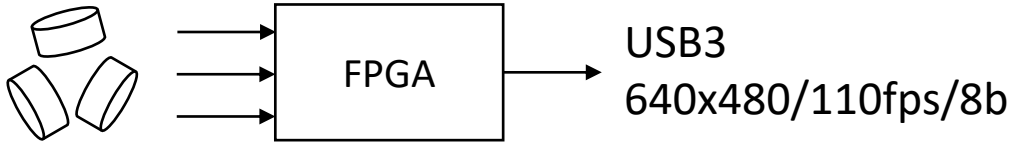
https://www.amazon.com/Lottery-Machine-Automatic-Electric-Colorful/dp/B0C1XGSZKQ/ref=sr_1_9?crid=1C4OQX8RFH0P4&keywords=lottery+ping+pong+ball+machine

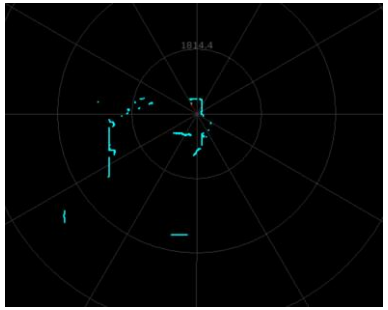
predict RPLidar 2-D LIDAR map from 3X Mira220 in 360 deg configuration (e.g. Roomba view)

demo concept

1280x1120/110fps/10b

Mira220, IMG.2/SPEED.1/ADC.2





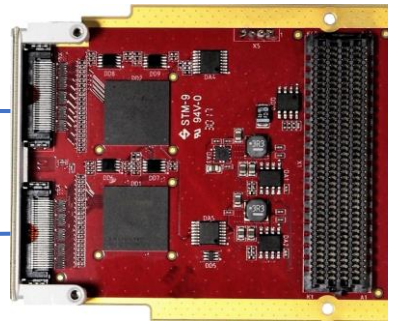
https://www.amazon.com/Lottery-Machine-Automatic-Electric-Colorful/dp/B0C1XGSZKQ/ref=sr_1_9?crid=1C4OQX8RFH0P4&keywords=lottery+ping+pong+ball+machine

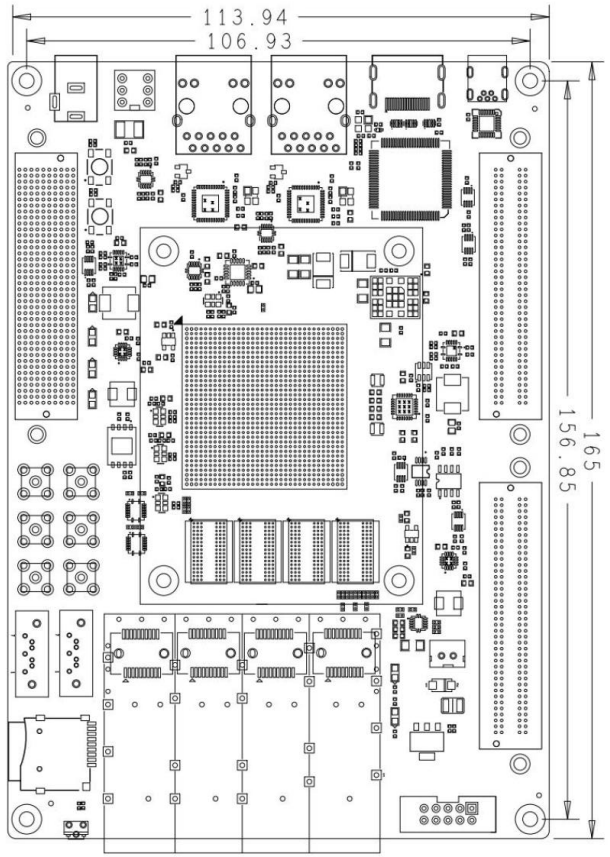
Mira-220

LVDS C+2D@1.5GHz

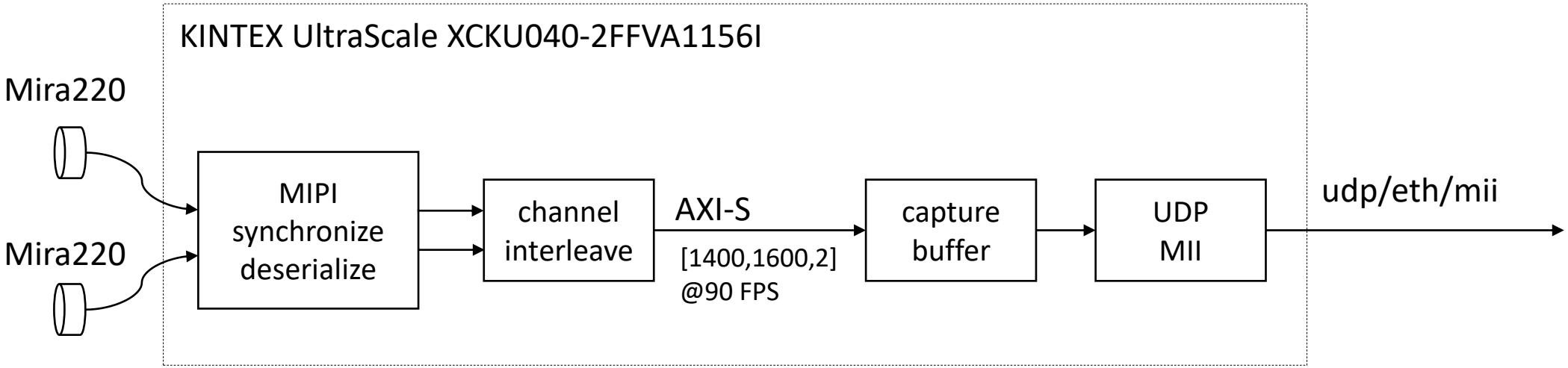
LVDS C+2D@1.5GHz

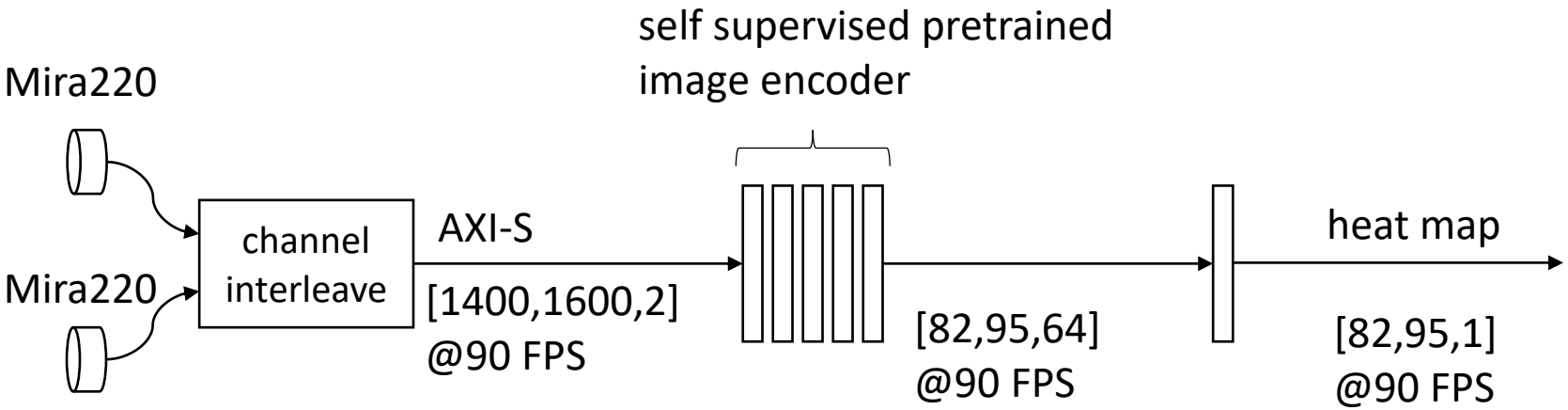
FMC-MIPI





https://docs.arducam.com/Nvidia-Jetson-Camera/type-and-pinout-mipi-csi-2/


KINTEX UltraScale XCKU040-2FFVA1156I
Mira220
Mira220
MIPI
synchronize
deserialize
channel
interleave
AXI-S
[1400,1600,2]
@90 FPS
capture
buffer
UDP
MII
udp/eth/mii



pretrain using self-supervised pretext tasks on unlabeled data

1. LR eye swap, predict LR vs RL
2. LR time shift, predict L[t]R[t] vs L[t-1]R[t] vs L[t]R[t-1]

Sensing platform

Silicon vision and sensing platform for next generation robotics

- target next generation robotics based on multimodal LLM technology
- the image encoder socket is the first stage in a multimodal LLM e.g. CLIP
- foundation/backbone vision models encode raw pixels into a compressed feature map
- domain specific FPGA enables low latency "pixels to features" at sensor speed
- extends to 360-degree vision, hyperspectral, video, tactile sensors, audio, RF

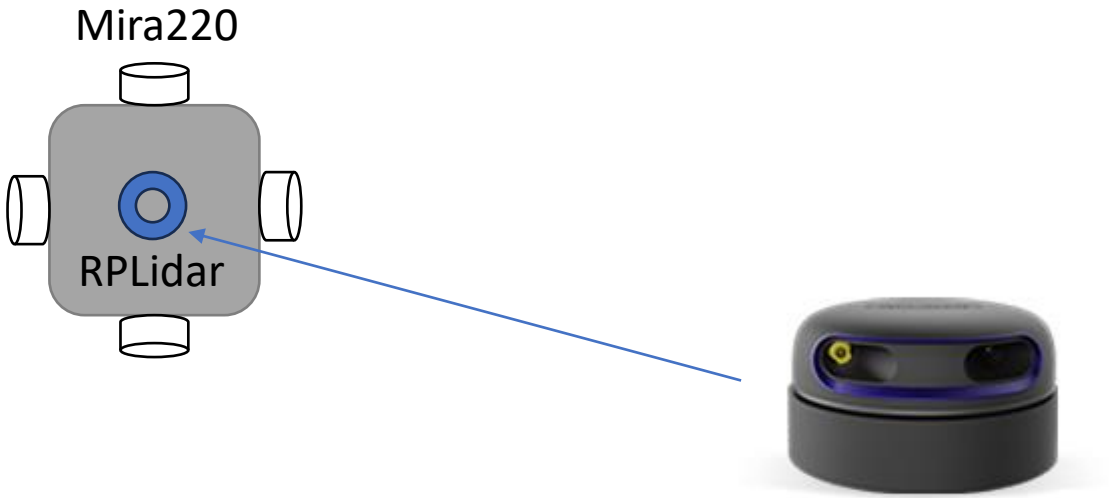
Strategy

- Open software
- Optimized FPGA silicon initially targeting the emerging **image encoder socket**
- Scale using chiplets
- Partner with image sensor, robotics ecosystems

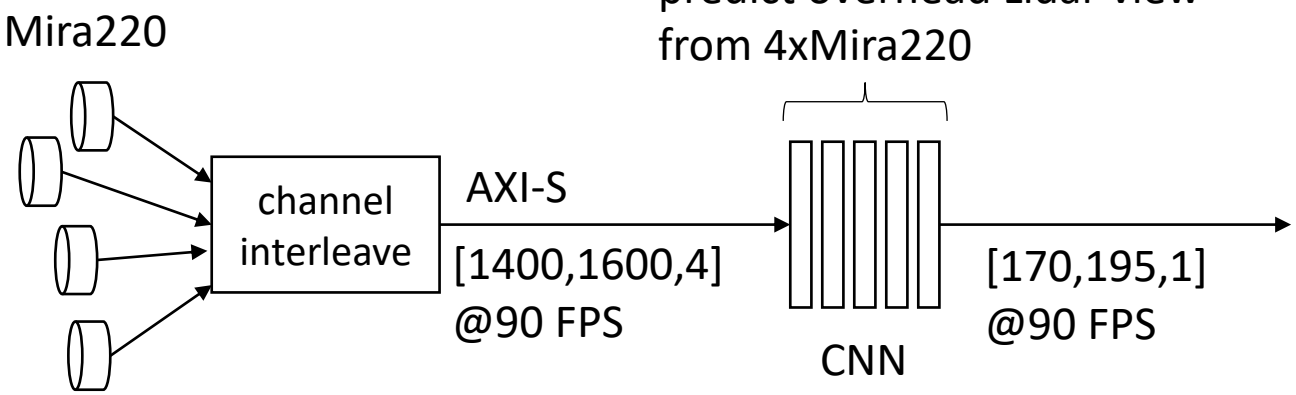
Execution

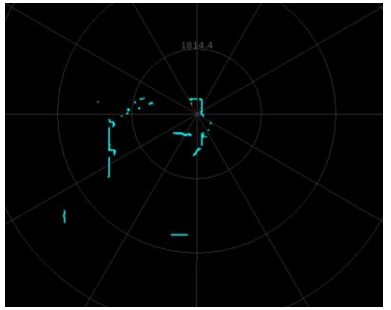
- TF to Verilog demo, real time CNN inference at high resolution and frame rate
- demonstrate vision model using self supervised pretraining on egocentric image data

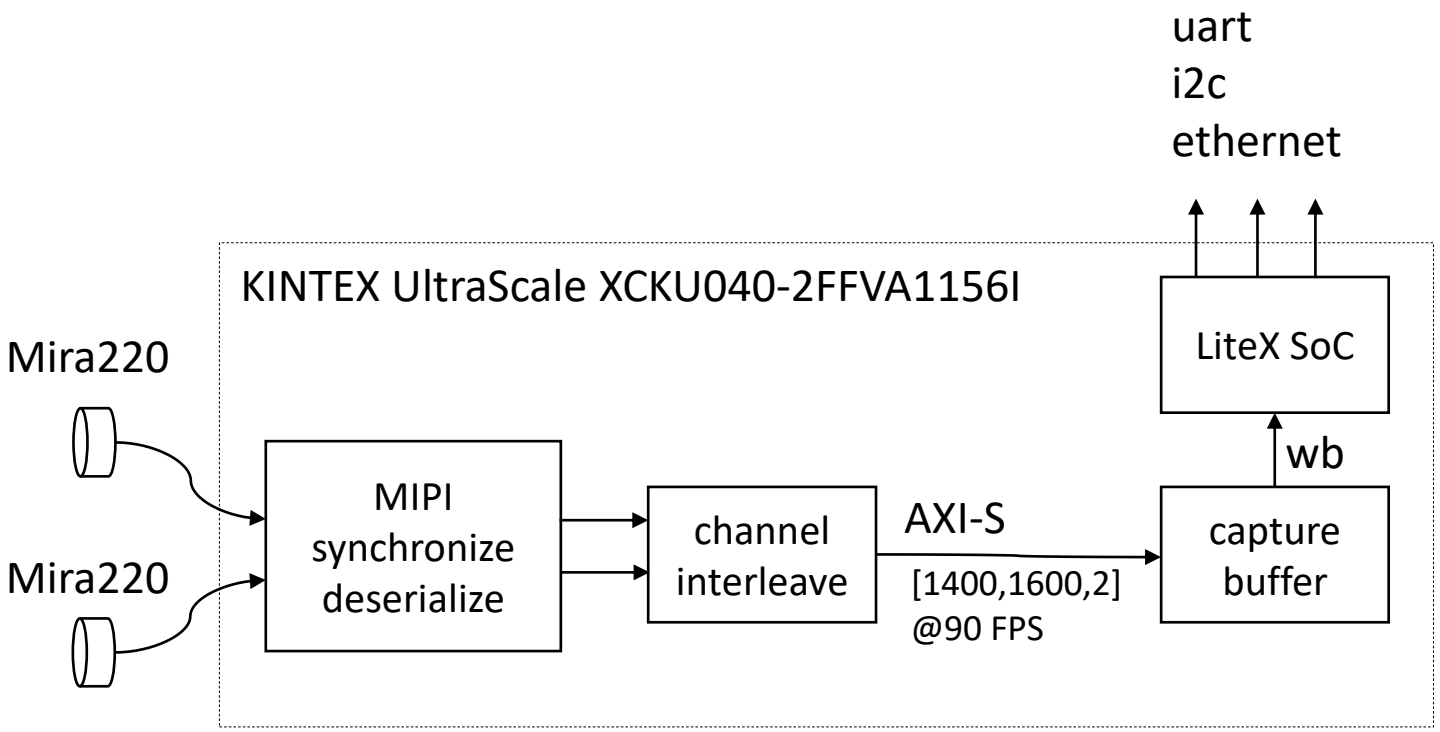
https://www.slamtec.ai/#

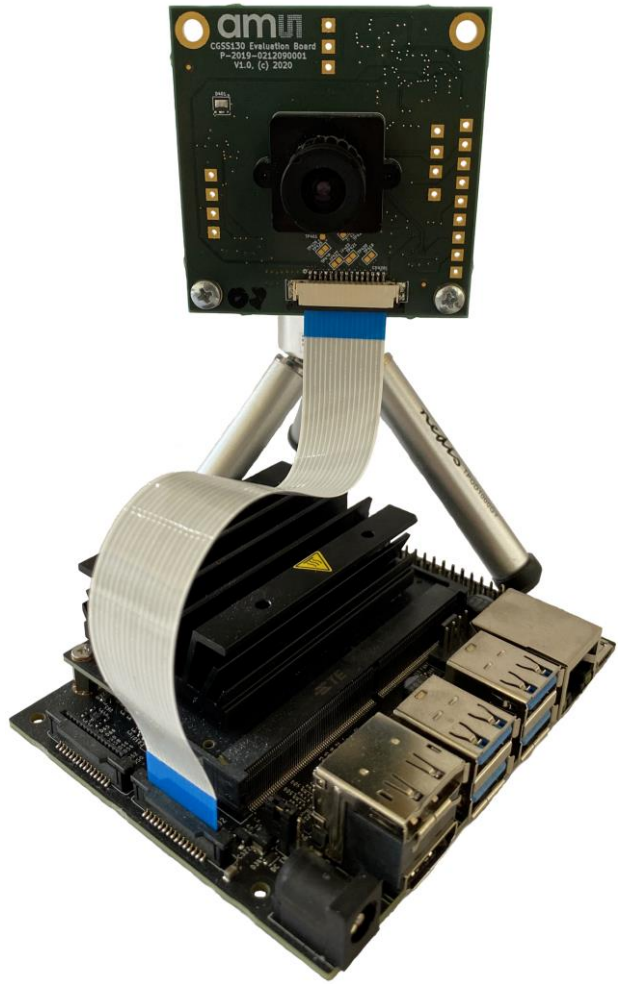


supervised training
predict overhead Lidar view
from 4xMira220






uart
i2c
ethernet
KINTEX UltraScale XCKU040-2FFVA1156I
LiteX SoC
wb
Mira220
Mira220
MIPI
synchronize
deserialize
channel
interleave
AXI-S
[1400,1600,2]
@90 FPS
capture
buffer


CGSS130 Evaluation Board
P-2019-0212090001
V1.0, (c) 2020

https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=142&No=1295#contents
https://www.xilinx.com/products/boards-and-kits/1-so55i9.html

FPGA Compute Platform

Vision

- FPGA as a flexible compute platform
  - embedded AI, edge and cloud applications, GPU alternative

Strategy

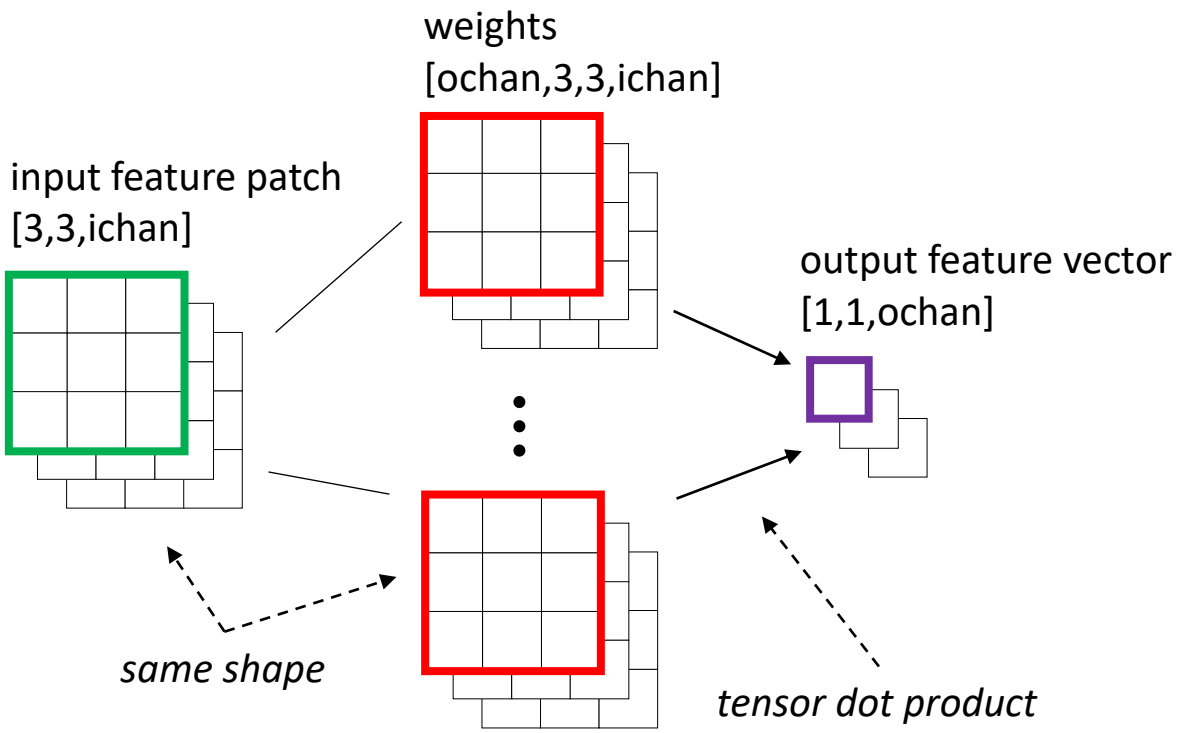
- GPT-friendly open IP library
- GPT-automated development flow
  - Raptor, HW/SW co-design (CUDA, RTOS)
- GPT-assisted dataflow algorithm implementation
  - graph based computation for deep models (CNN, Transformer)
    - TF to Verilog
  - traditional DSP pipelines (vision, RF)
    - MATLAB to Verilog
  - multiple architectures (training accelerators, embedded streaming inference)
- Compute-optimized FPGA silicon product line

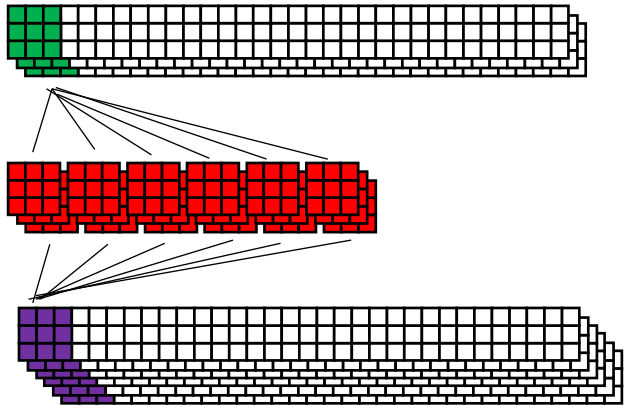
Execution

- "hello world" TF to Verilog demo (embedded real time edge inference)

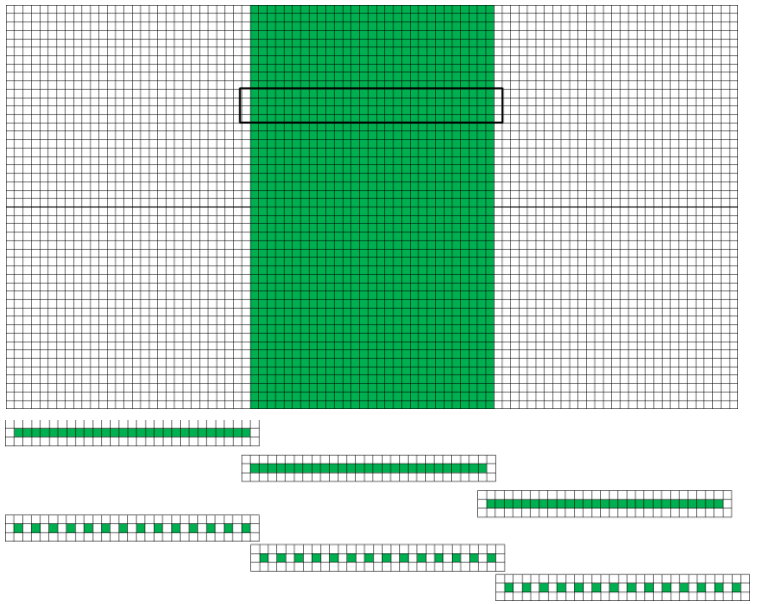
1. move parameters on-chip, remove IO bottleneck, large aggregate bandwidth with many small RAMs
2. ~~conv2d can be streamed using a row buffer, i.e. 'C' order input tensors (framed by TLAST) can be evaluated in order to produce an output feature map also in 'C' order -> reduced data storage, e.g. able to process 3840x2160 frames in raster order~~
3. conv2d can be pipelined layer by layer

Parallel computation of CNN dot products





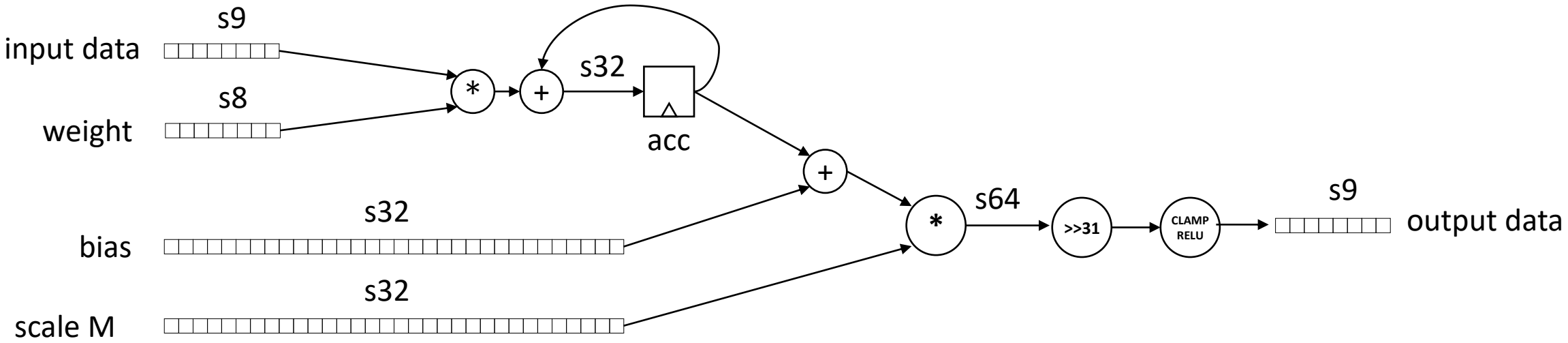
parallel ochan



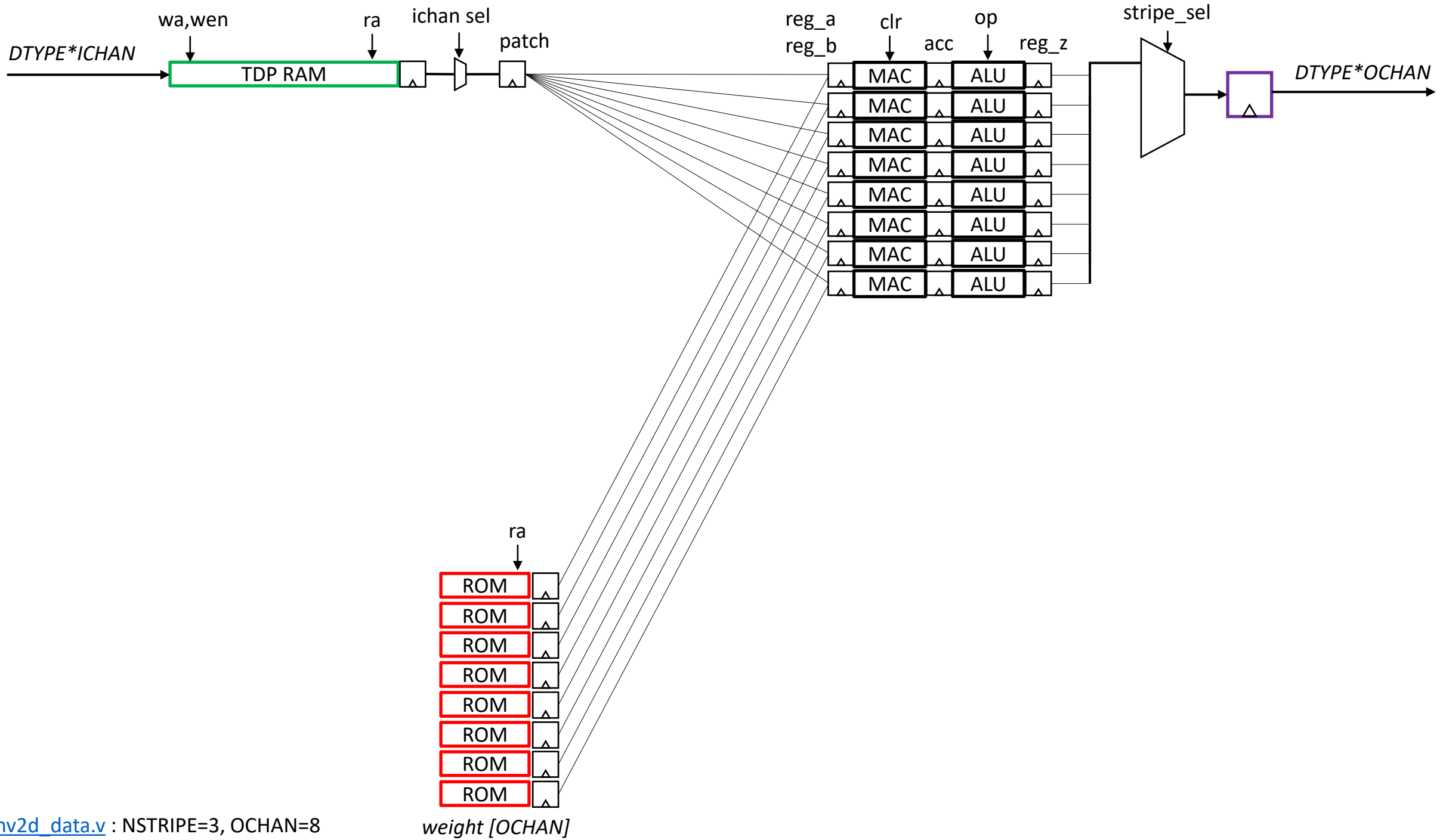
parallel nstripe

Backup

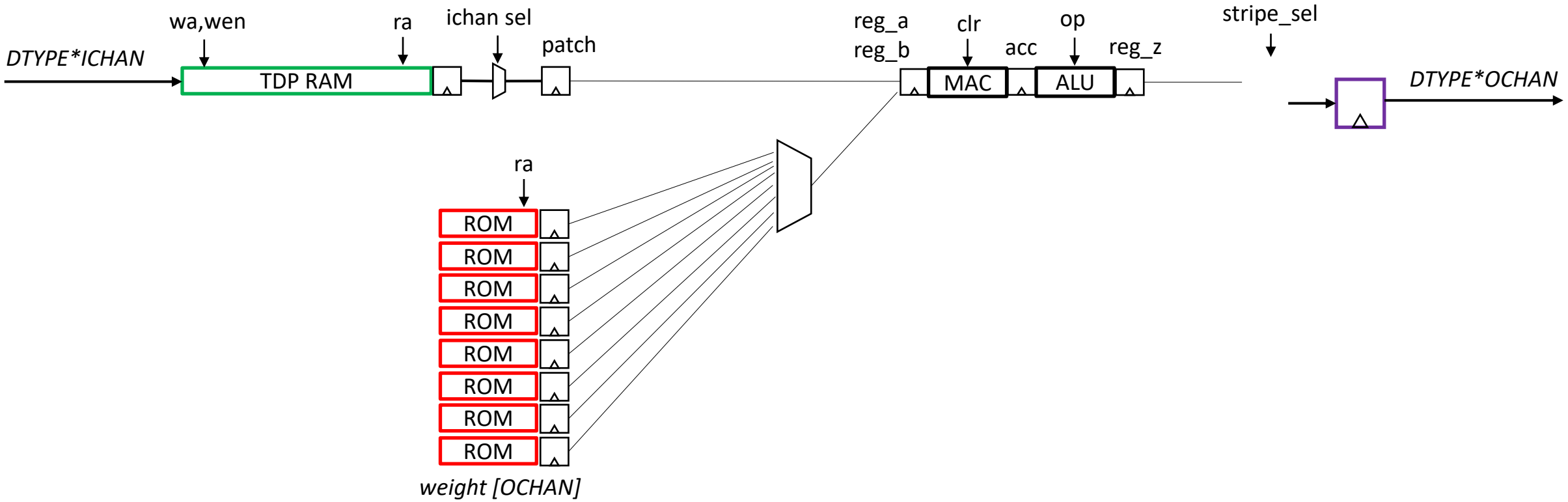
TFLite int8 quantization



1: compute dot product (s9*s9+s32)
2: add bias (s32+s32)
3. multiply by scale (s32*s32)
4. shift right >>31
5. clamp to s9 bits and RELU



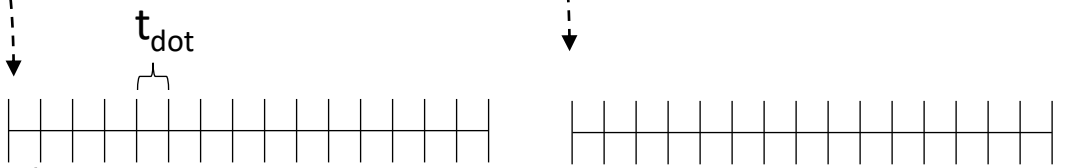
conv2d_data.v : NSTRIPE=3, OCHAN=8



conv2d_data.v : NSTRIPE=3, OCHAN=8

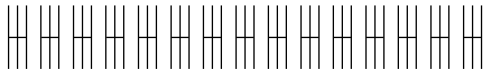
fill row of padded stripes from previous layer

compute NSTRIPE*OCHAN parallel dot products, quant scale, RELU
stride through input row data



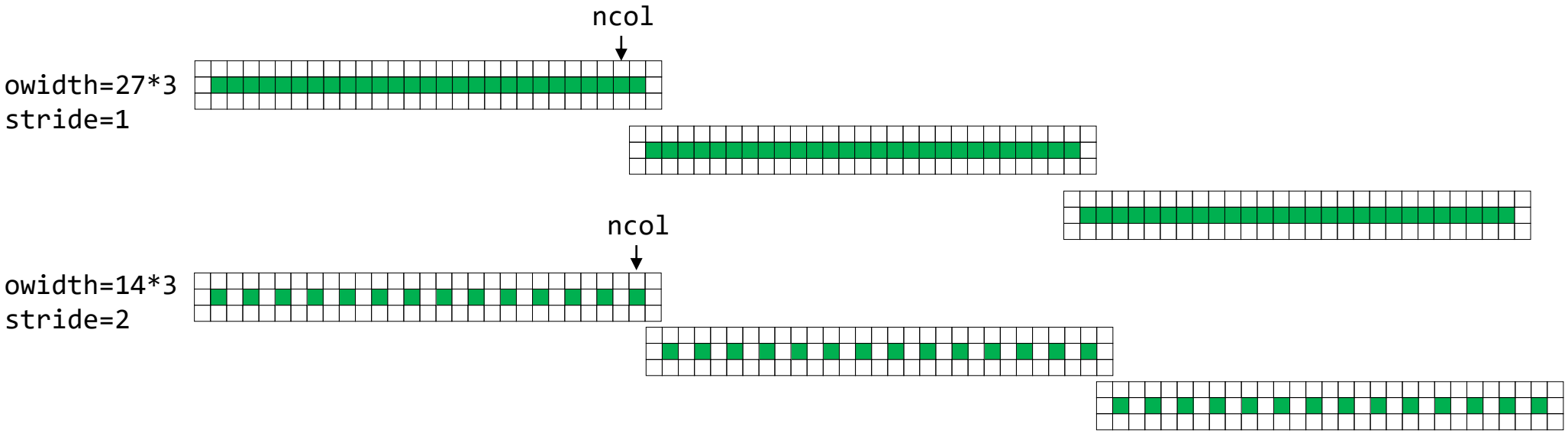
$(owidth/nstripe) \cdot t_{dot} < t_{row}$

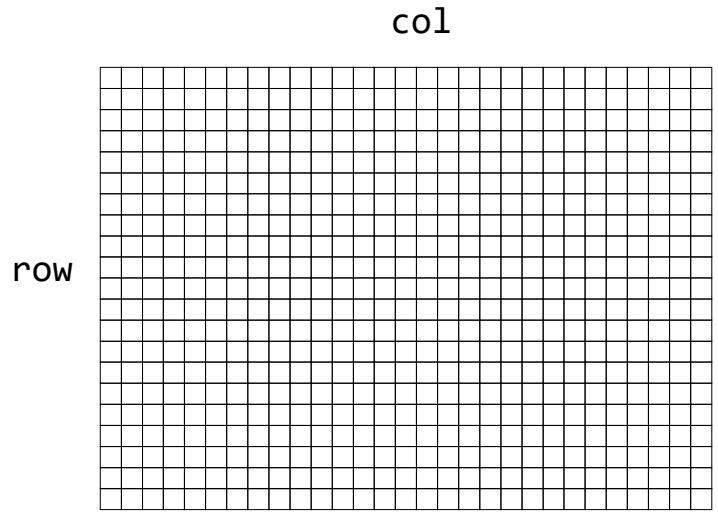
serialize nstripe features over output TDATA*OCHAN



$nstripe \cdot t_{clk} < t_{dot}$

ndata = # of input features since TLAST (for current layer)
stripe = ndata % istripe


ncol
owidth=27*3
stride=1
ncol
owidth=14*3
stride=2


col
row

t3.py (4K video @30fps)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 2158, 3838, 8)	224
conv2d_1 (Conv2D)	(None, 1078, 1918, 8)	584
conv2d_2 (Conv2D)	(None, 1076, 1916, 16)	1168
conv2d_3 (Conv2D)	(None, 537, 957, 16)	2320
conv2d_4 (Conv2D)	(None, 535, 955, 32)	4640
conv2d_5 (Conv2D)	(None, 267, 477, 32)	9248
conv2d_6 (Conv2D)	(None, 265, 475, 32)	9248
conv2d_7 (Conv2D)	(None, 132, 237, 32)	9248
conv2d_8 (Conv2D)	(None, 130, 235, 32)	9248
conv2d_9 (Conv2D)	(None, 64, 117, 32)	9248
conv2d_10 (Conv2D)	(None, 62, 115, 32)	9248
conv2d_11 (Conv2D)	(None, 30, 57, 32)	9248
conv2d_12 (Conv2D)	(None, 28, 55, 32)	9248
conv2d_13 (Conv2D)	(None, 13, 27, 32)	9248
conv2d_14 (Conv2D)	(None, 11, 25, 16)	4624
conv2d_15 (Conv2D)	(None, 5, 12, 16)	2320
conv2d_16 (Conv2D)	(None, 3, 10, 8)	1160
conv2d_17 (Conv2D)	(None, 1, 4, 8)	584
flatten (Flatten)	(None, 32)	0

Total params: 100,856
Trainable params: 100,856
Non-trainable params: 0

Ref Name	Used	Functional Category
LUT6	18400	LUT
FDRE	12853	Flop & Latch
MUXF7	5218	MuxFx
RAMD64E	2112	Distributed Memory
RAMB36E1	1851	Block Memory
CARRY4	1568	CarryLogic
LUT4	1517	LUT
DSP48E1	1032	Block Arithmetic
LUT5	807	LUT
LUT2	249	LUT
FDSE	146	Flop & Latch
LUT1	131	LUT
LUT3	117	LUT
OBUF	66	IO
IBUF	26	IO
MUXF8	15	MuxFx
OBUFT	1	IO
BUFG	1	Clock

andromeda.py log file

```
rliston@uscomputefe1:~/andromeda$ python3 andromeda.py --model t3 --fps 30
numpy 1.23.5
Namespace(model='t3', clk=500000000.0, fps=30.0, tdata=8)
layer    0 nstripe   14 stride    1 sdepth   3291 wdepth     31 rate 5.375e+10 nmac    108 shapes [   1 2160 3840    3] [   1 2158 3838    8] [8 3 3 3] [8]
layer    1 nstripe    9 stride    2 sdepth  17057 wdepth     76 rate 3.578e+10 nmac     72 shapes [   1 2158 3838    8] [   1 1078 1918    8] [8 3 3 8] [8]
layer    2 nstripe    9 stride    1 sdepth   6819 wdepth     76 rate 7.146e+10 nmac    143 shapes [   1 1078 1918    8] [   1 1076 1916   16] [16  3  3  8] [16]
layer    3 nstripe    5 stride    2 sdepth  30656 wdepth    148 rate 3.562e+10 nmac     72 shapes [   1 1076 1916   16] [  1 537 957  16] [16  3  3 16] [16]
layer    4 nstripe    5 stride    1 sdepth  12249 wdepth    148 rate 7.104e+10 nmac    143 shapes [  1 537 957  16] [  1 535 955  32] [32  3  3 16] [32]
layer    5 nstripe    3 stride    2 sdepth  50933 wdepth    292 rate 3.532e+10 nmac     71 shapes [  1 535 955  32] [  1 267 477  32] [32  3  3 32] [32]
layer    6 nstripe    3 stride    1 sdepth  20352 wdepth    292 rate 3.521e+10 nmac     71 shapes [  1 267 477  32] [  1 265 475  32] [32  3  3 32] [32]
layer    7 nstripe    1 stride    2 sdepth  76000 wdepth    292 rate 8.700e+09 nmac     18 shapes [  1 265 475  32] [  1 132 237  32] [32  3  3 32] [32]
layer    8 nstripe    1 stride    1 sdepth  30336 wdepth    292 rate 8.649e+09 nmac     18 shapes [  1 132 237  32] [  1 130 235  32] [32  3  3 32] [32]
layer    9 nstripe    1 stride    2 sdepth  37600 wdepth    292 rate 2.112e+09 nmac      5 shapes [  1 130 235  32] [  1  64 117  32] [32  3  3 32] [32]
layer   10 nstripe    1 stride    1 sdepth  14976 wdepth    292 rate 2.070e+09 nmac      5 shapes [  1  64 117  32] [  1  62 115  32] [32  3  3 32] [32]
layer   11 nstripe    1 stride    2 sdepth  18400 wdepth    292 rate 4.928e+08 nmac      1 shapes [  1  62 115  32] [ 1 30 57 32] [32  3  3 32] [32]
layer   12 nstripe    1 stride    1 sdepth   7296 wdepth    292 rate 4.728e+08 nmac      1 shapes [ 1 30 57 32] [ 1 28 55 32] [32  3  3 32] [32]
layer   13 nstripe    1 stride    2 sdepth   8800 wdepth    292 rate 1.064e+08 nmac      1 shapes [ 1 28 55 32] [ 1 13 27 32] [32  3  3 32] [32]
layer   14 nstripe    1 stride    1 sdepth   3456 wdepth    292 rate 4.852e+07 nmac      1 shapes [ 1 13 27 32] [ 1 11 25 16] [16  3  3 32] [16]
layer   15 nstripe    1 stride    2 sdepth   2000 wdepth    148 rate 4.752e+06 nmac      1 shapes [ 1 11 25 16] [ 1  5 12 16] [16  3  3 16] [16]
layer   16 nstripe    1 stride    1 sdepth    768 wdepth    148 rate 2.074e+06 nmac      1 shapes [ 1  5 12 16] [ 1  3 10  8] [ 8  3  3 16] [8]
layer   17 nstripe    1 stride    2 sdepth    400 wdepth     76 rate 1.296e+05 nmac      1 shapes [ 1  3 10  8] [1 1 4 8] [8 3 3 8] [8]

total stripe RAM bits    147841520
total weight RAM bits       816832
total required MAC units          733
total used MAC units         1032
rliston@uscomputefe1:~/andromeda$
```

t3.py (4K video @30fps)

3. Memory

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	1851	0	0	1880	98.46
RAMB36/FIFO*	1851	0	0	1880	98.46
RAMB36E1 only	1851				
RAMB18	0	0	0	3760	0.00

* Note: Each Block RAM Tile only has one FIFO logic available and therefore can accommodate only one FIFO36E1 or one FIFO18E1. However, if a FIFO18E1 occupies a Block RAM Tile, that tile can still accommodate a RAMB18E1

4. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	1032	0	0	3360	30.71
DSP48E1 only	1032				

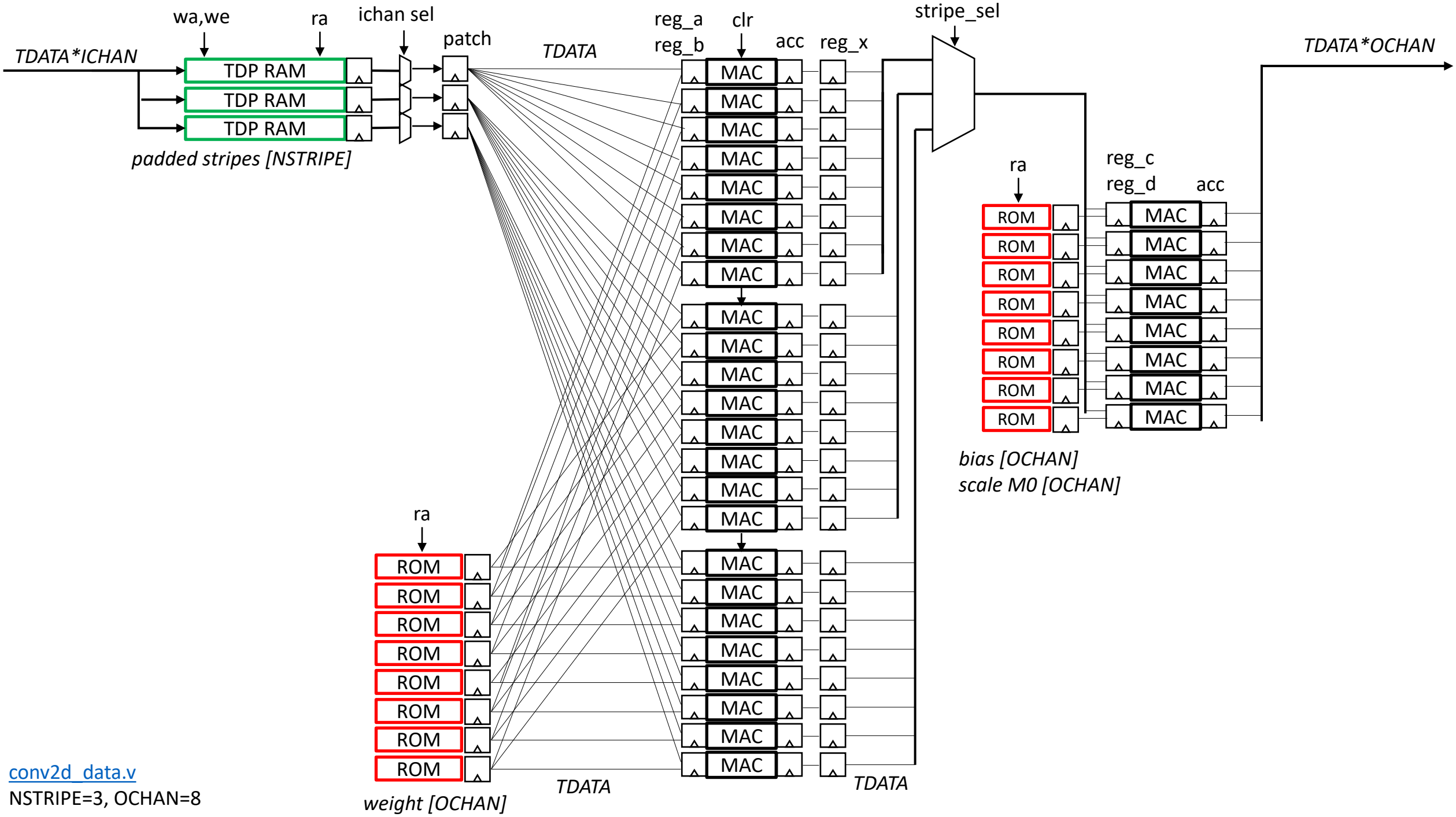
Target applications

- high speed, low latency inference at the edge
  - ex: 3840x2160 @100 fps image classification for industrial applications
  - process raw uncompressed pixels directly from sensor (e.g. HDMI,MIPI)
  - hyperspectral image sensors, RF IQ baseband, 300+ frames/s

- inference only, supports small but useful subset of TFLite (conv2d only, padding='valid', stride={1,2}, activation=RELU)
- no external memory required
- real time data stream -> real time prediction stream (or feature map)
- tool to compile TFLite model into synthesizable Verilog
- generated Verilog code is self contained, includes streaming inference and weights
- compiler generates parallel code for OCHAN*NSTRIPE MAC operations
- compiler generates ROM file with the weights

Next steps

1. ~~synthesize conv2d_dp.v using Vivado, Quartus, Raptor~~
2. add formal assertion and cover statements to conv2d_dp.v
3. simulate unit testbench for conv2d_dp.v using iverilog
4. ~~TFLite FlatBuffer model to Verilog datapath compiler, using pipeline of conv2d_dp(), incorporating performance spreadsheet~~
5. control logic synthesis using HLS, MLIR, LLM, manual FSM coding
6. full testbench which compares the simulated Verilog results for a complete TFLite model with the TF software reference
7. develop real time 3840x2160@100fps AI vision demo using Agilex/Xilinx eval board
8. develop wafer scale FPGA fabric using mobile process

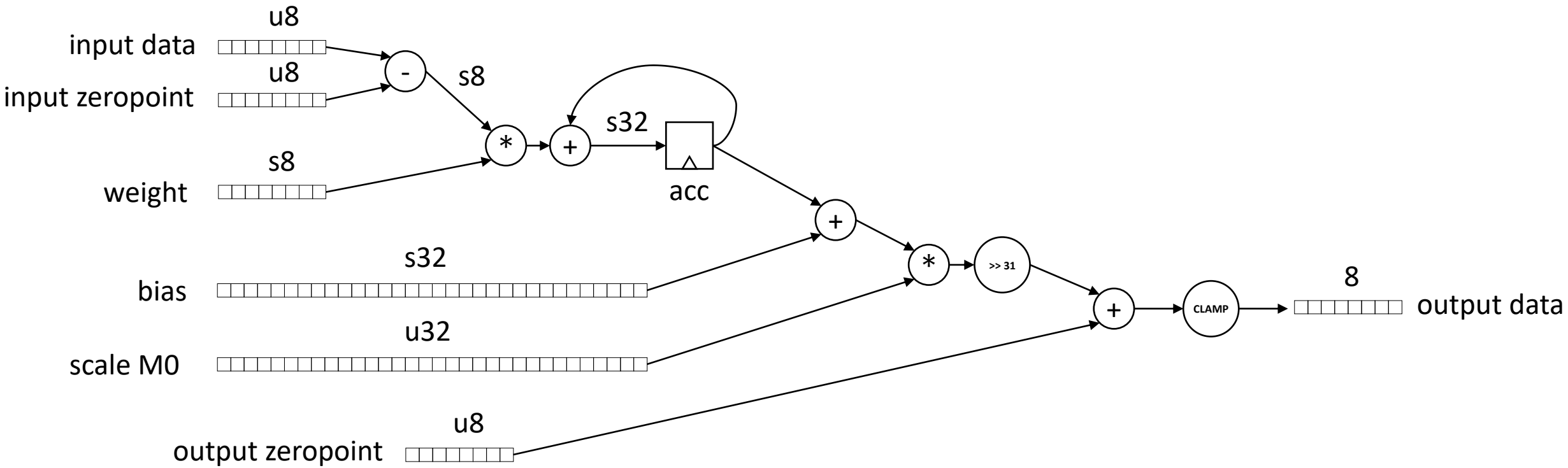


conv2d_data.v

NSTRIPE=3, OCHAN=8

https://github.com/RapidSilicon/andromeda/blob/main/conv2d_data.v

```
// conv2d.v data path
module conv2d_data #(parameter TDATA=8, parameter NSTRIPE=16, parameter ICHAN=64, parameter OCHAN=64, parameter
SADDR=12) (
    input clk, reset,
    input [2:0] dsp_op, // 0=NOP, 1=CLEAR, 2=MAC, 3=RELU, 4=MULT, 5=RSHIFT, 6=EMIT
    input [31:0] dsp_arg, // m0 = normalized scaling factor, 0.5 to 1.0
    input [NSTRIPE*SADDR-1:0] stripe_wa,
    input [NSTRIPE-1:0] stripe_wen,
    input [NSTRIPE*SADDR-1:0] stripe_ra,
    input wire [OCHAN*TDATA-1:0] weight_rd, // data from ROM
    input [ICHAN-1:0] ichan_sel, // 1-hot channel select
    input [NSTRIPE-1:0] stripe_sel, // 1-hot select for tdata_o
    input [ICHAN*TDATA-1:0] tdata_i,
    output reg [OCHAN*TDATA-1:0] tdata_o
);
genvar i,j;
integer k,m;

// padded stripes TDP BRAM state
reg [ICHAN*TDATA-1:0] stripe [NSTRIPE-1:0][2**SADDR-1:0];
reg [ICHAN*TDATA-1:0] stripe_rd [NSTRIPE-1:0];
reg [ICHAN*TDATA-1:0] stripe_rq [NSTRIPE-1:0];

// padded stripes WRITE PORT
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        always @ (posedge clk) begin
            if(stripe_wen[i])
                stripe[i][stripe_wa[i*SADDR +: SADDR]] <= tdata_i;
        end
    end
endgenerate

// padded stripes READ PORT
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        always @ (posedge clk) begin
            stripe_rd[i] <= stripe[i][stripe_ra[i*SADDR +: SADDR]];
            stripe_rq[i] <= stripe_rd[i]; // pipeline register
        end
    end
endgenerate

// for each NSTRIPE, generate a ICHAN:1 mux which is TDATA bits wide, using ichan_sel to select
reg [ICHAN-1:0] patch_mux [NSTRIPE-1:0][TDATA-1:0];
reg [TDATA-1:0] patch [NSTRIPE-1:0];
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        for (j=0;j<TDATA;j=j+1) begin
            always @(posedge clk) begin
                for (k=0;k<ICHAN;k=k+1)
                    patch_mux[i][j][k] = (stripe_rq[i][k*TDATA+j] & ichan_sel[k]);
                patch[i][j] <= |patch_mux[i][j];
            end
        end
    end
endgenerate

// weight ROM per OCHAN
//wire [TDATA*OCHAN-1:0] weight_rd;
//weight_rom weight_rom (.clk(clk), .addr(weight_ra), .data(weight_rd));
wire [TDATA-1:0] weight [OCHAN-1:0];
generate for (i=0;i<OCHAN;i=i+1)
        assign weight[i] = weight_rd[i*TDATA +: TDATA];
endgenerate

// NSTRIDE*OCHAN DSP instances
reg signed [31:0] mult [NSTRIPE-1:0][OCHAN-1:0];
reg signed [31:0] acc [NSTRIPE-1:0][OCHAN-1:0];
reg signed [31:0] reg_z [NSTRIPE-1:0][OCHAN-1:0];
reg signed [15:0] reg_a [NSTRIPE-1:0][OCHAN-1:0];
reg signed [15:0] reg_b [NSTRIPE-1:0][OCHAN-1:0];
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        for (j=0;j<OCHAN;j=j+1) begin
            always @(posedge clk) begin
                mult[i][j] <= reg_a[i][j] * reg_b[i][j];
                if (dsp_op=='d3)
                    acc[i][j] <= 'd0;
                else
                    acc[i][j] <= mult[i][j] + acc[i][j];
            end
        end
    end
endgenerate

generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        for (j=0;j<OCHAN;j=j+1) begin
            always @(posedge clk) begin
                reg_a[i][j] <= patch[i];
                reg_b[i][j] <= weight[j];
                case (dsp_op)
                    'd0 : reg_z[i][j] <= reg_z[i][j];
                    'd1 : reg_z[i][j] <= acc[i][j];
                    'd2 : reg_z[i][j] <= reg_z[i][j][31:24];
                    'd3 : reg_z[i][j] <= 'b0;
                    //'d4 : reg_z[i][j] <= saturate/shift/round
                    default : reg_z[i][j] <= 'bx;
                endcase
            end
        end
    end
endgenerate

/*
// dsp
reg signed [47:0] mult,acc;
reg acc_clear;
always @(posedge clk) begin
                    mult <= rowbuf_rd*weights_rd;
                    if (acc_clear)
                                        acc <= 'd0;
                    else
                                        acc <= mult+acc;
end
*/


// for each NSTRIPE, generate a OCHAN:1 mux which is TDATA bits wide, using stripe_sel to select
generate
    for (j=0;j<OCHAN;j=j+1) begin
        always @(posedge clk) begin
            tdata_o[j*TDATA +: TDATA] = 0;
            for (m=0;m<NSTRIPE;m=m+1) begin
                tdata_o[j*TDATA +: TDATA] |= reg_z[m][j] & stripe_sel[m];
            end
        end
    end
endgenerate

endmodule
```

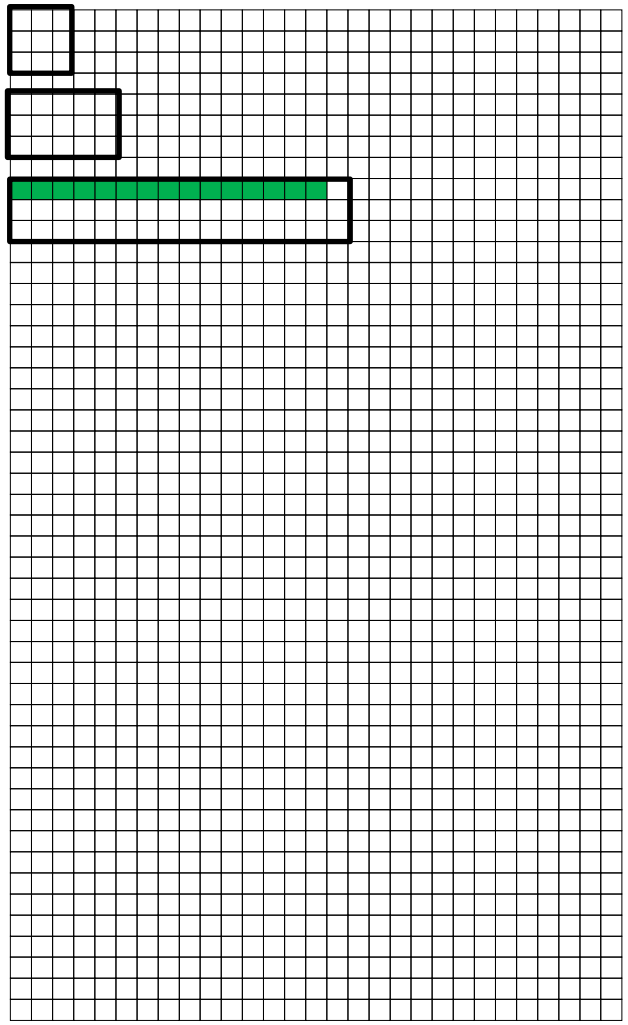


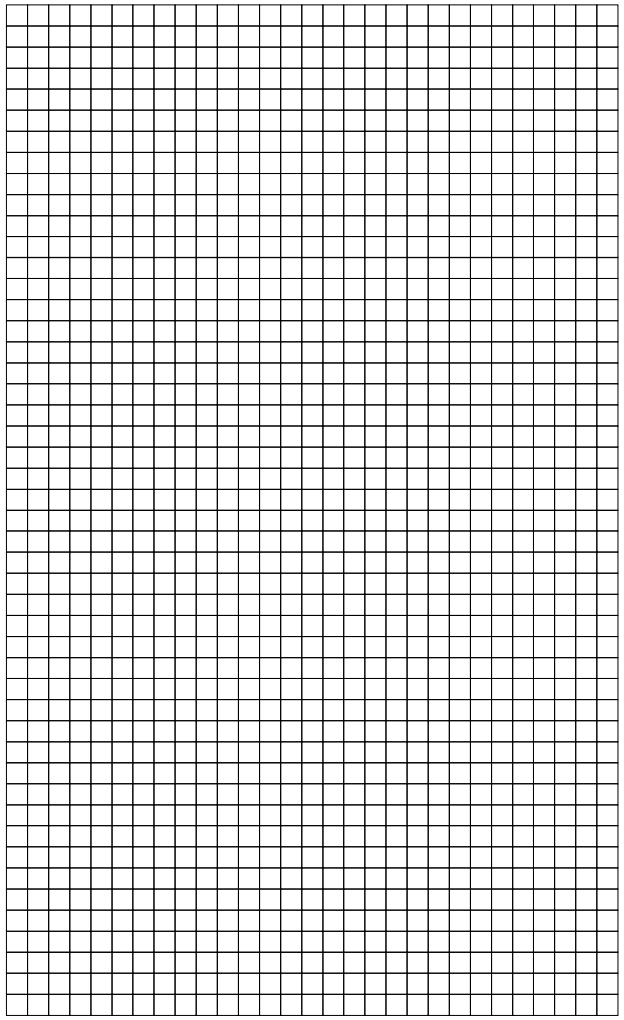
1: compute dot product (8*8+32)
2: add bias (32+32)
3. multiply by M0 scale (32*32)
4. shift right 31 bits with round
5. clamp to 8 bits + RELU

nstripes=1

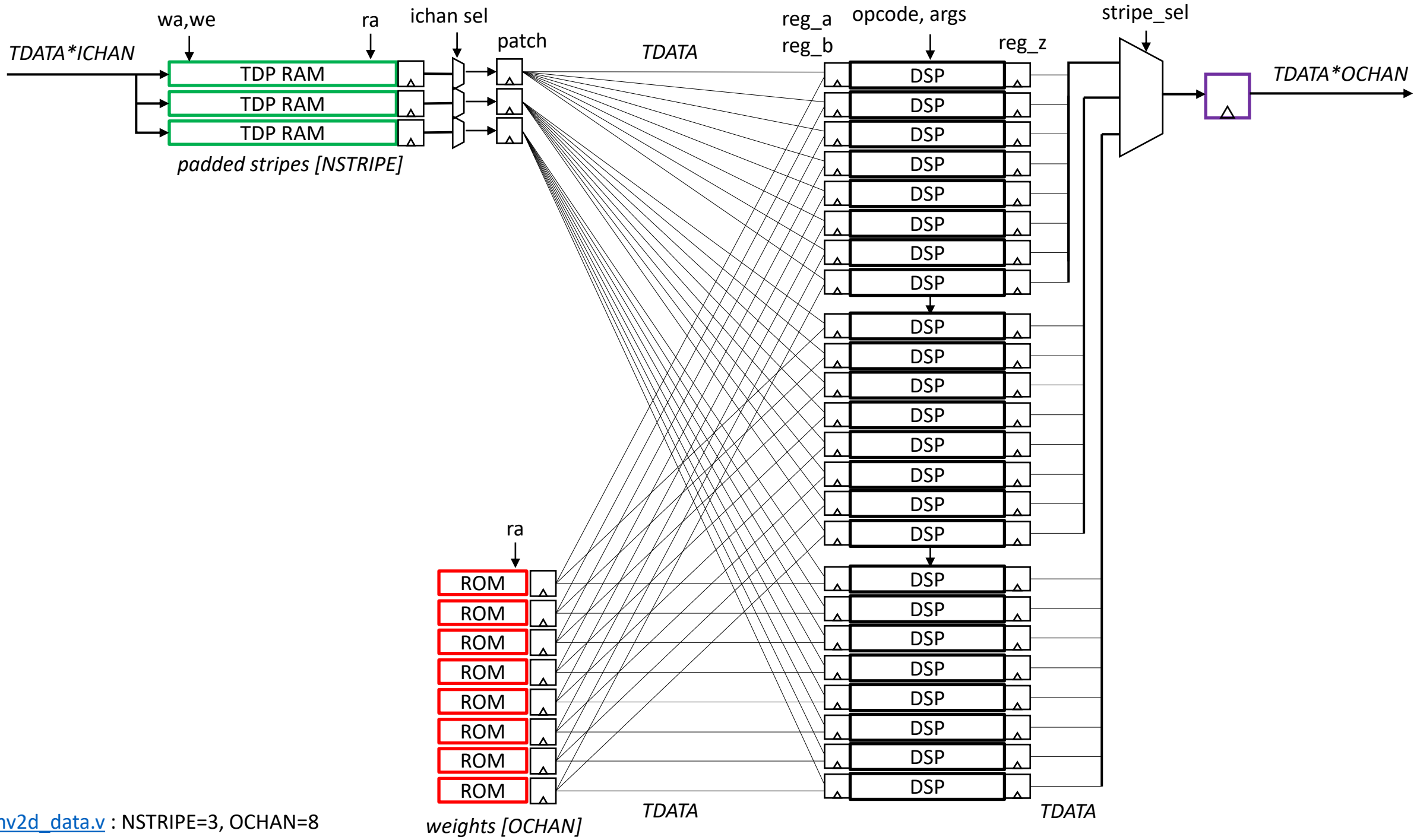
nstripes=3

nstripes=15

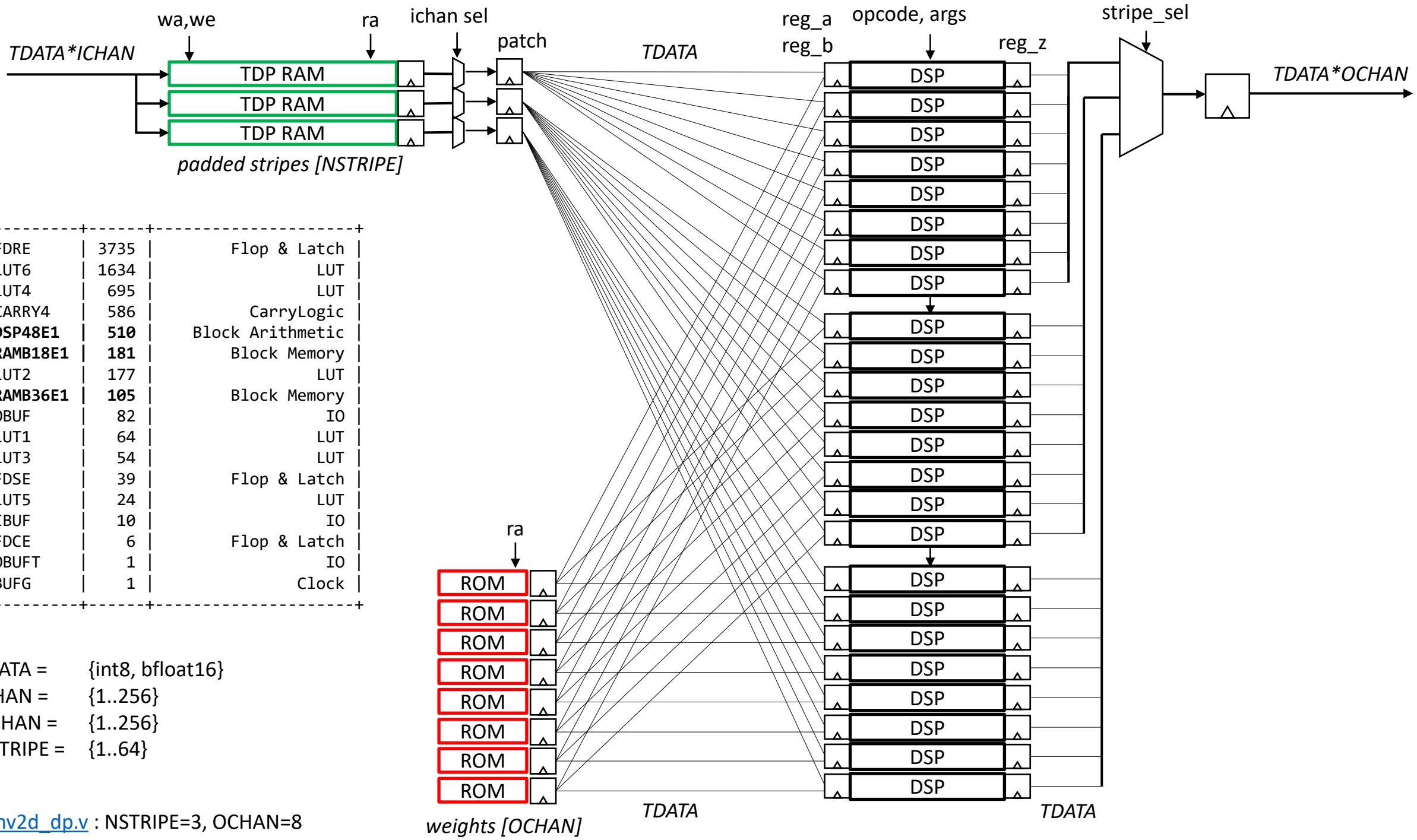




fps	100															
width	3,840															
height	2,160															
channels	3															
clk	500,000,000															
layer	iwidth	iheight	ichan	stride	kheight	kwidth	owidth	oheight	ochan	patches/s	MAC/patch	MAC/s	**required MAC**	nstripe	nstripe*ochan	
1	3,840	2160	3	1	3	3	3,838	2,158	16	829,440,000	432	358,318,080,000	717	45	720	
2	3,838	2,158	16	2	3	3	1,918	1,078	16	206,760,400	2,304	476,375,961,600	953	60	960	
3	1,918	1,078	16	1	3	3	1,916	1,076	32	206,161,600	4,608	949,992,652,800	1,900	60	1,920	
4	1,916	1,076	32	2	3	3	957	537	32	51,390,900	9,216	473,618,534,400	948	30	960	
5	957	537	32	1	3	3	955	535	64	51,092,500	18,432	941,736,960,000	1,884	30	1,920	
6	955	535	64	2	3	3	477	267	64	12,735,900	36,864	469,496,217,600	939	15	960	
7	477	267	64	1	3	3	475	265	64	12,587,500	36,864	464,025,600,000	929	15	960	
8	475	265	64	2	3	3	237	132	64	3,128,400	36,864	115,325,337,600	231	4	256	
9	237	132	64	1	3	3	235	130	64	3,055,000	36,864	112,619,520,000	226	4	256	
10	235	130	64	2	3	3	117	64	64	748,800	36,864	27,603,763,200	56	1	64	
11	117	64	64	1	3	3	115	62	64	713,000	36,864	26,284,032,000	53	1	64	
12	115	62	64	2	3	3	57	30	64	171,000	36,864	6,303,744,000	13	1	64	
13	57	30	64	1	3	3	55	28	64	154,000	36,864	5,677,056,000	12	1	64	
14	55	28	64	2	3	3	27	13	64	35,100	36,864	1,293,926,400	3	1	64	
15	27	13	64	1	3	3	25	11	32	27,500	18,432	506,880,000	2	1	32	
16	25	11	32	2	3	3	12	5	32	6,000	9,216	55,296,000	1	1	32	
17	12	5	32	1	3	3	10	3	16	3,000	4,608	13,824,000	1	1	16	
18	10	3	16	2	3	3	4	1	16	400	2,304	921,600	1	1	16	
												4,429,178,265,600	**8,866**		**9,264**	



conv2d_data.v : NSTRIPE=3, OCHAN=8



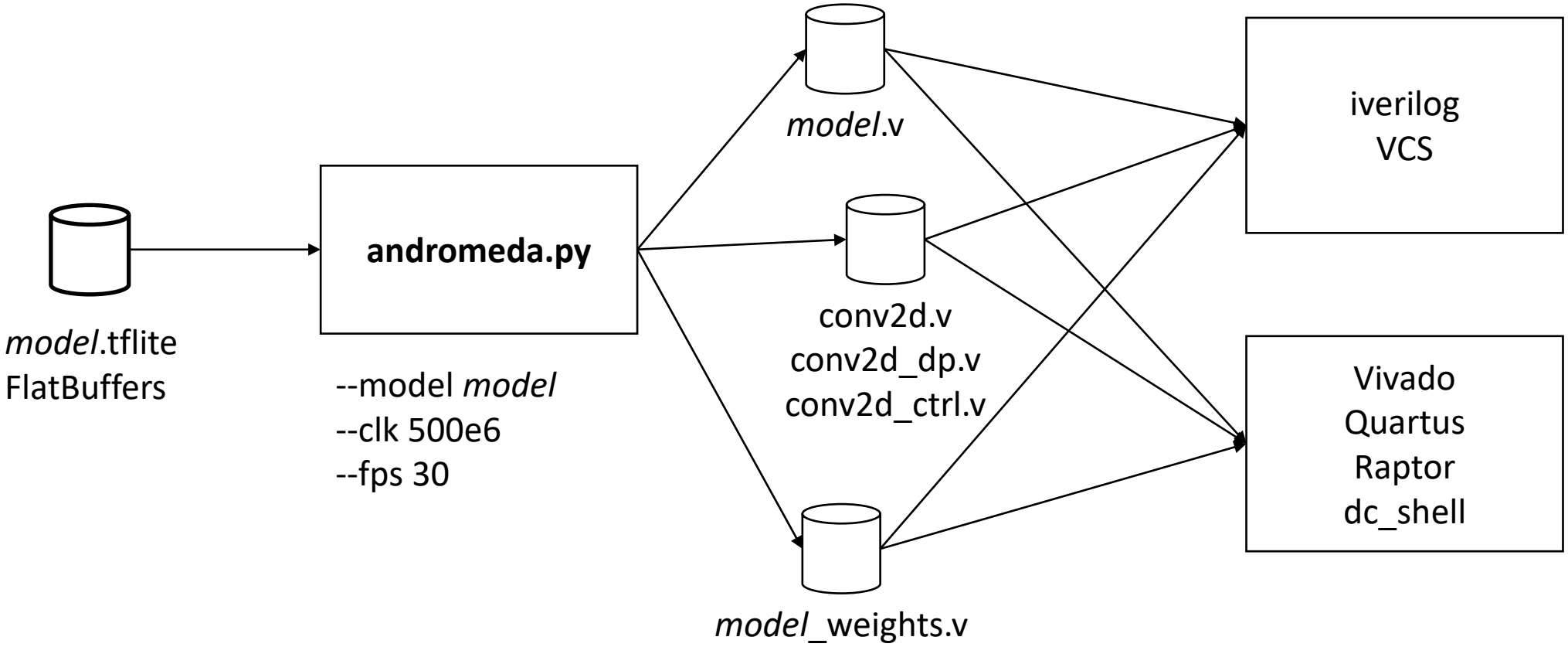
FDRE	3735	Flop & Latch
LUT6	1634	LUT
LUT4	695	LUT
CARRY4	586	CarryLogic
DSP48E1	**510**	Block Arithmetic
RAMB18E1	**181**	Block Memory
LUT2	177	LUT
RAMB36E1	**105**	Block Memory
OBUF	82	IO
LUT1	64	LUT
LUT3	54	LUT
FDSE	39	Flop & Latch
LUT5	24	LUT
IBUF	10	IO
FDCE	6	Flop & Latch
OBUFT	1	IO
BUFG	1	Clock

TDATA = {int8, bfloat16}
ICHAN = {1..256}
OCHAN = {1..256}
NSTRIPE = {1..64}

conv2d_dp.v : NSTRIPE=3, OCHAN=8

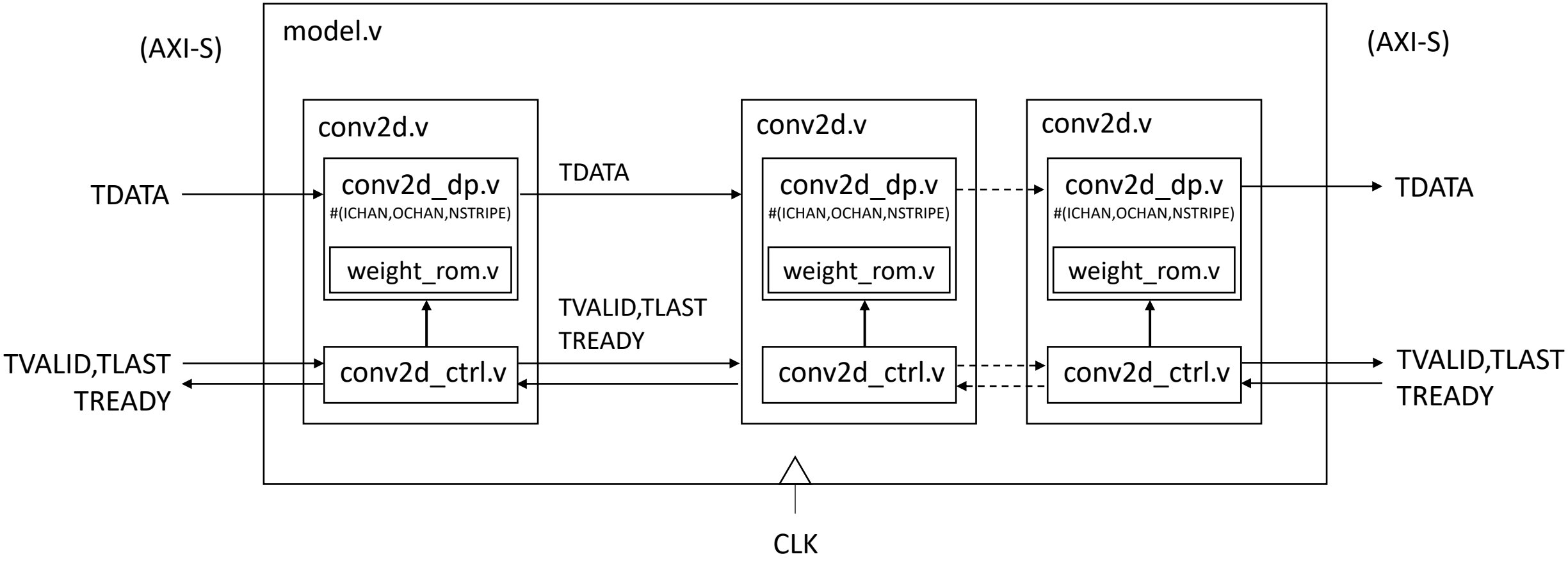
```
// conv2d.v data path
module conv2d_dp #(parameter TDATA=8, parameter NSTRIPE=3, parameter ICHAN=8, parameter
OCHAN=16, parameter SADDR=15, parameter WADDR=12) (
    input clk, reset,
    input [2:0] dsp_op, // 0=NOP, 1=CLEAR, 2=MAC, 3=RELU, 4=MULT, 5=RSHIFT, 6=EMIT
    input [31:0] dsp_arg, // m0 = normalized scaling factor, 0.5 to 1.0
    input [NSTRIPE*SADDR-1:0] stripe_wa,
    input [NSTRIPE-1:0] stripe_wen,
    input [NSTRIPE*SADDR-1:0] stripe_ra,
    input [ICHAN-1:0] ichan_sel, // 1-hot channel select
    input [OCHAN*WADDR-1:0] weight_ra,
    input [NSTRIPE-1:0] stripe_sel, // 1-hot select for tdata_o
    input [ICHAN*TDATA-1:0] tdata_i,
    output reg [OCHAN*TDATA-1:0] tdata_o
);
genvar i,j;
integer k,m;

// padded stripes TDP BRAM state
reg [ICHAN*TDATA-1:0] stripe [NSTRIPE-1:0][2**SADDR-1:0];
reg [ICHAN*TDATA-1:0] stripe_rd [NSTRIPE-1:0];

// padded stripes WRITE PORT
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        always @ (posedge clk) begin
            if(stripe_wen[i])
                stripe[i][stripe_wa[i*SADDR +: SADDR]] <= tdata_i;
        end
    end
endgenerate

// padded stripes READ PORT
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        always @ (posedge clk) begin
            stripe_rd[i] <= stripe[i][stripe_ra[i*SADDR +: SADDR]];
        end
    end
endgenerate

// for each NSTRIPE, generate a ICHAN:1 mux which is TDATA bits wide, using ichan_sel to
select
reg [TDATA-1:0] patch [NSTRIPE-1:0];
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        for (j=0;j<TDATA;j=j+1) begin
            always @(posedge clk) begin
                patch[i][j] = 0;
                for (k=0;k<ICHAN;k=k+1)
                    patch[i][j] |= (stripe_rd[i][k*TDATA+j] & ichan_sel[k]);
            end
        end
    end
endgenerate

// weights RAM
reg [TDATA-1:0] weight [OCHAN-1:0][2**WADDR-1:0];
reg [TDATA-1:0] weight_rd [OCHAN-1:0];

// weights READ PORT
generate
    for (i=0;i<OCHAN;i=i+1) begin
        always @ (posedge clk) begin
            weight_rd[i] <= weight[i][weight_ra[i*WADDR +: WADDR]];
        end
    end
endgenerate

// NSTRIDE*OCHAN DSP instances
reg [31:0] reg_z [NSTRIPE-1:0][OCHAN-1:0];
reg [TDATA-1:0] reg_a [NSTRIPE-1:0][OCHAN-1:0];
reg [TDATA-1:0] reg_b [NSTRIPE-1:0][OCHAN-1:0];
generate
    for (i=0;i<NSTRIPE;i=i+1) begin
        for (j=0;j<OCHAN;j=j+1) begin
            always @(posedge clk) begin
                reg_a[i][j] = patch[i];
                reg_b[i][j] = weight_rd[j];
                case (dsp_op)
                    'd0 : reg_z[i][j] = reg_z[i][j];
                    'd1 : reg_z[i][j] = reg_a[i][j] * reg_b[i][j] + reg_z[i][j];
                    'd2 : reg_z[i][j] = reg_z[i][j][31:24];
                    'd3 : reg_z[i][j] = 'b0;
                    //'d4 : reg_z[i][j] = saturate/shift/round
                endcase
            end
        end
    end
endgenerate

// for each NSTRIPE, generate a OCHAN:1 mux which is TDATA bits wide, using stripe_sel to
select
generate
    for (j=0;j<OCHAN;j=j+1) begin
        always @(posedge clk) begin
            tdata_o[j*TDATA +: TDATA] = 0;
            for (m=0;m<NSTRIPE;m=m+1) begin
                tdata_o[j*TDATA +: TDATA] |= reg_z[m][j] & stripe_sel[m];
            end
        end
    end
endgenerate

endmodule
```

andromeda.py streaming CNN compiler

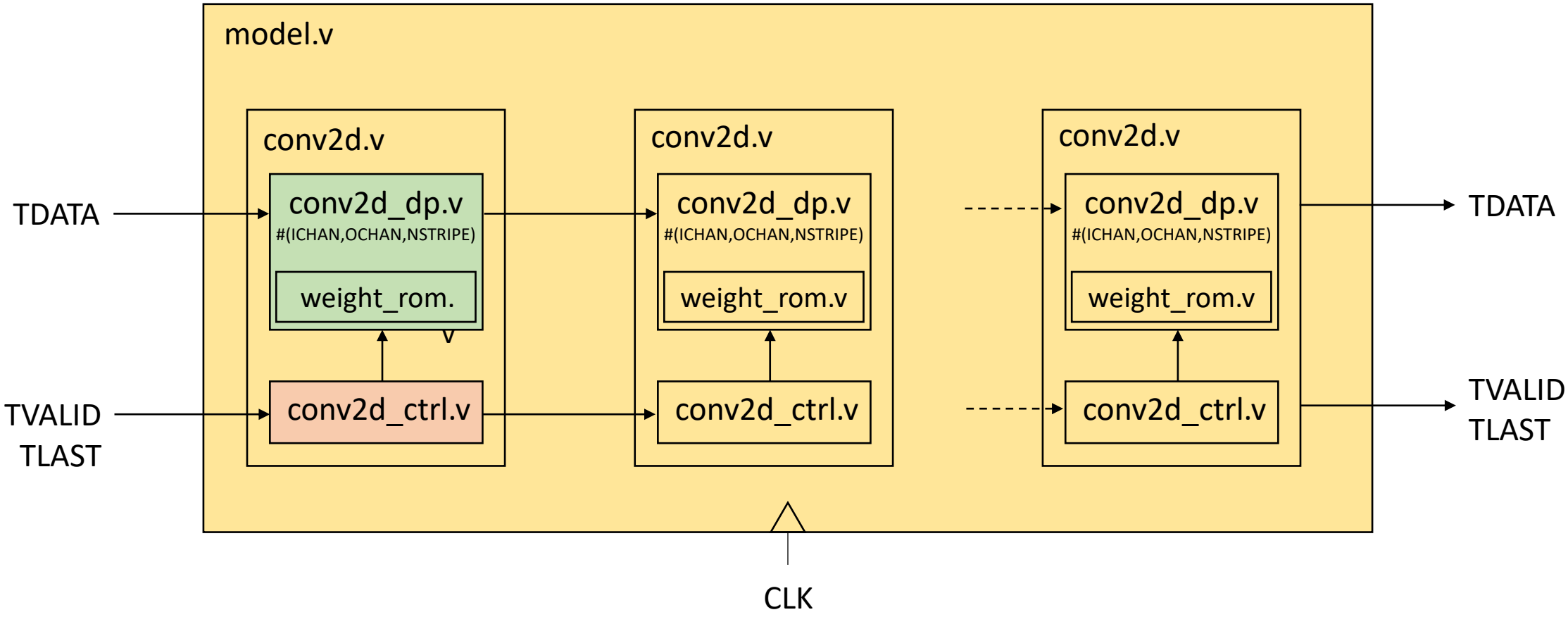


automated tool to compile TFLite model into synthesizable Verilog
compiler generates parallel code for OCHAN*NSTRIDE MAC operations per layer

Streaming CNN Verilog Hierarchy



Streaming CNN Verilog Hierarchy



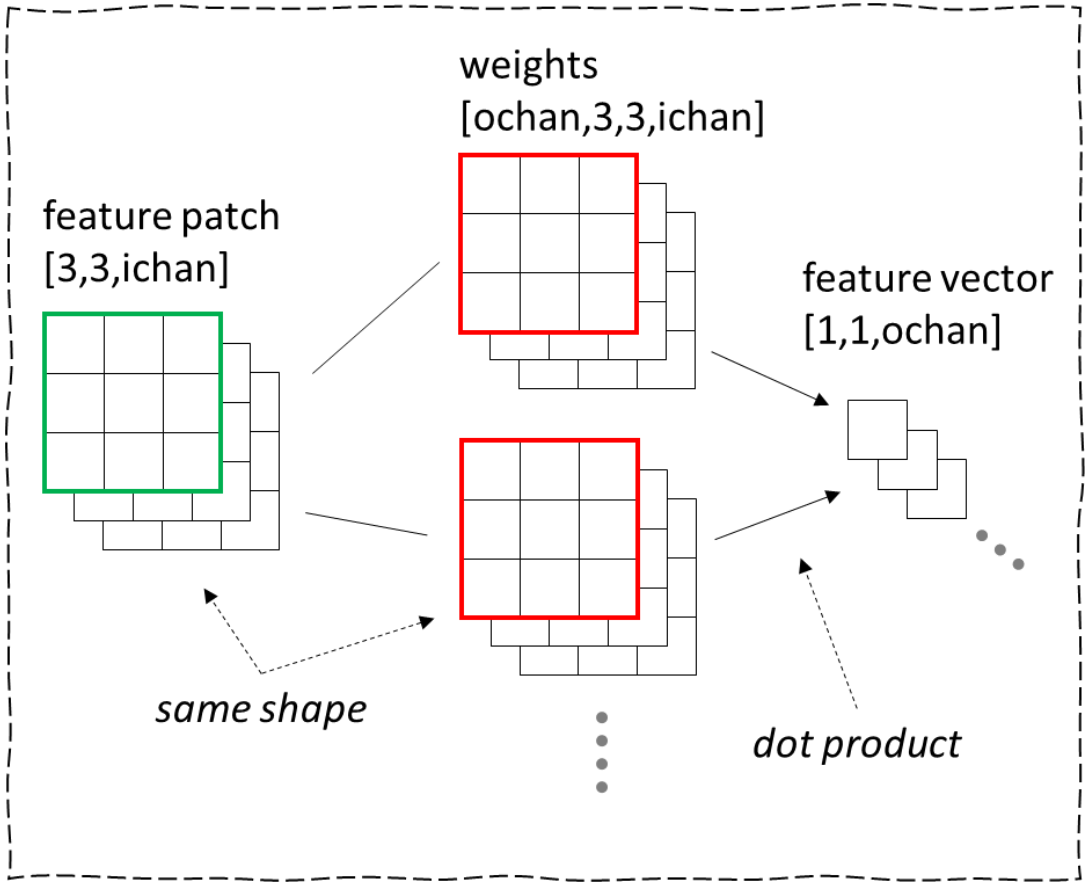
https://arxiv.org/pdf/1712.05877.pdf
https://www.tensorflow.org/lite/performance/quantization_spec#int8_quantized_operator_specifications

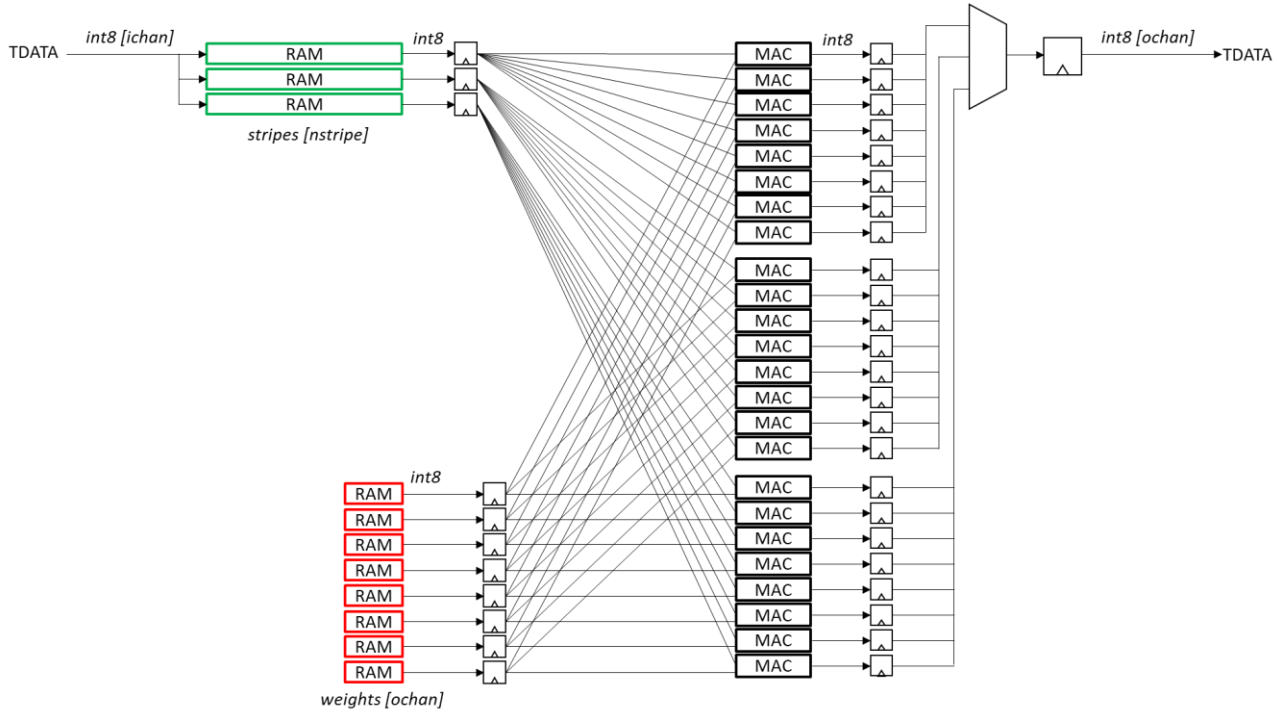
requirements for TFLite quant:

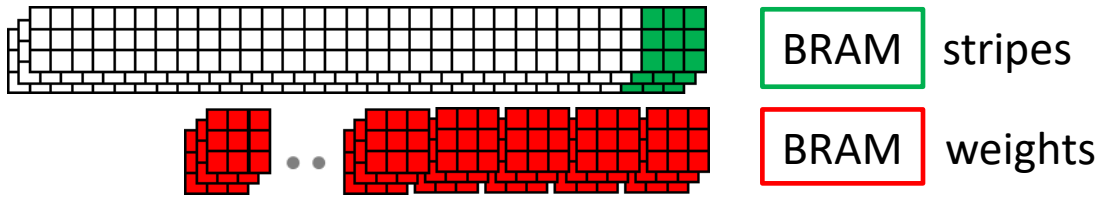
1. saturating 8*8+32 bit MAC
2. rounding-to-nearest right-shift

https://en.wikipedia.org/wiki/Wafer-scale_integration
https://en.wikipedia.org/wiki/Cerebras
https://arxiv.org/pdf/2112.07571.pdf

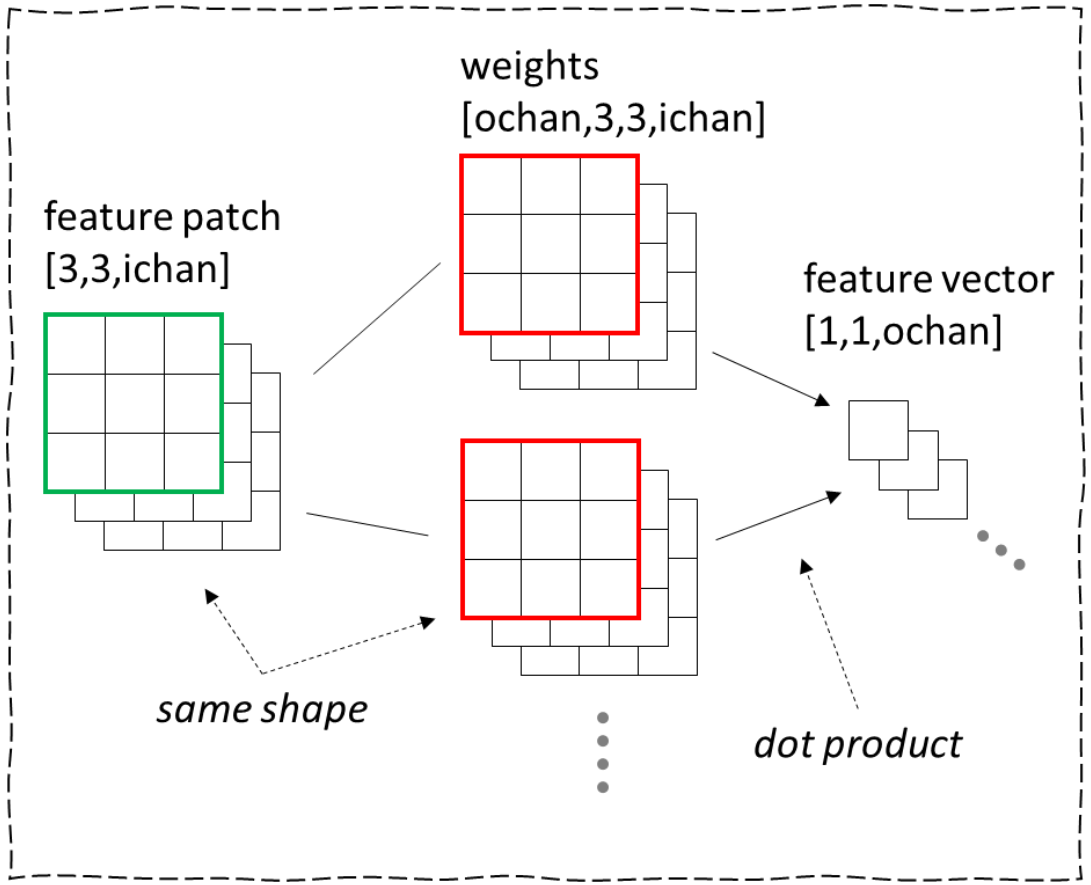
CNN dot product using FPGA

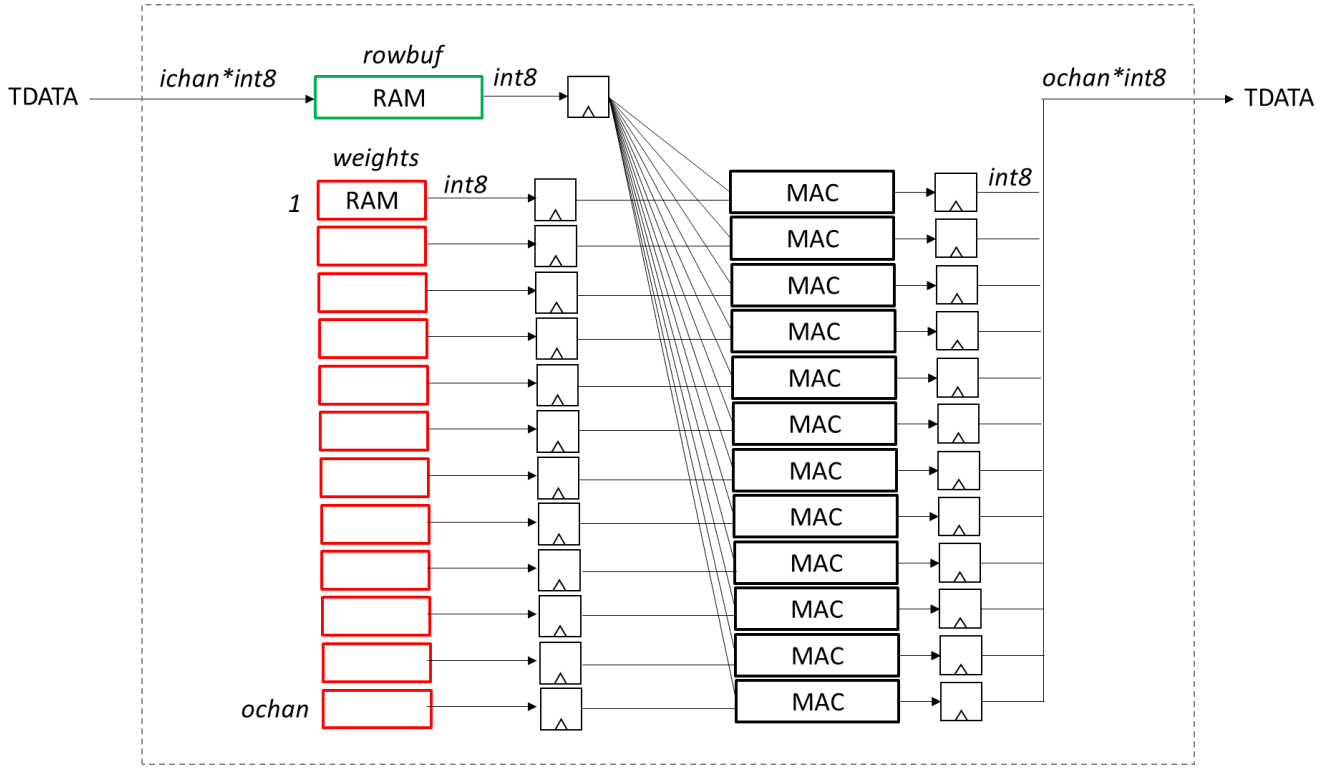


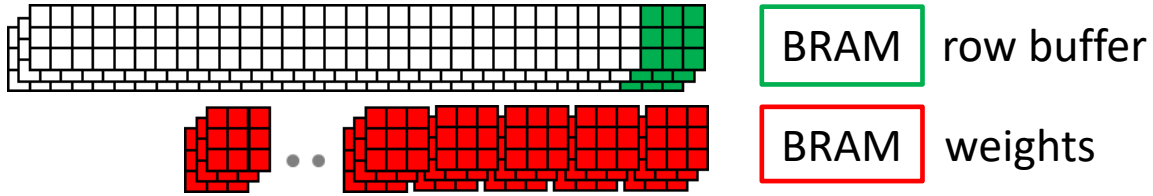


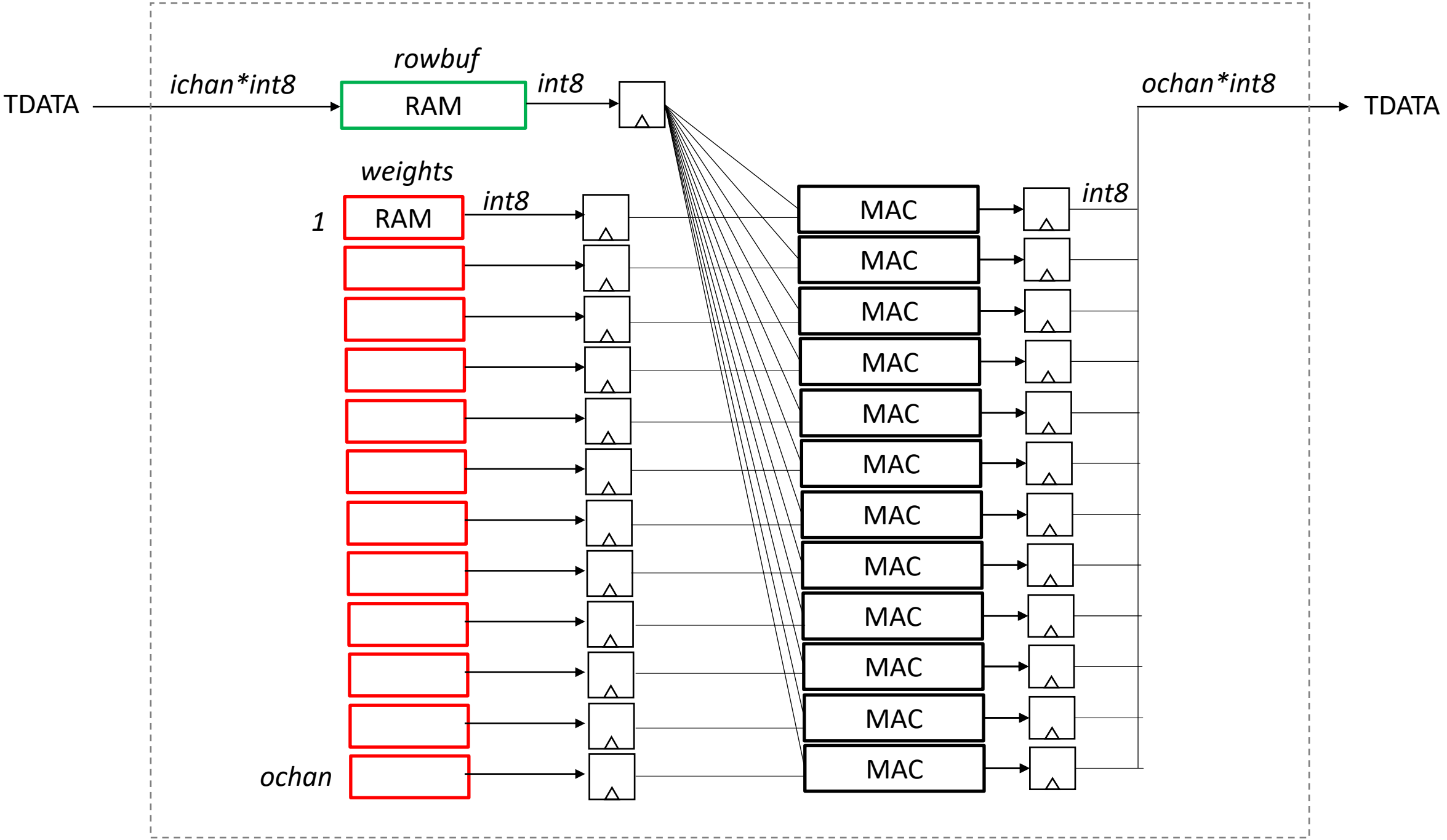


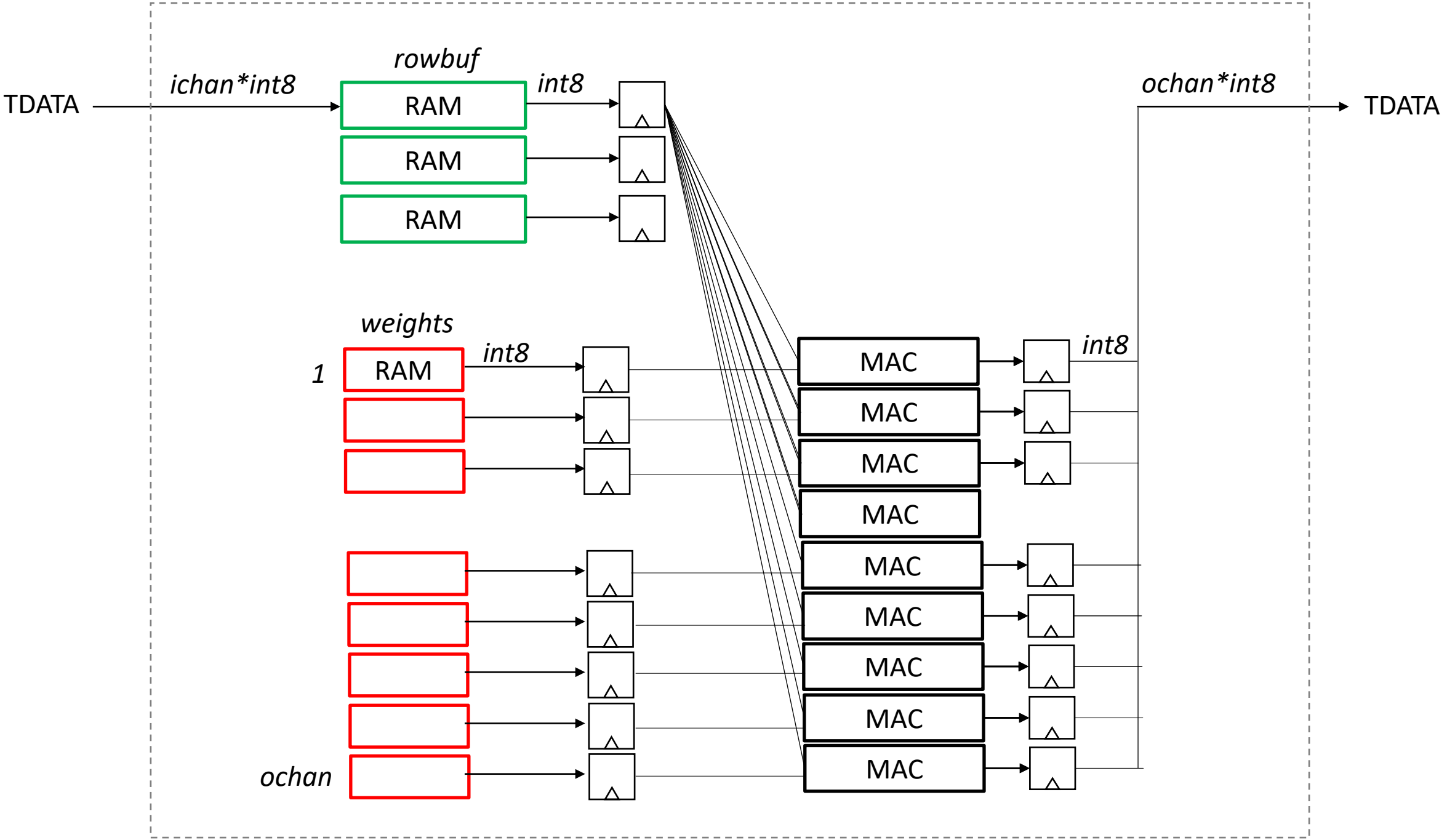
CNN dot product using FPGA

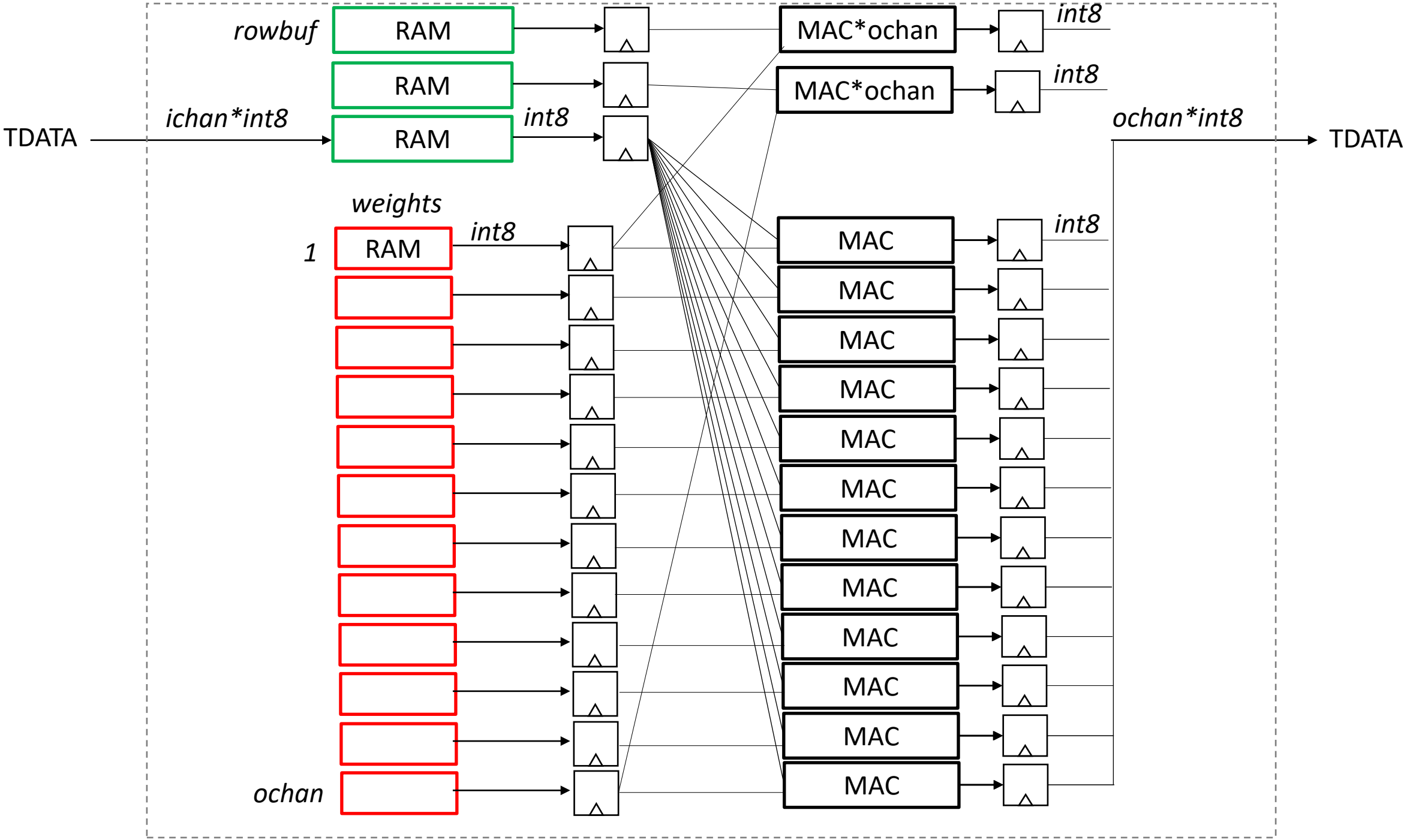


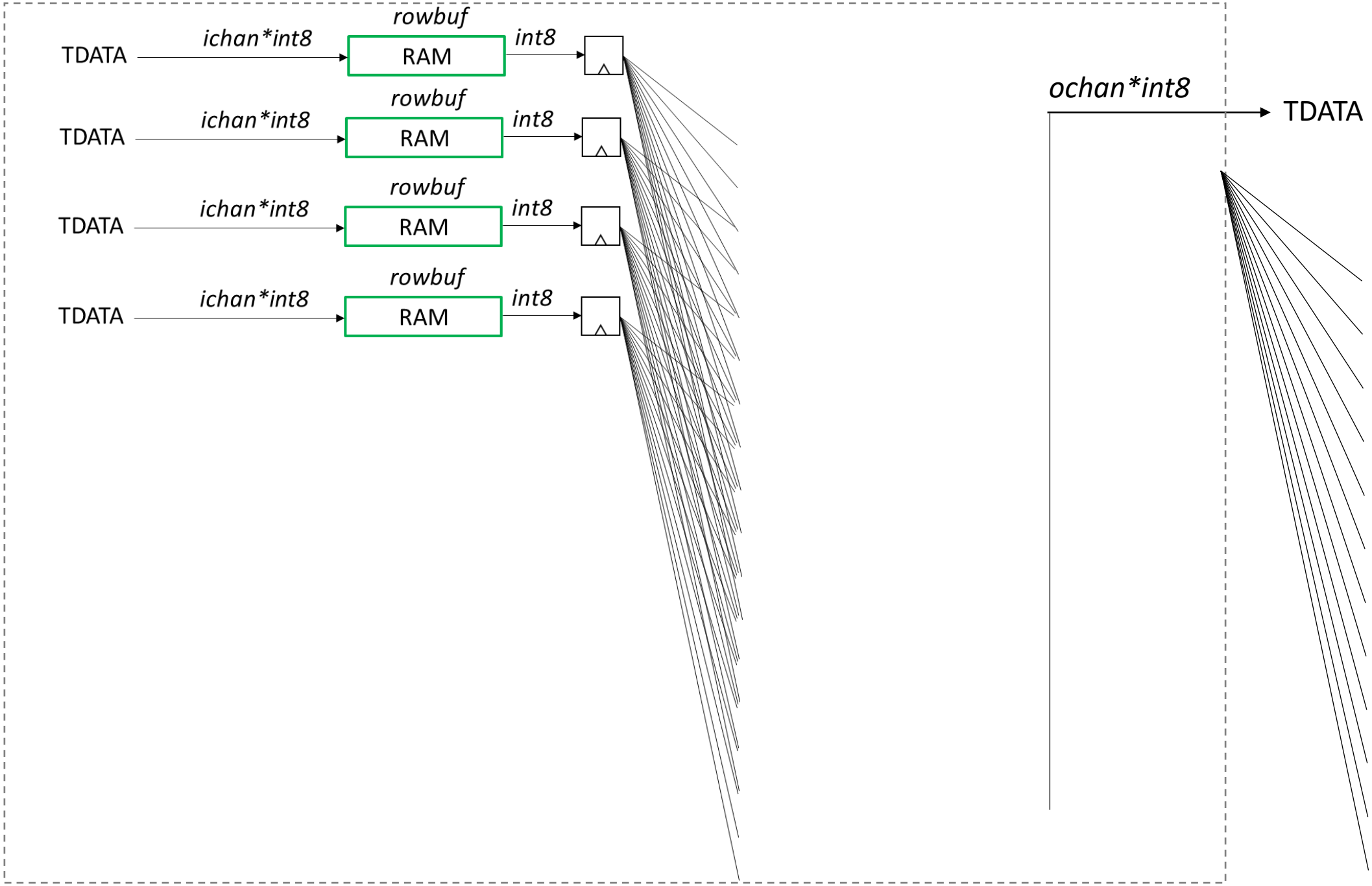




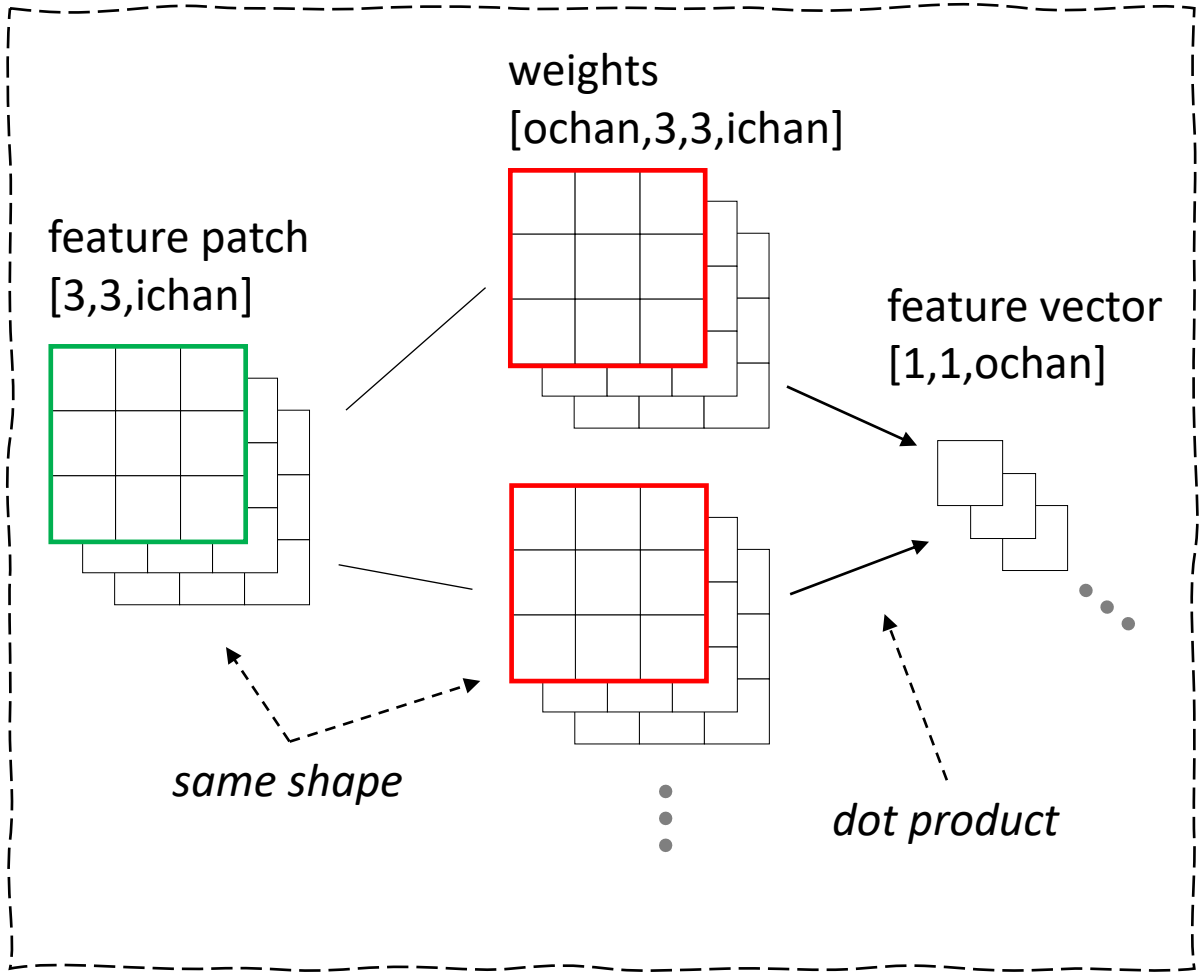

TDATA
ichan*int8
rowbuf
RAM
int8
ochan*int8
TDATA
weights
1
RAM
int8
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
int8
ochan

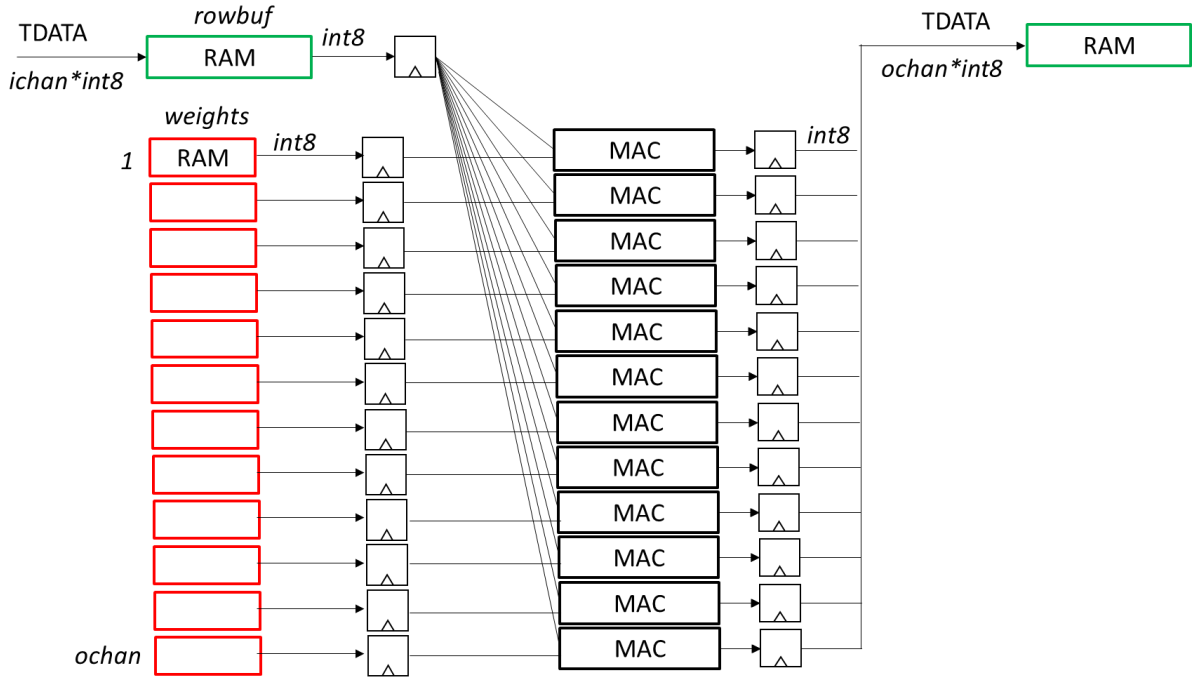

TDATA
ichan*int8
rowbuf
RAM
RAM
RAM
int8
weights
1
RAM
int8
ochan
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
int8
ochan*int8
TDATA

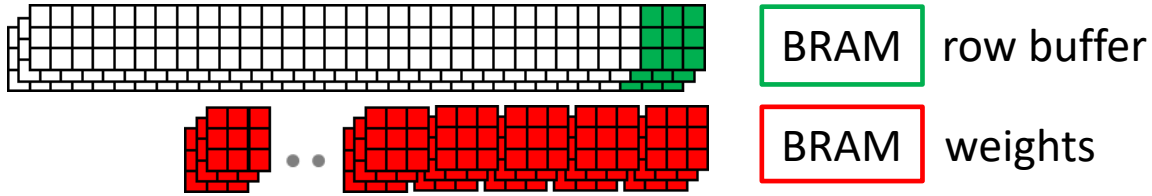

rowbuf
RAM
RAM
RAM
TDATA
ichan*int8
int8
weights
1
RAM
int8
ochan
MAC*ochan
MAC*ochan
int8
int8
ochan*int8
TDATA
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
int8

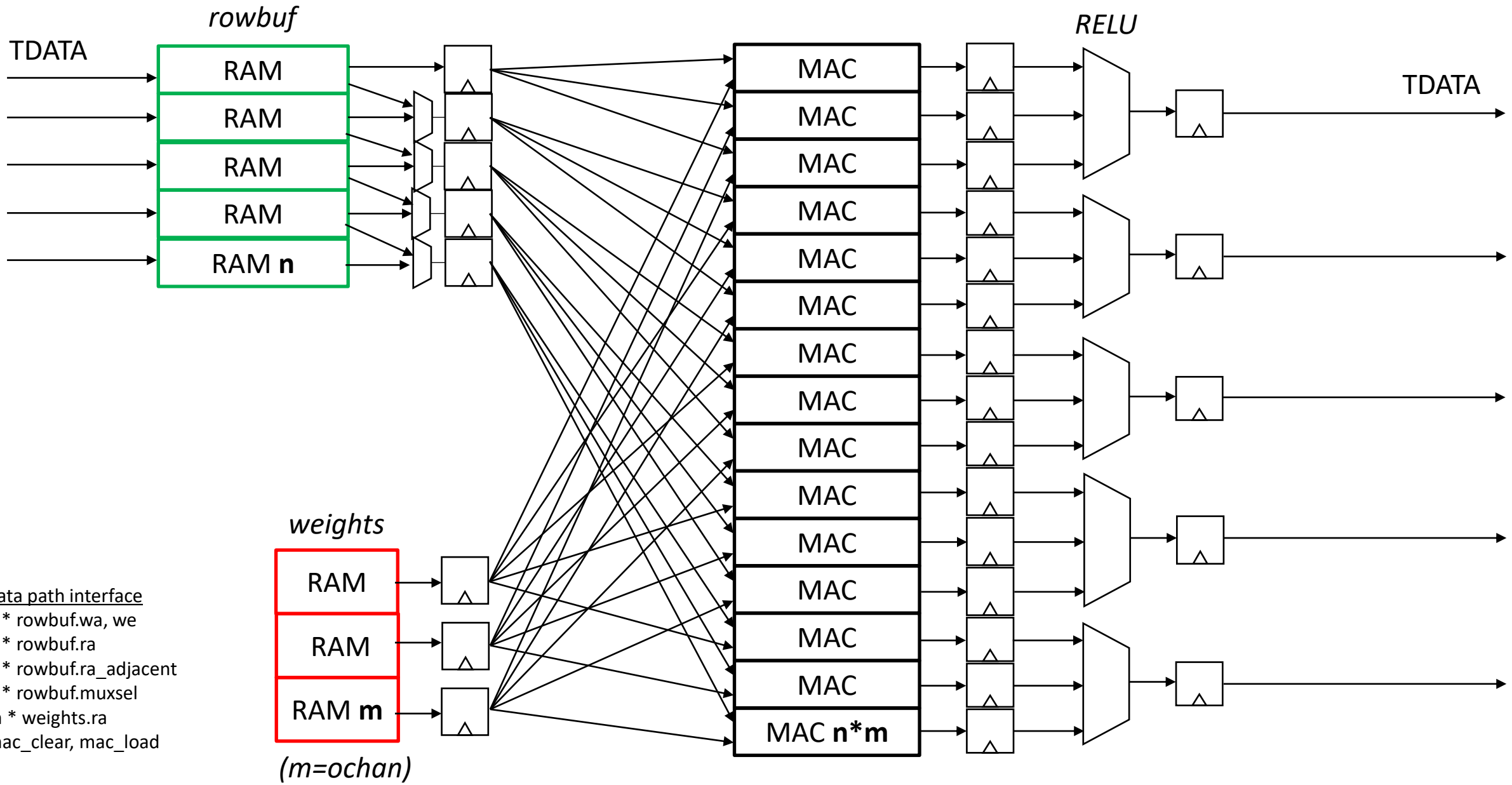

rowbuf
TDATA
ichan*int8
RAM
int8
rowbuf
TDATA
ichan*int8
RAM
int8
rowbuf
TDATA
ichan*int8
RAM
int8
rowbuf
TDATA
ichan*int8
RAM
int8
ochan*int8
TDATA

CNN dot product using FPGA

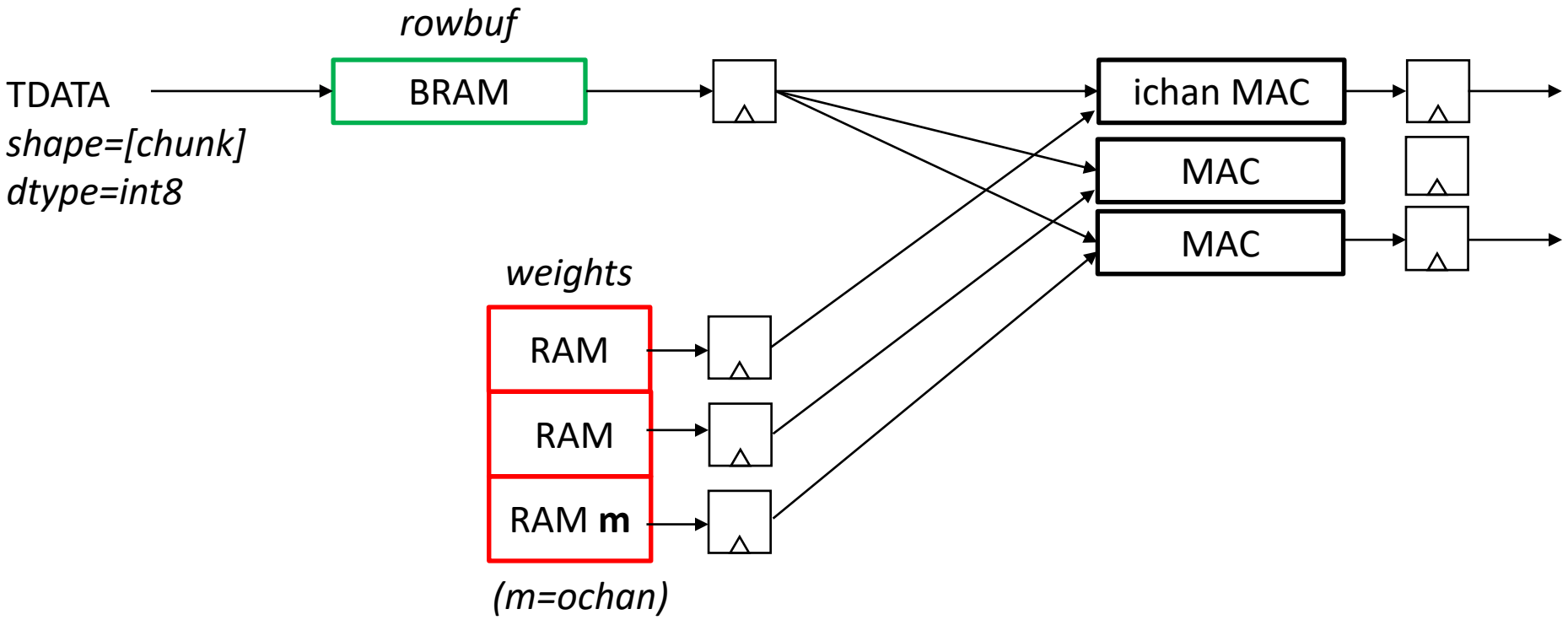








data path interface
n * rowbuf.wa, we
n * rowbuf.ra
n * rowbuf.ra_adjacent
n * rowbuf.muxsel
m * weights.ra
mac_clear, mac_load



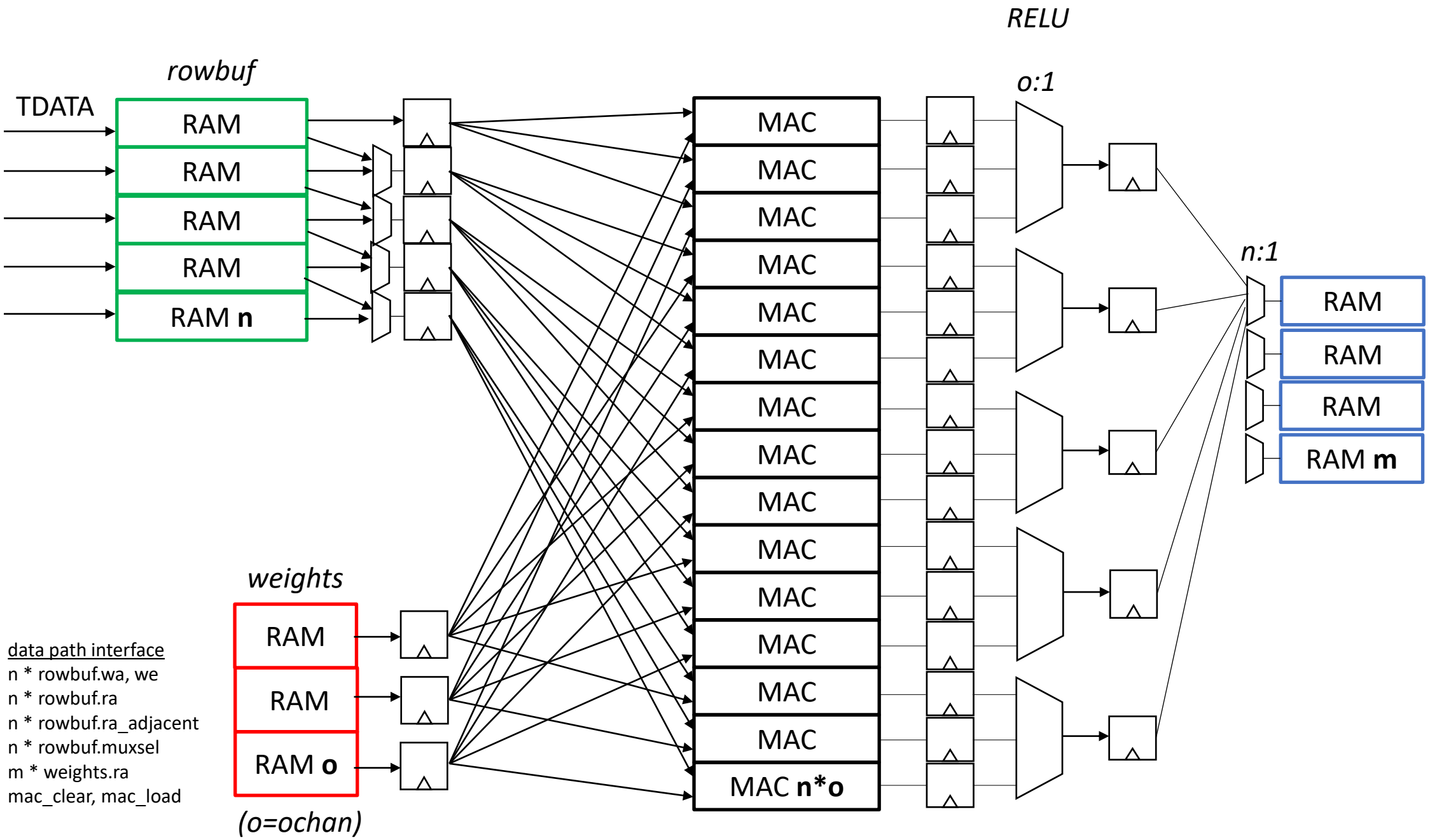
1. serialize flattened tensor
2. send tensor over TDATA in S*dtype size chunks
3. write and read BRAM in S*dtype size chunks

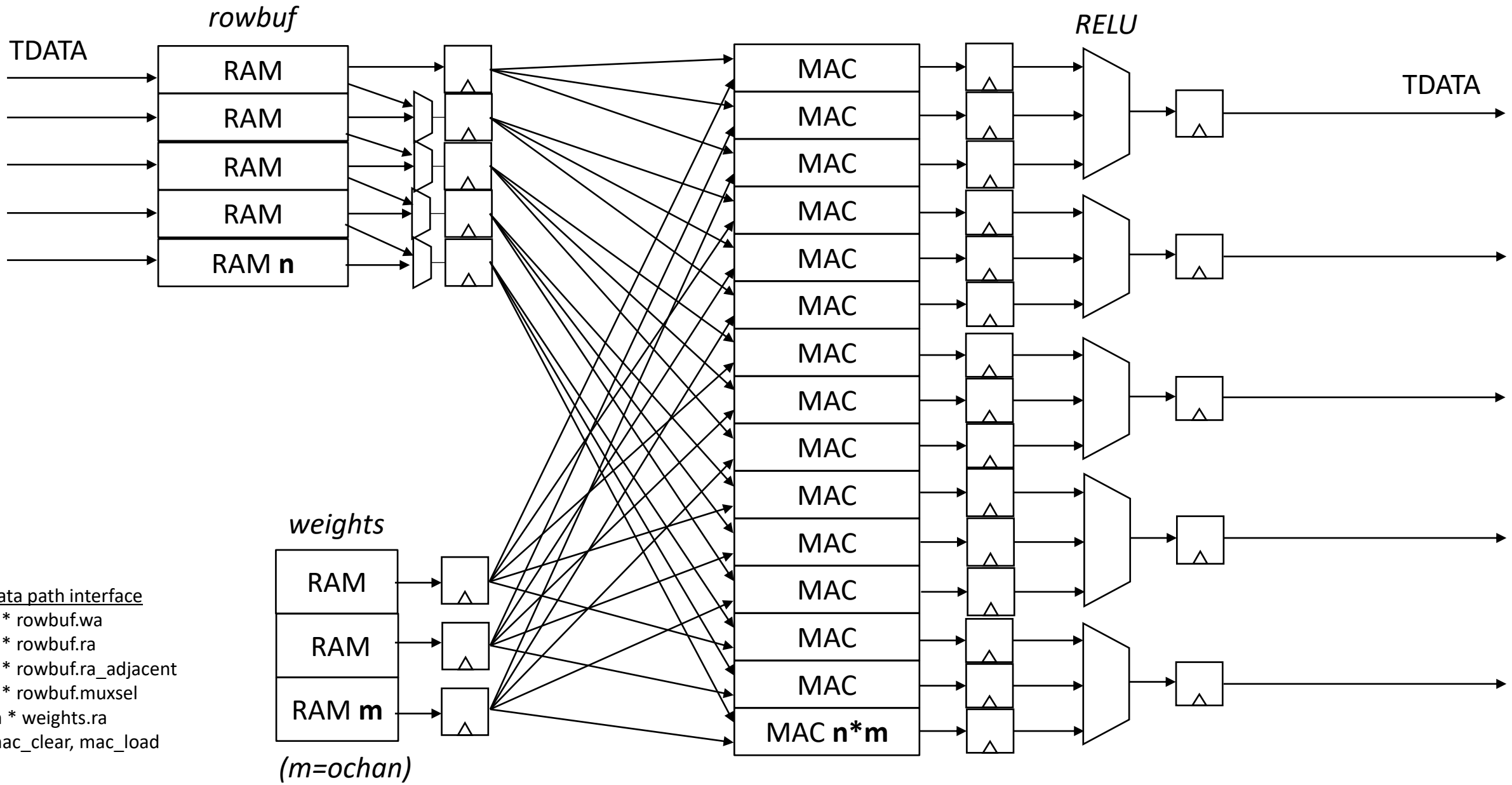
data path interface
n * rowbuf.wa, we
n * rowbuf.ra
n * rowbuf.ra_adjacent
n * rowbuf.muxsel
m * weights.ra
mac_clear, mac_load

fps	289
width	728
height	544
channels	3
clk	500,000,000

layer	iwidth	iheight	ichan	stride	kheight	kwidth	owidth	oheight	ochan	patches/s	MAC/patch	MAC/s	required MAC	row buffer words	weight words	n	n*m	rowbuf depth
1	728	544	3	1	3	3	726	542	32	114,453,248	864	98,887,606,272	198	6,552	864	7	224	936
2	726	542	32	2	3	3	362	270	32	28,246,860	9,216	260,323,061,760	521	69,696	9,216	17	544	4,100
3	362	270	32	1	3	3	360	268	32	27,882,720	9,216	256,967,147,520	514	34,752	9,216	17	544	2,044
4	360	268	32	2	3	3	179	133	32	6,880,223	9,216	63,408,135,168	127	34,560	9,216	4	128	8,640
5	179	133	32	1	3	3	177	131	32	6,701,043	9,216	61,756,812,288	124	17,184	9,216	4	128	4,296
6	177	131	32	2	3	3	88	65	32	1,653,080	9,216	15,234,785,280	31	16,992	9,216	1	32	16,992
7	88	65	32	1	3	3	86	63	32	1,565,802	9,216	14,430,431,232	29	8,448	9,216	1	32	8,448
8	86	63	32	2	3	3	42	31	32	376,278	9,216	3,467,778,048	7	8,256	9,216	1	32	8,256
9	42	31	32	1	3	3	40	29	32	335,240	9,216	3,089,571,840	7	4,032	9,216	1	32	4,032
10	40	29	32	2	3	3	19	14	32	76,874	9,216	708,470,784	2	3,840	9,216	1	32	3,840
11	19	14	32	1	3	3	17	12	32	58,956	9,216	543,338,496	2	1,824	9,216	1	32	1,824
12	17	12	32	2	3	3	8	5	32	11,560	9,216	106,536,960	1	1,632	9,216	1	32	1,632
13	8	5	32	1	3	3	6	3	32	5,202	9,216	47,941,632	1	768	9,216	1	32	768
14	6	3	32	2	3	3	2	1	32	578	9,216	5,326,848	1	576	9,216	1	32	576
15	2	1	32	1	3	3	0	-1	32	-	9,216	-	-	192	9,216	-	-	#DIV/0!
												778,976,944,128	1,565	209,304	129,888	58	1,856	

1. write incoming features from AXI-S to rowbuf[]
2. when a compete row of patches has arrived, begin computing dot products per-inputslice per-chanout
3. n*m features will complete simultaneously, write them sequentially into the reorder buffer

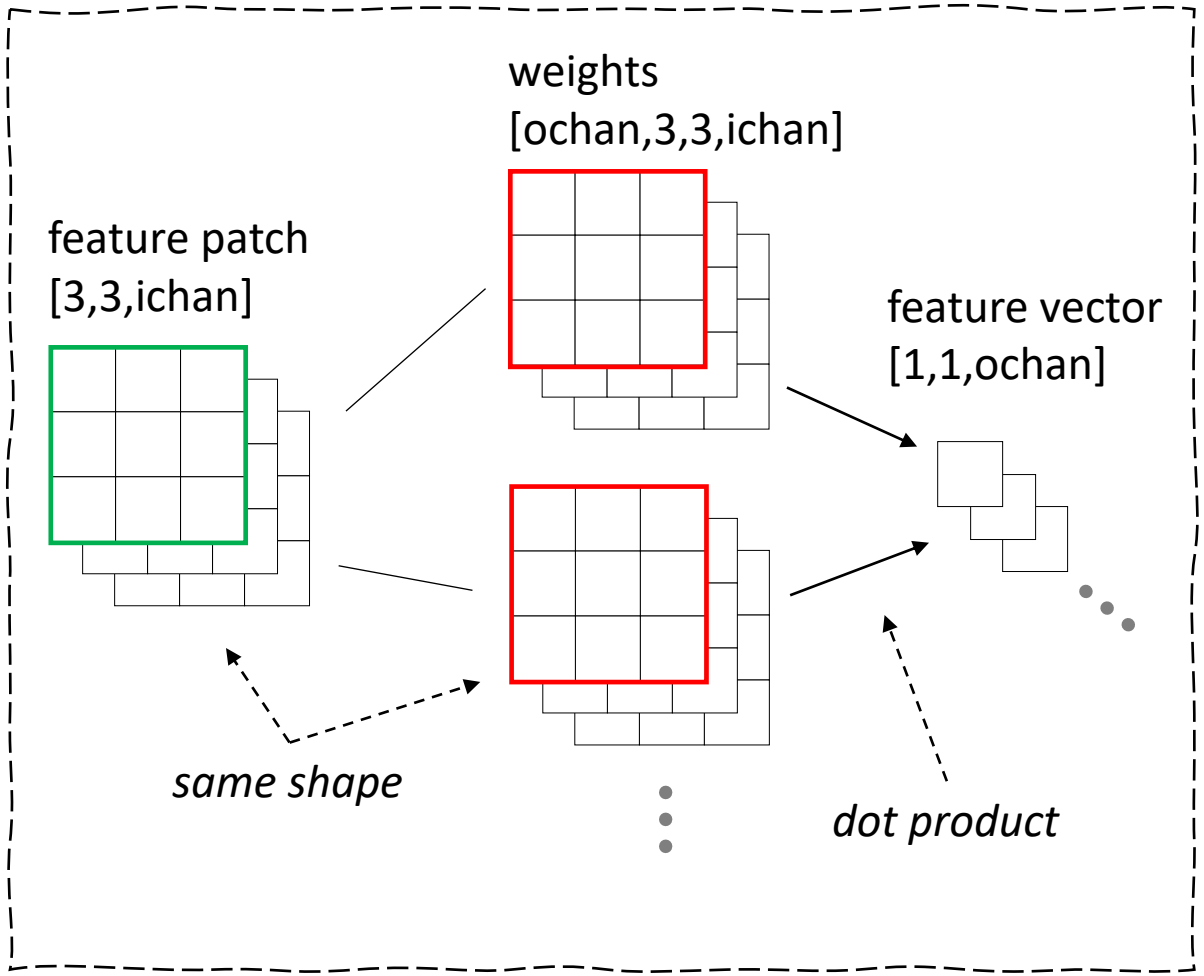

RELU
rowbuf
o:1
TDATA
RAM
RAM
RAM
RAM
RAM n
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*o
n:1
RAM
RAM
RAM
RAM m
weights
RAM
RAM
RAM o
(o=ochan)
data path interface
n * rowbuf.wa, we
n * rowbuf.ra
n * rowbuf.ra_adjacent
n * rowbuf.muxsel
m * weights.ra
mac_clear, mac_load

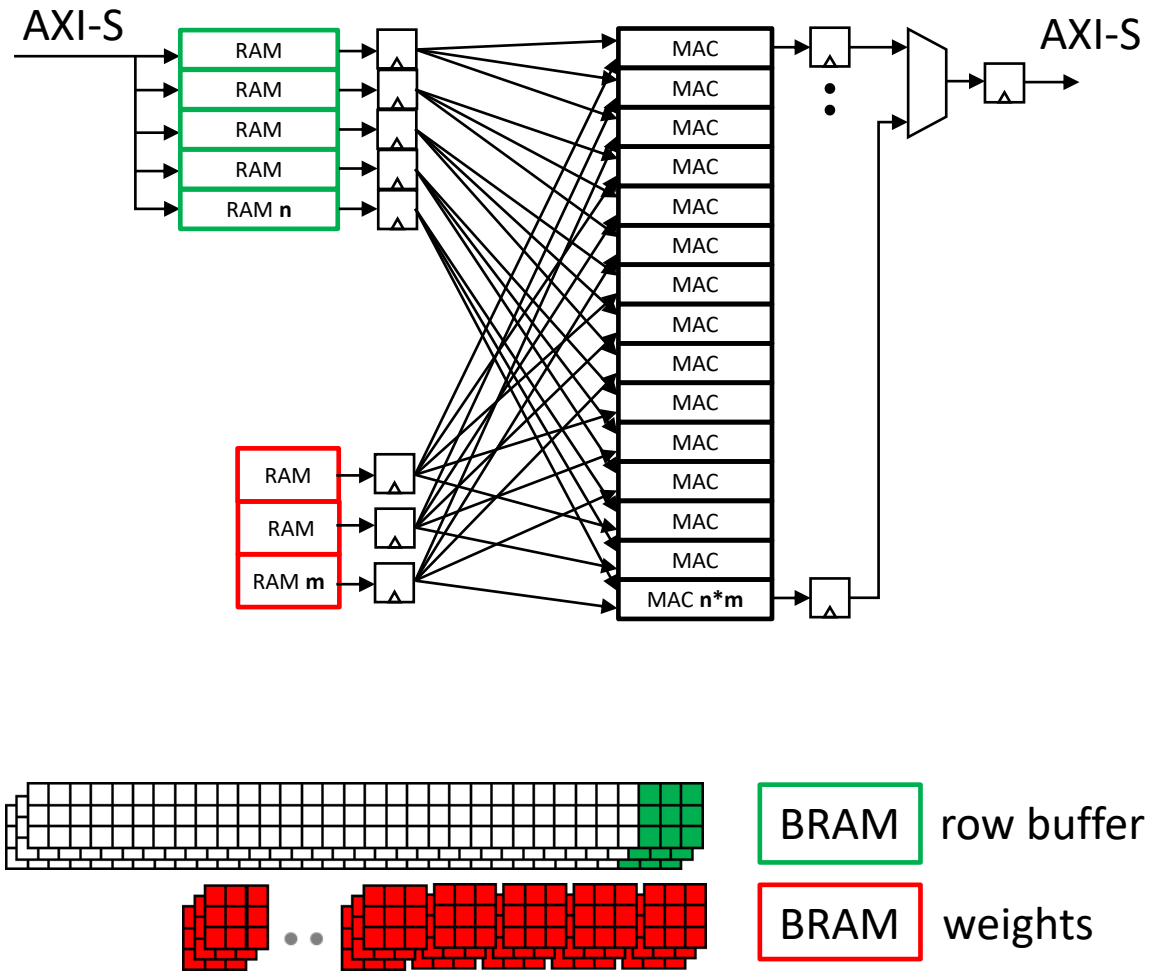


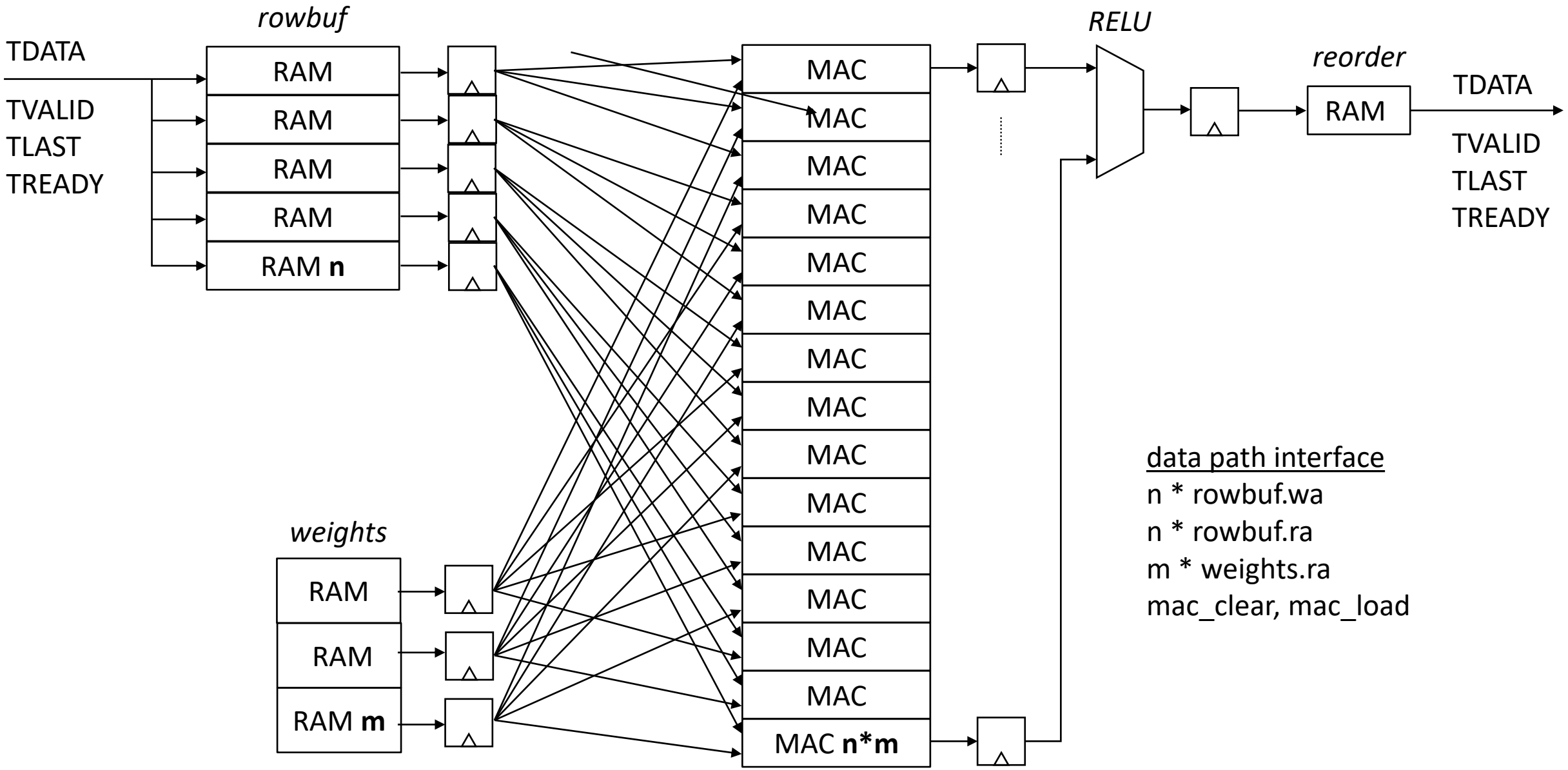
data path interface
n * rowbuf.wa
n * rowbuf.ra
n * rowbuf.ra_adjacent
n * rowbuf.muxsel
m * weights.ra
mac_clear, mac_load

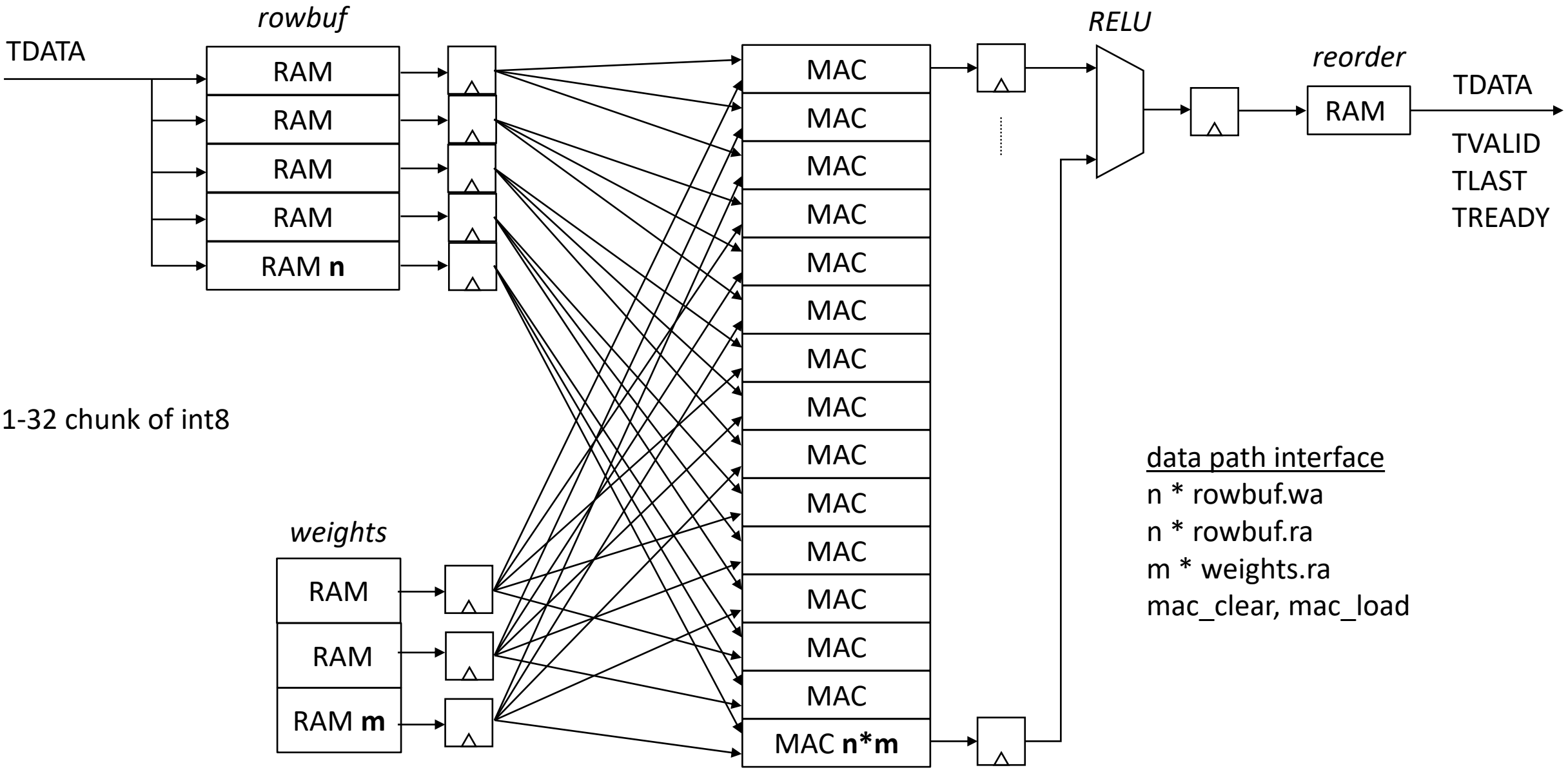
conv2d.v

CNN dot product using FPGA

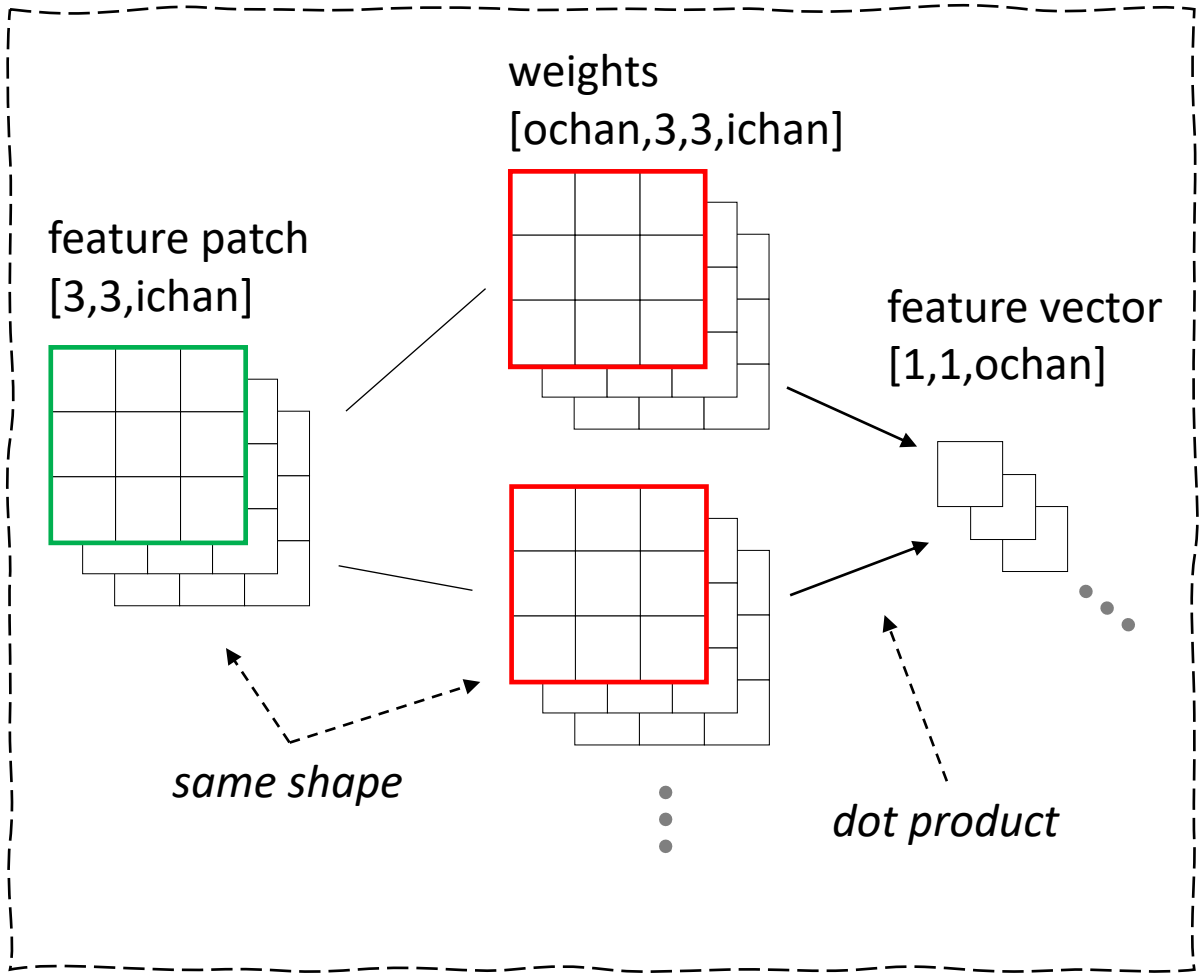


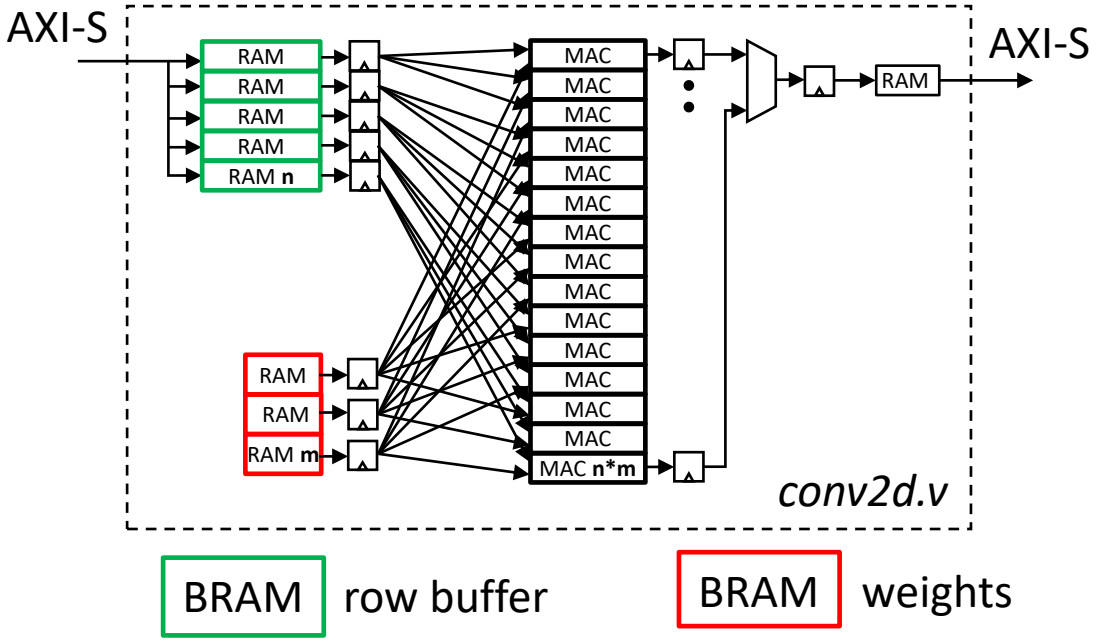


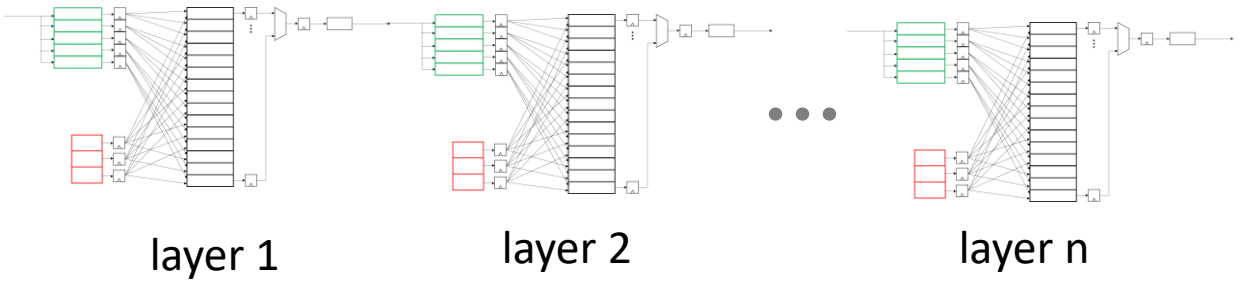

rowbuf
TDATA
TVALID
TLAST
TREADY
RAM
RAM
RAM
RAM
RAM n
weights
RAM
RAM
RAM m
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*m
RELU
reorder
RAM
TDATA
TVALID
TLAST
TREADY
data path interface
n * rowbuf.wa
n * rowbuf.ra
m * weights.ra
mac_clear, mac_load

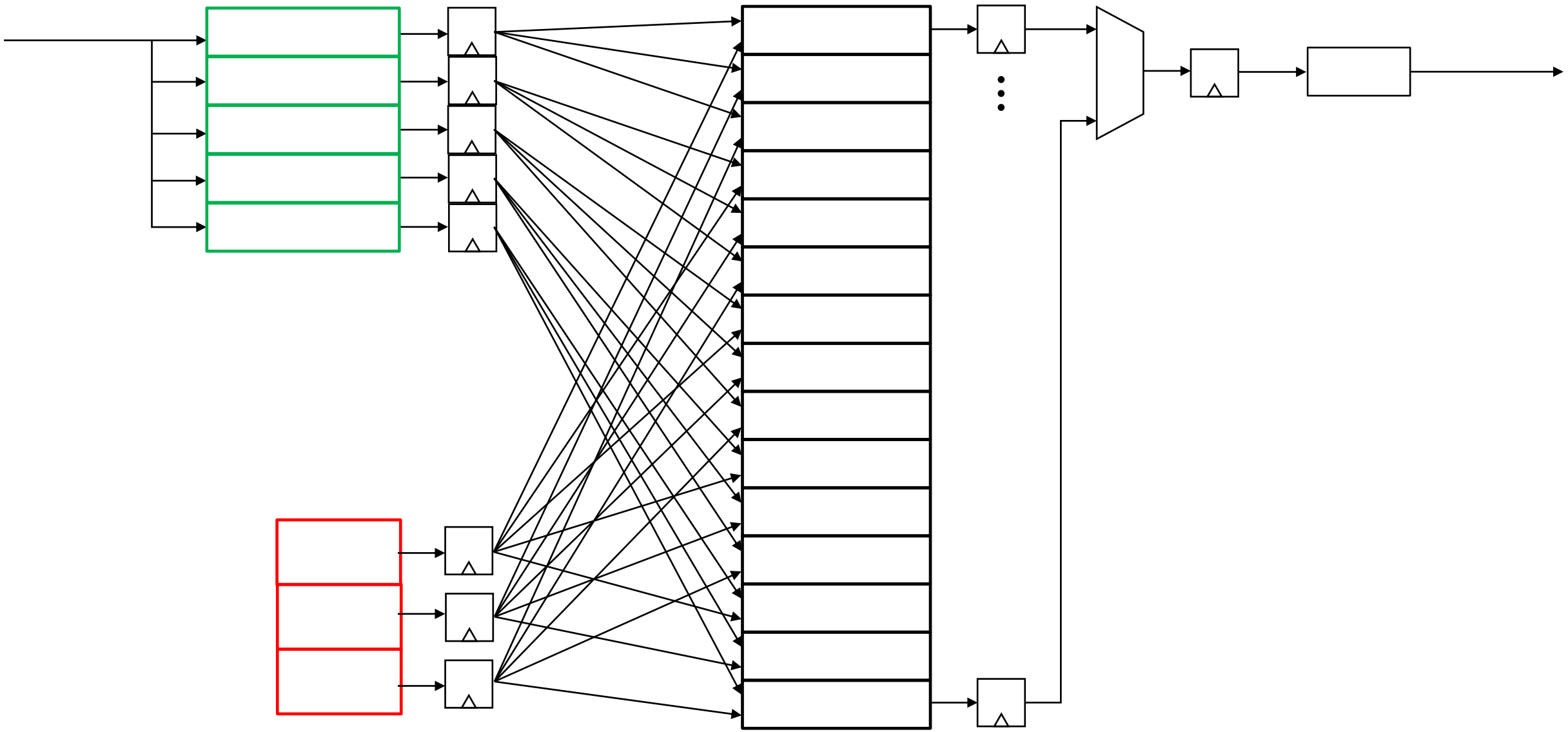

rowbuf
TDATA
RAM
RAM
RAM
RAM
RAM n
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*m
RELU
reorder
RAM
TDATA
TVALID
TLAST
TREADY
1-32 chunk of int8
weights
RAM
RAM
RAM m
data path interface
n * rowbuf.wa
n * rowbuf.ra
m * weights.ra
mac_clear, mac_load

CNN dot product using FPGA

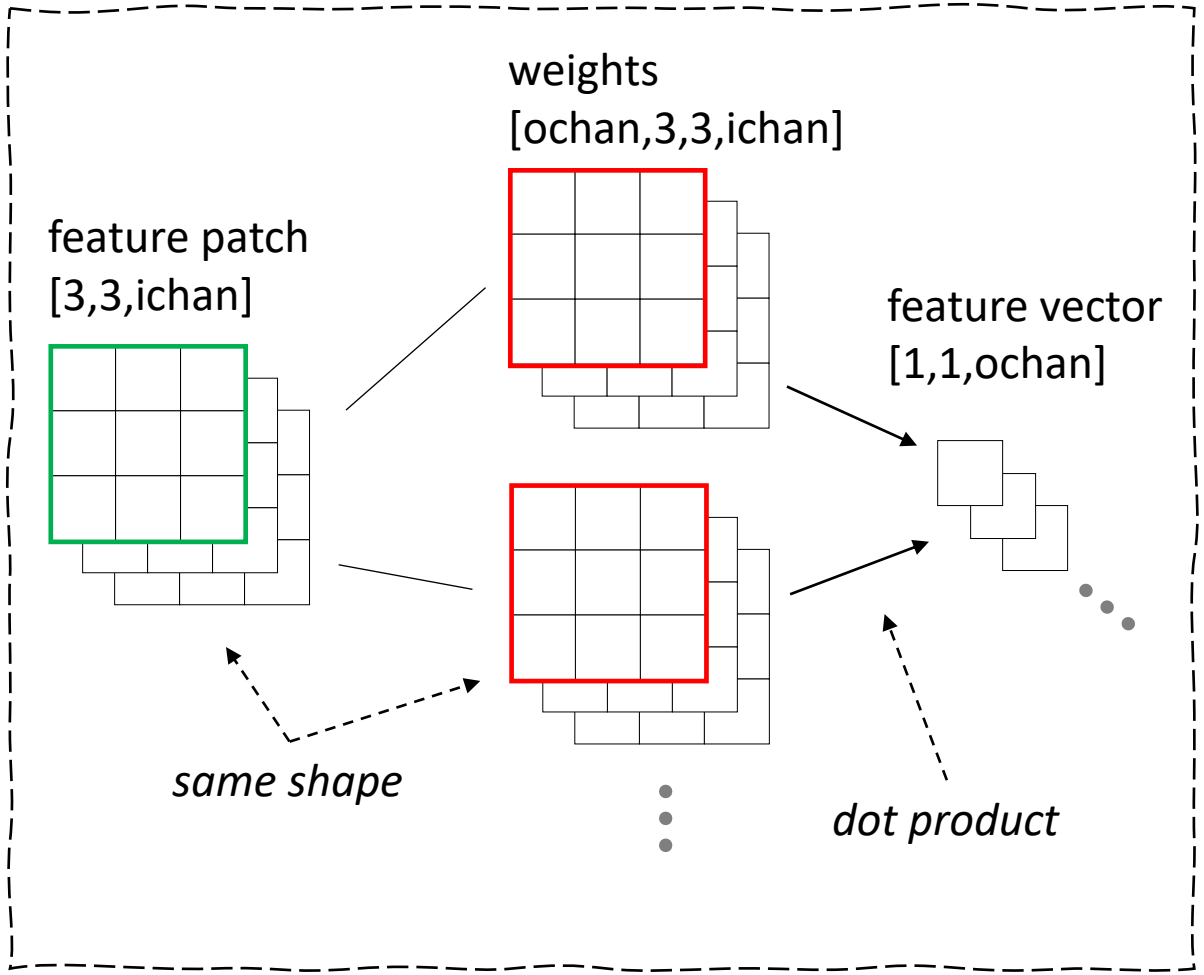


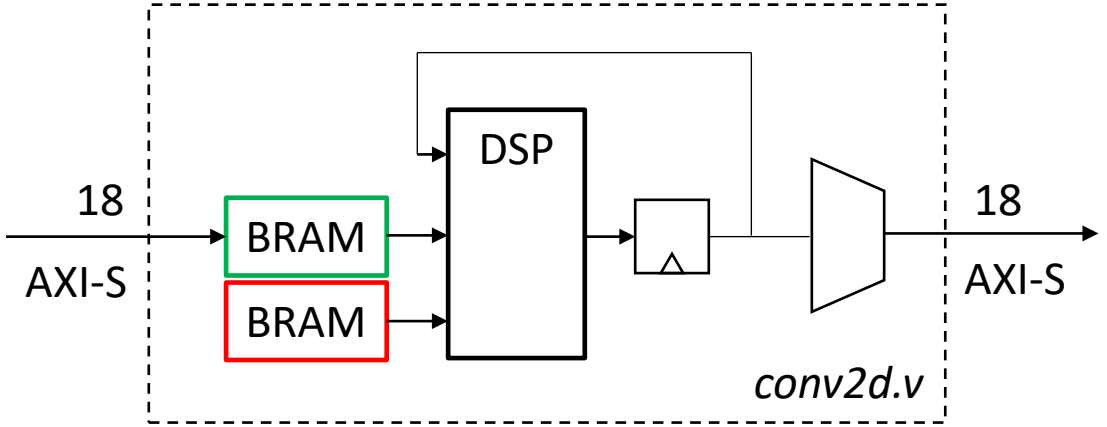


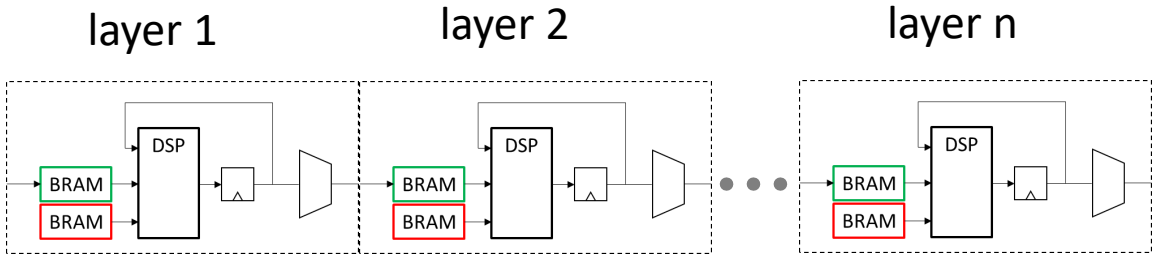


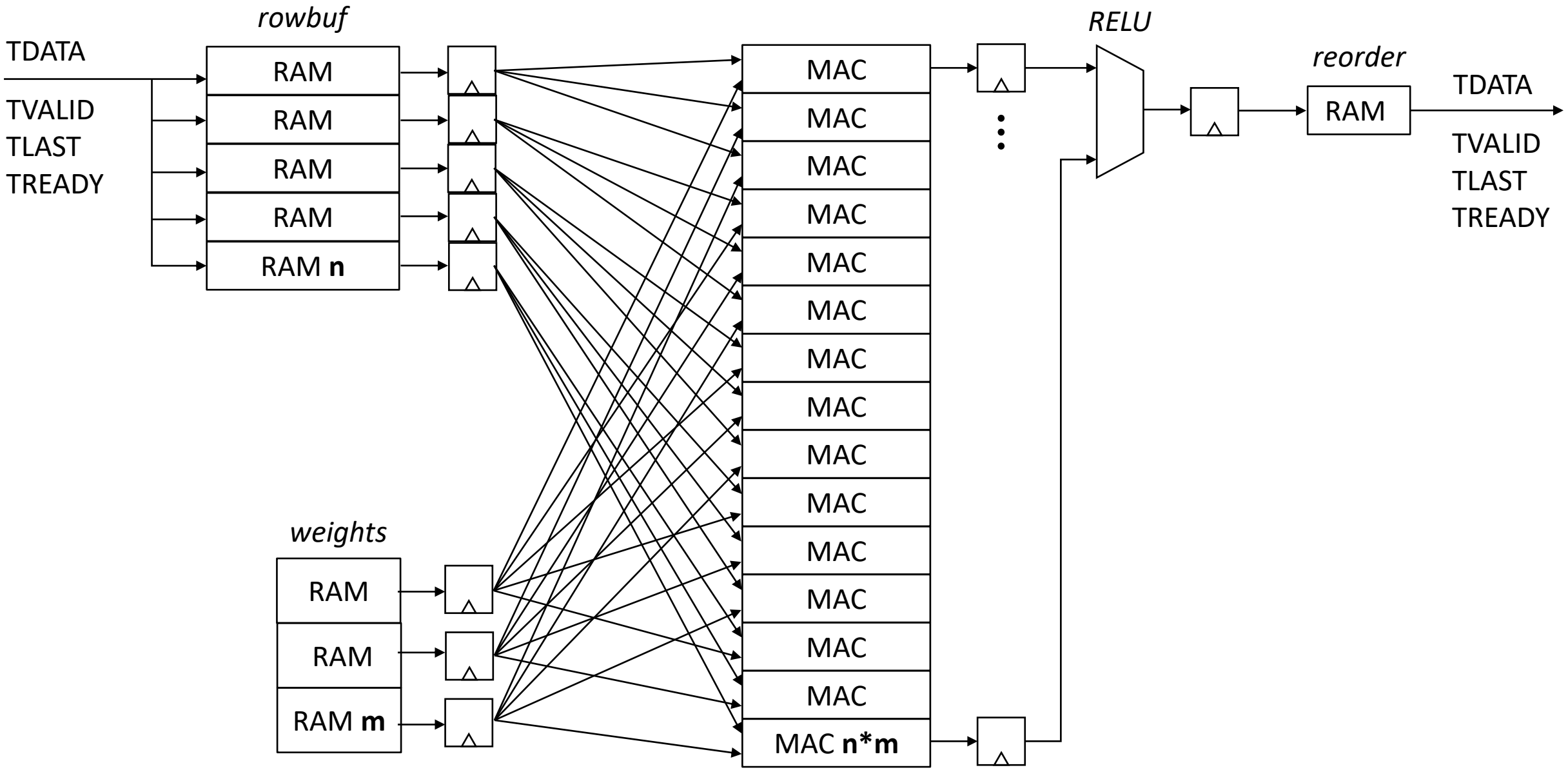


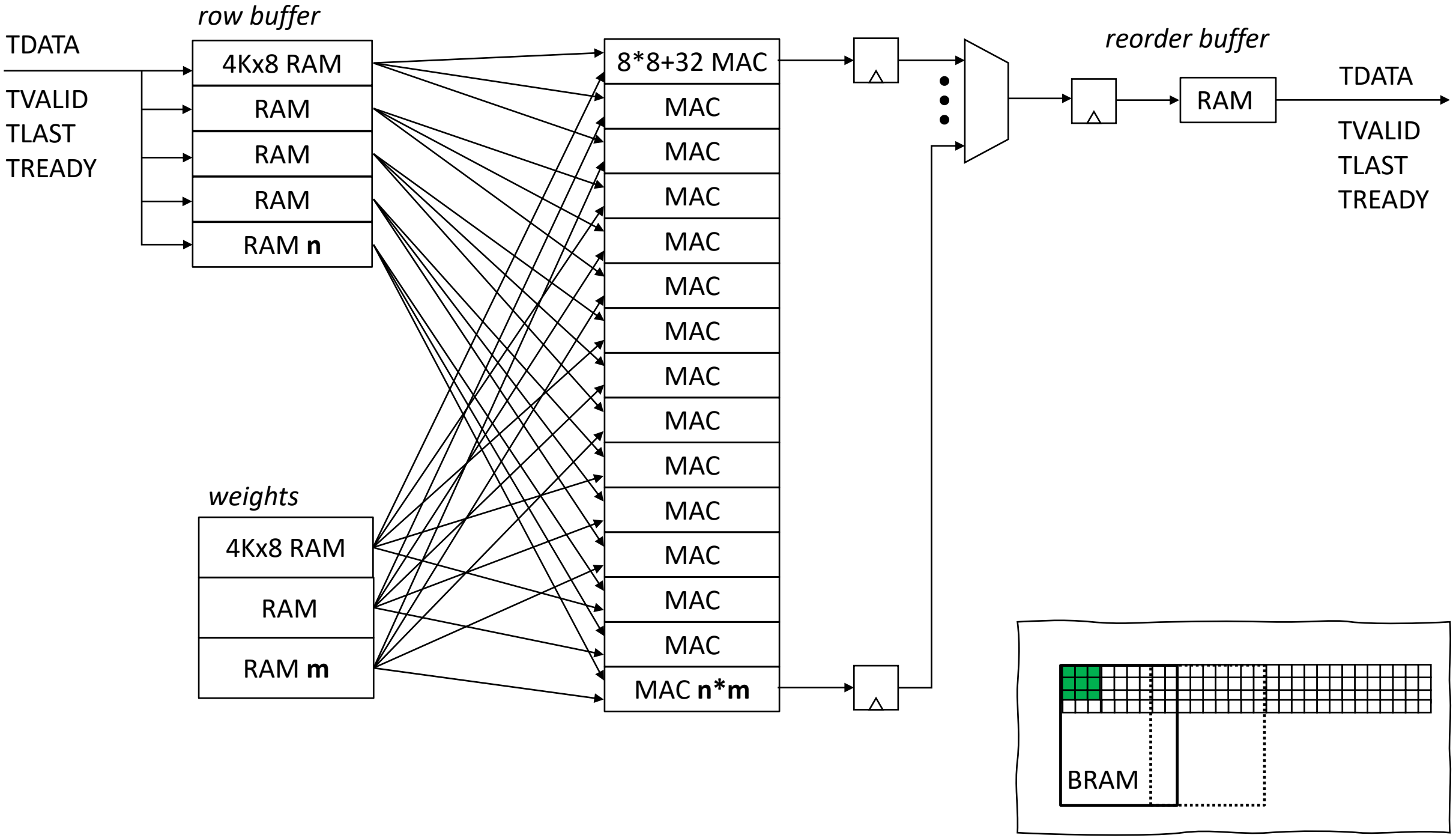
CNN dot product using FPGA

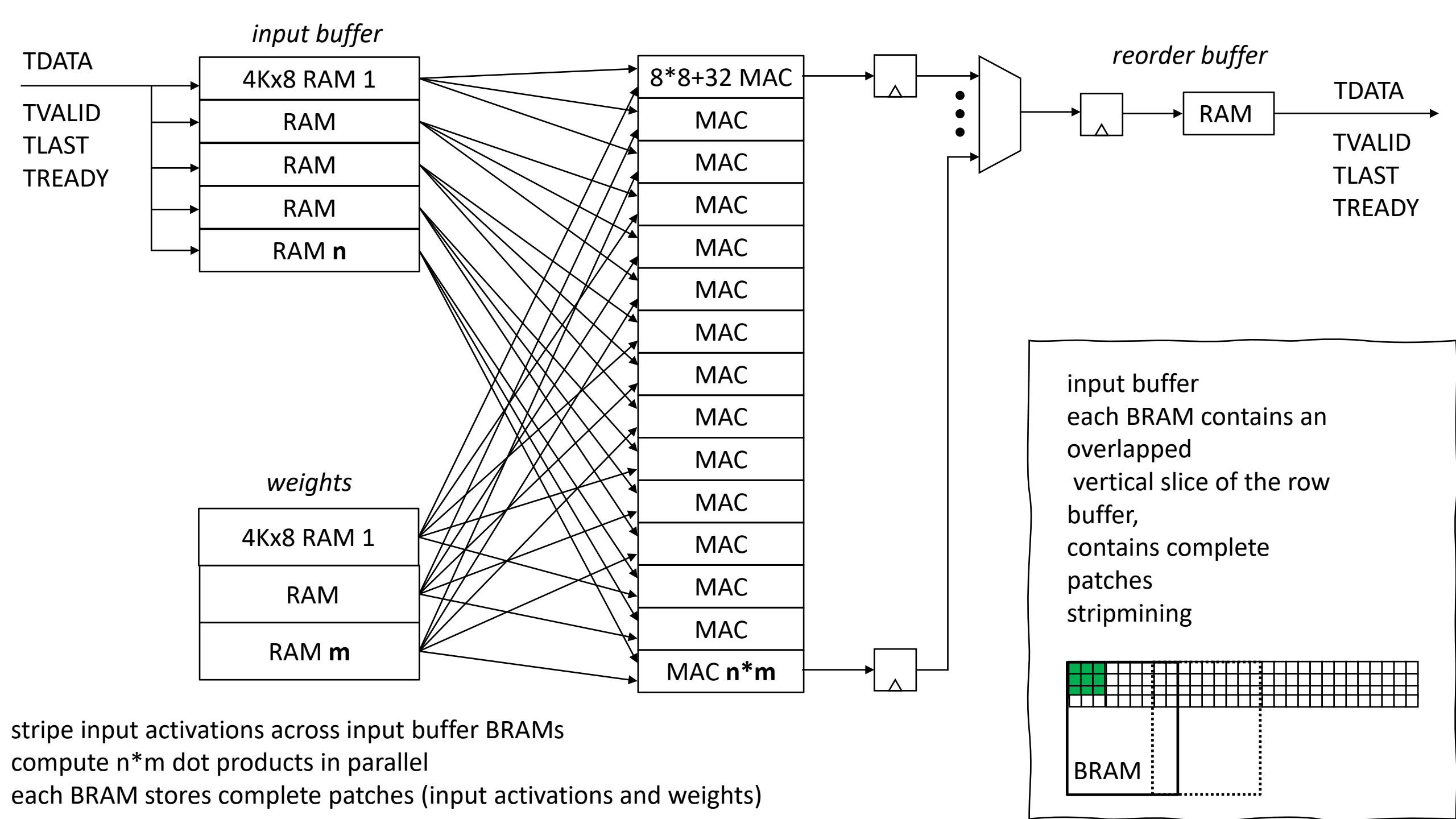




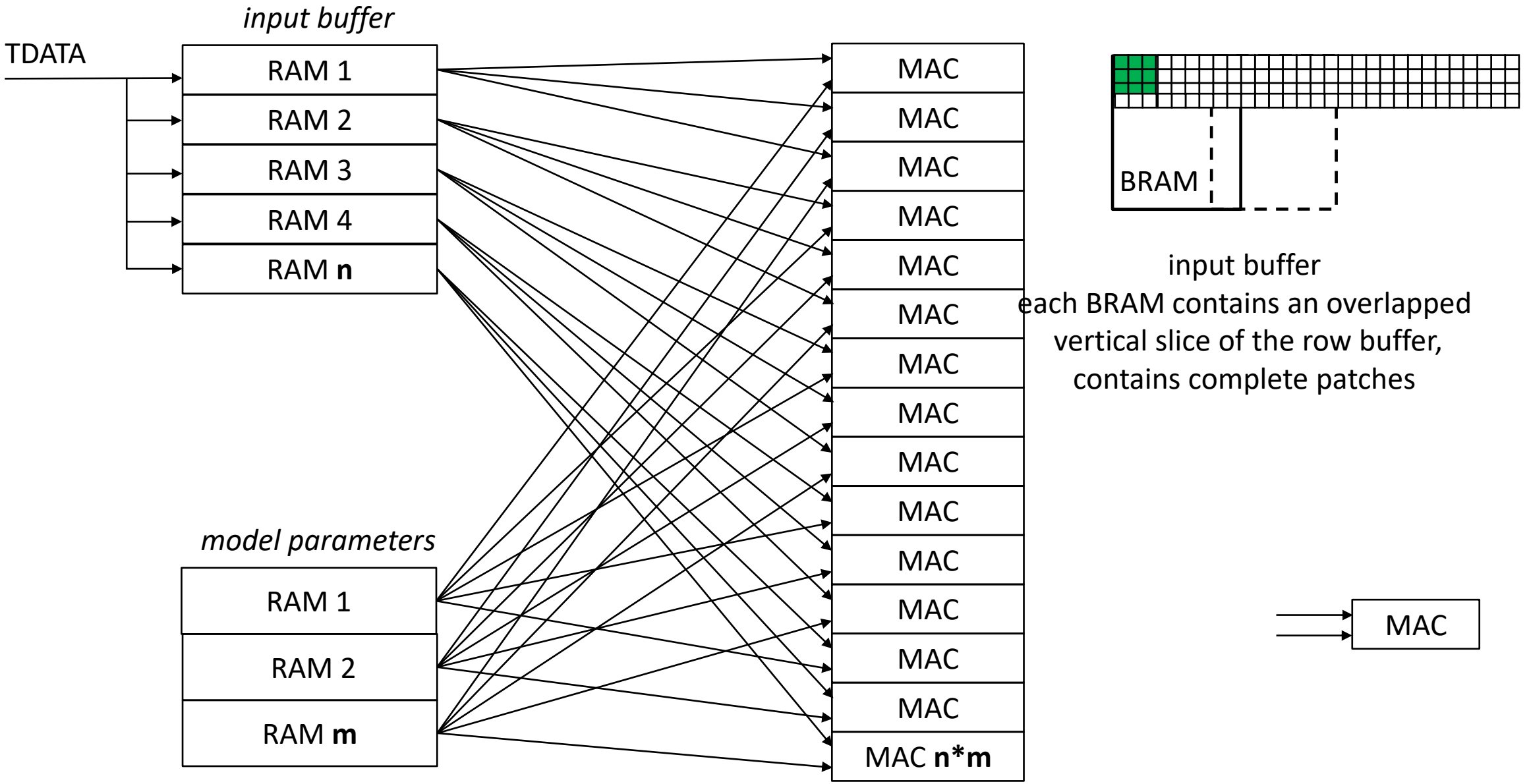


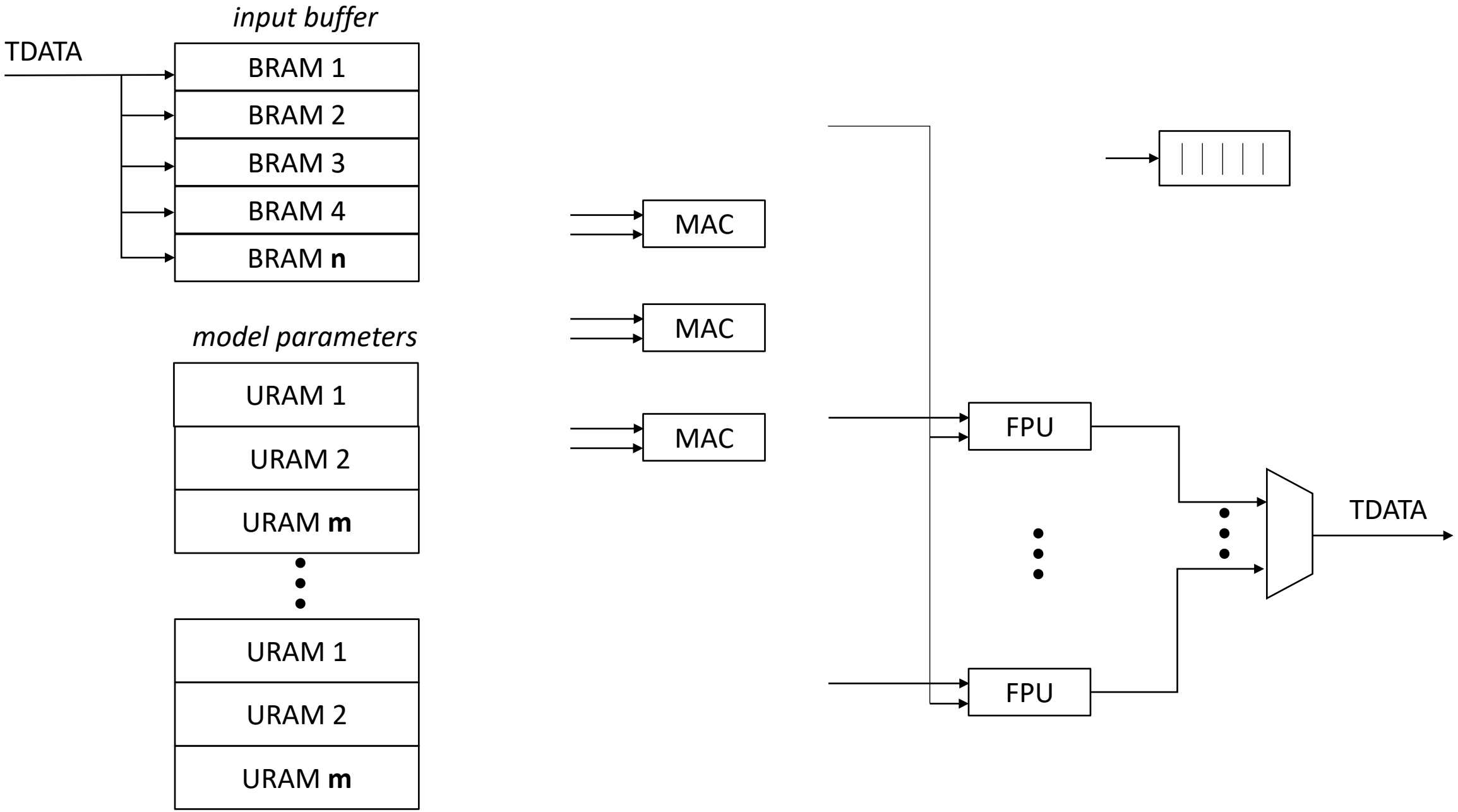

rowbuf
TDATA
TVALID
TLAST
TREADY
RAM
RAM
RAM
RAM
RAM n
weights
RAM
RAM
RAM m
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*m
RELU
reorder
RAM
TDATA
TVALID
TLAST
TREADY


row buffer
TDATA
TVALID
TLAST
TREADY
4Kx8 RAM
RAM
RAM
RAM
RAM n
weights
4Kx8 RAM
RAM
RAM m
8*8+32 MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*m
reorder buffer
RAM
TDATA
TVALID
TLAST
TREADY
BRAM



stripe input activations across input buffer BRAMs
compute n*m dot products in parallel
each BRAM stores complete patches (input activations and weights)


input buffer
TDATA
RAM 1
RAM 2
RAM 3
RAM 4
RAM n
model parameters
RAM 1
RAM 2
RAM m
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC
MAC n*m
BRAM
input buffer
each BRAM contains an overlapped
vertical slice of the row buffer,
contains complete patches
MAC
compute n*m dot products in parallel


input buffer
TDATA
BRAM 1
BRAM 2
BRAM 3
BRAM 4
BRAM n
model parameters
URAM 1
URAM 2
URAM m
URAM 1
URAM 2
URAM m
MAC
MAC
MAC
FPU
FPU
TDATA

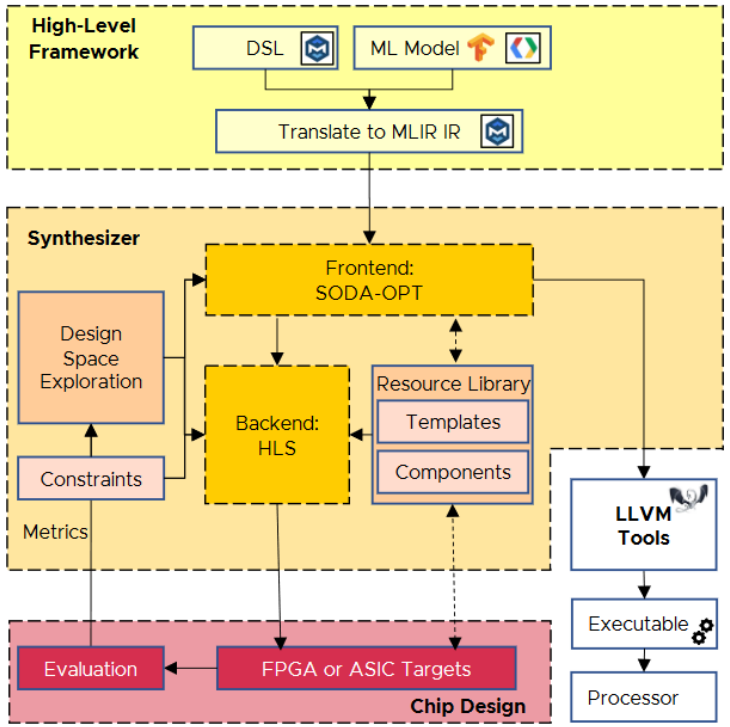
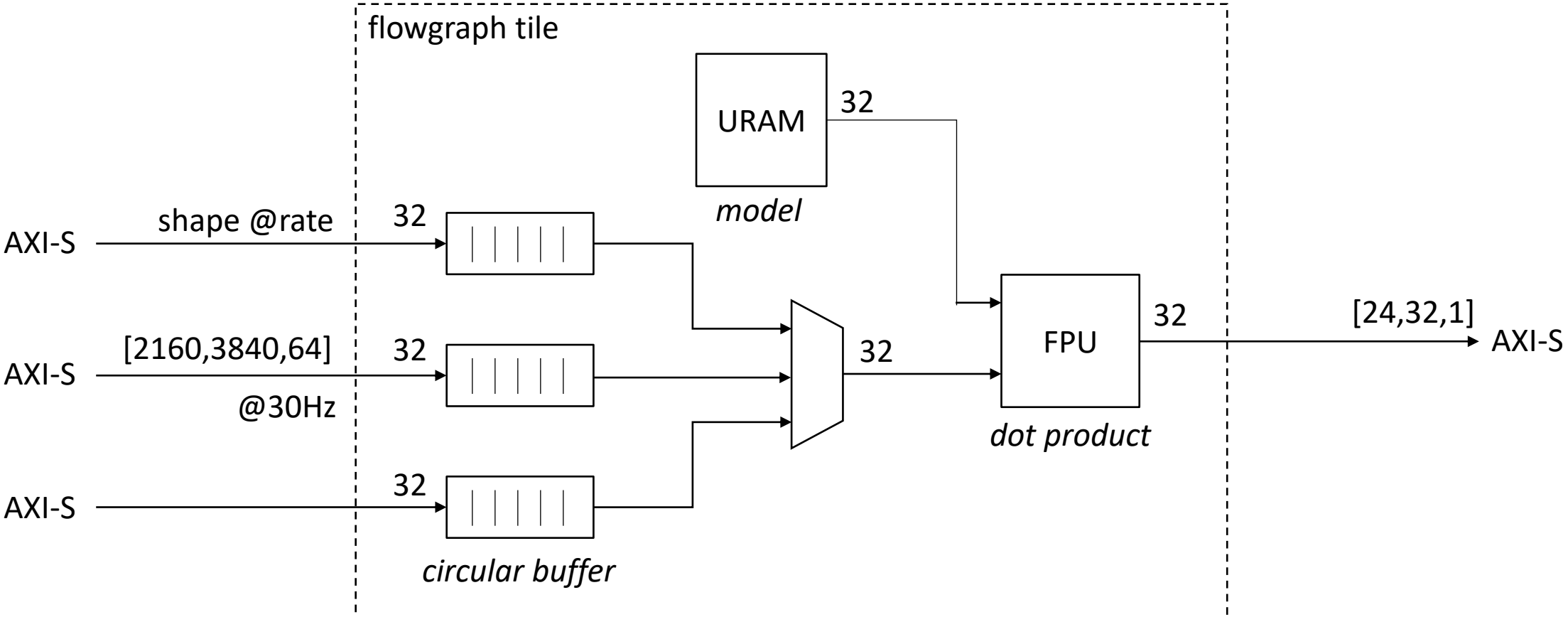
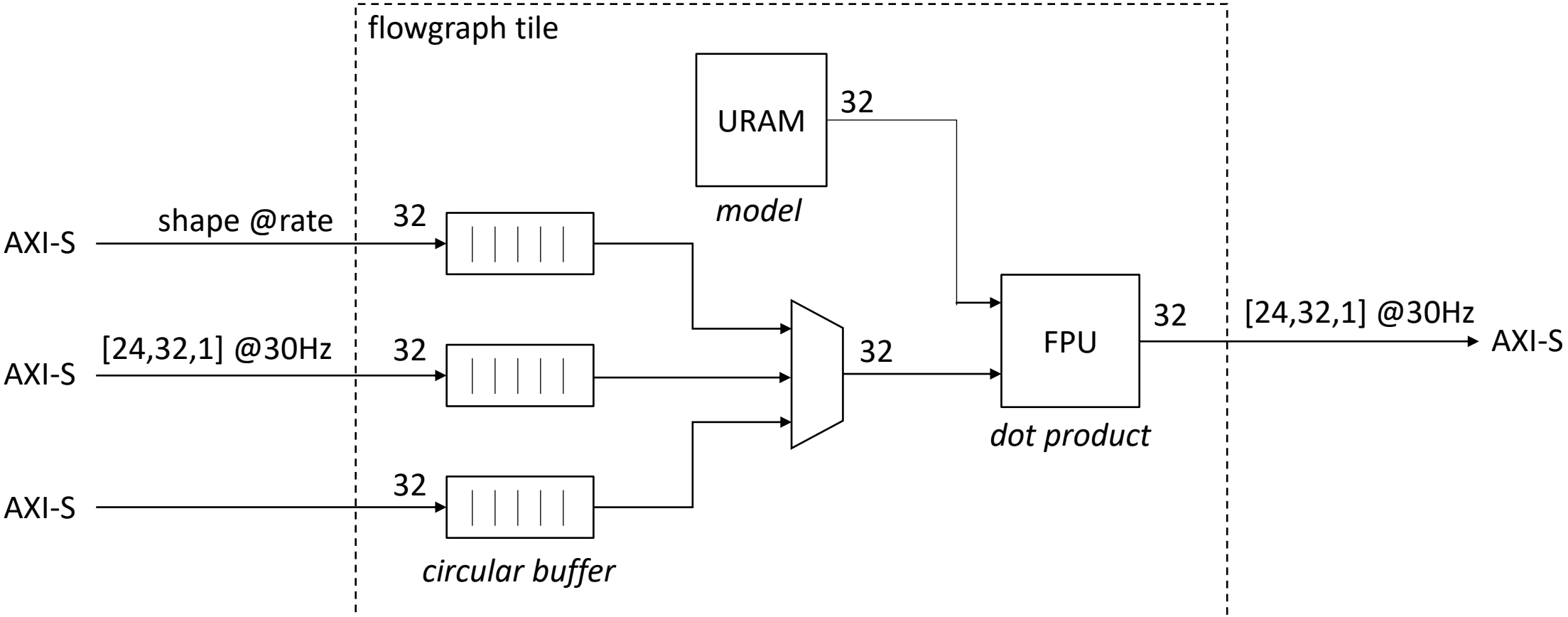
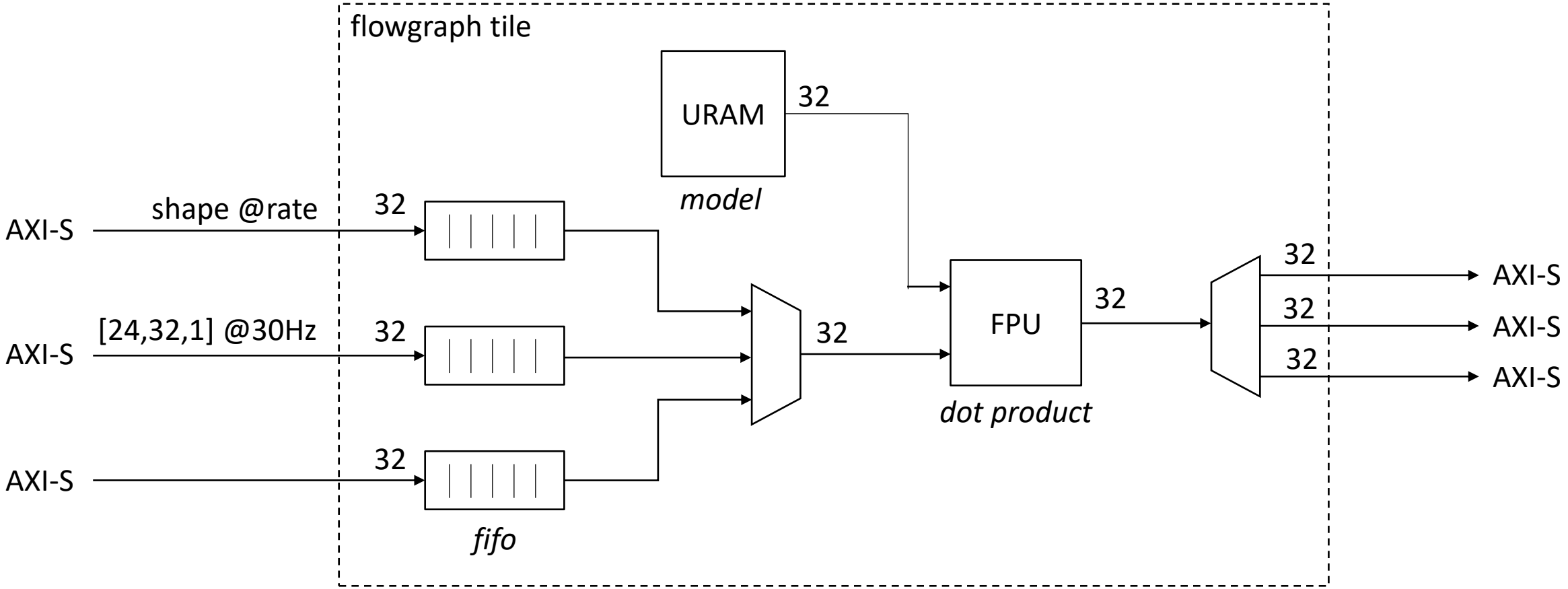


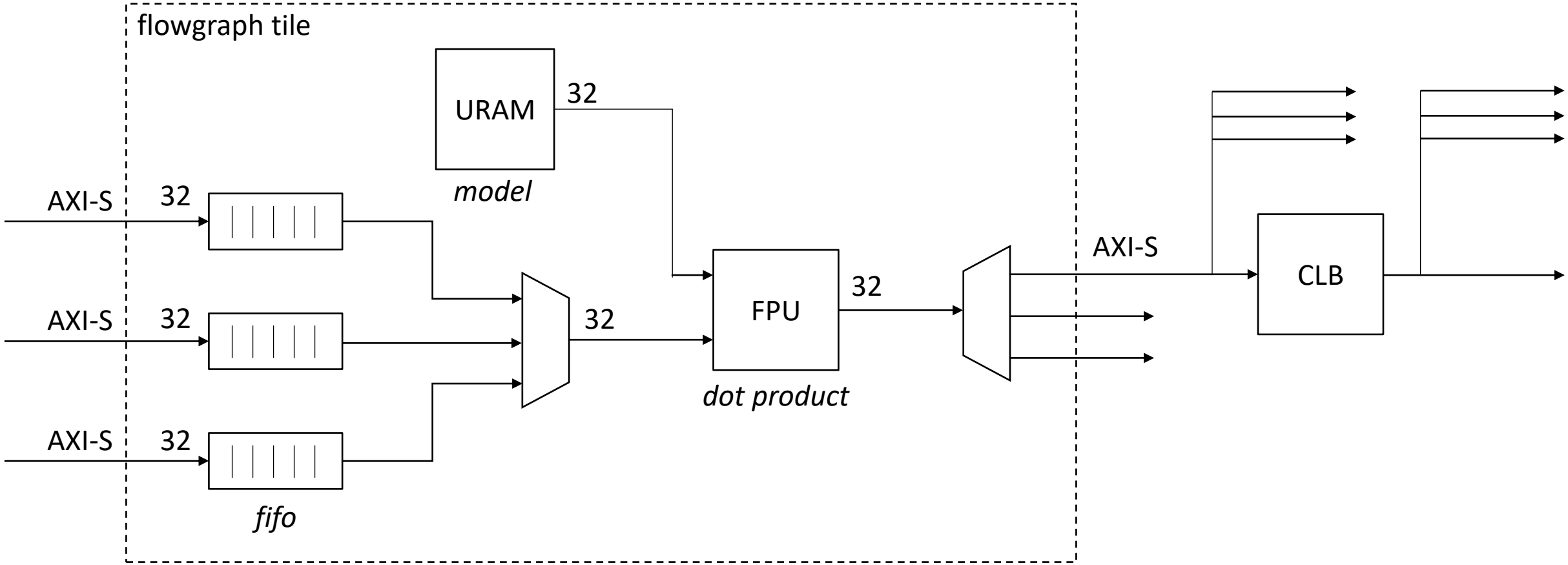
Fig. 1: The SODA Synthesizer.

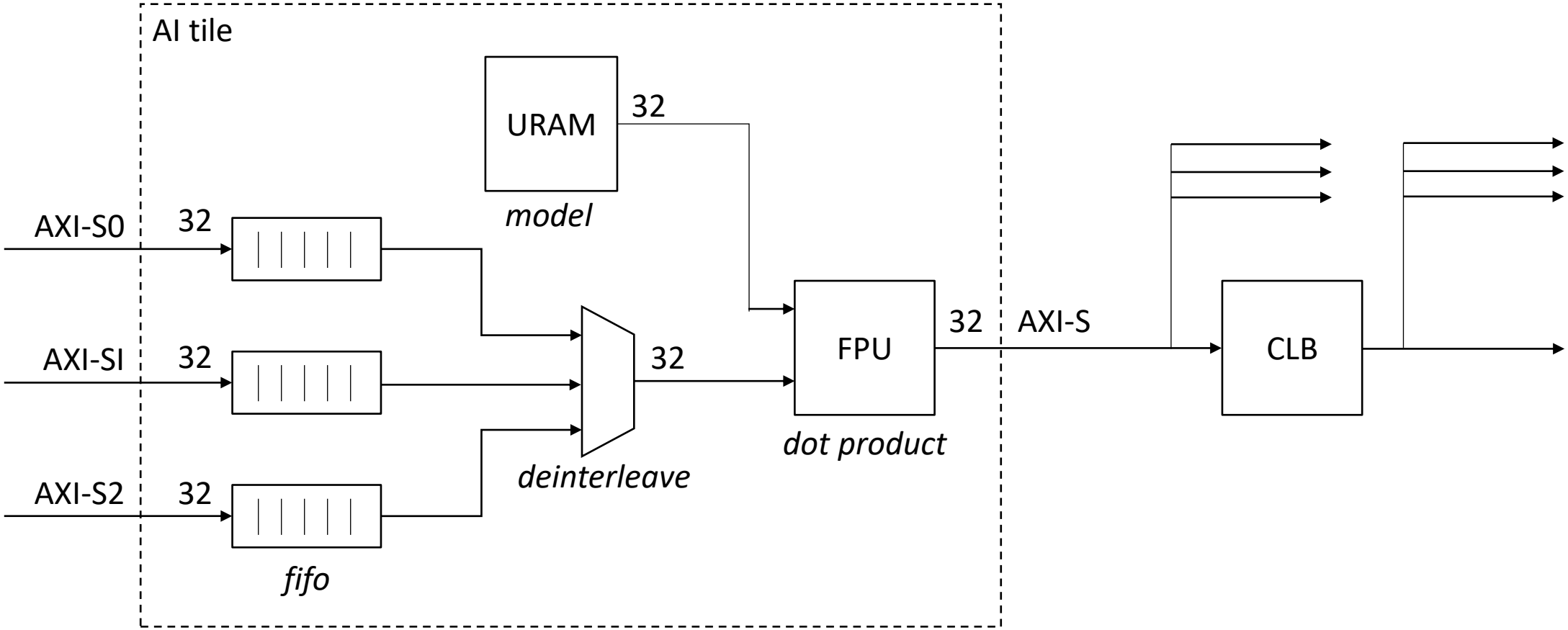
← TFLite to MLIR, starting with Conv2D

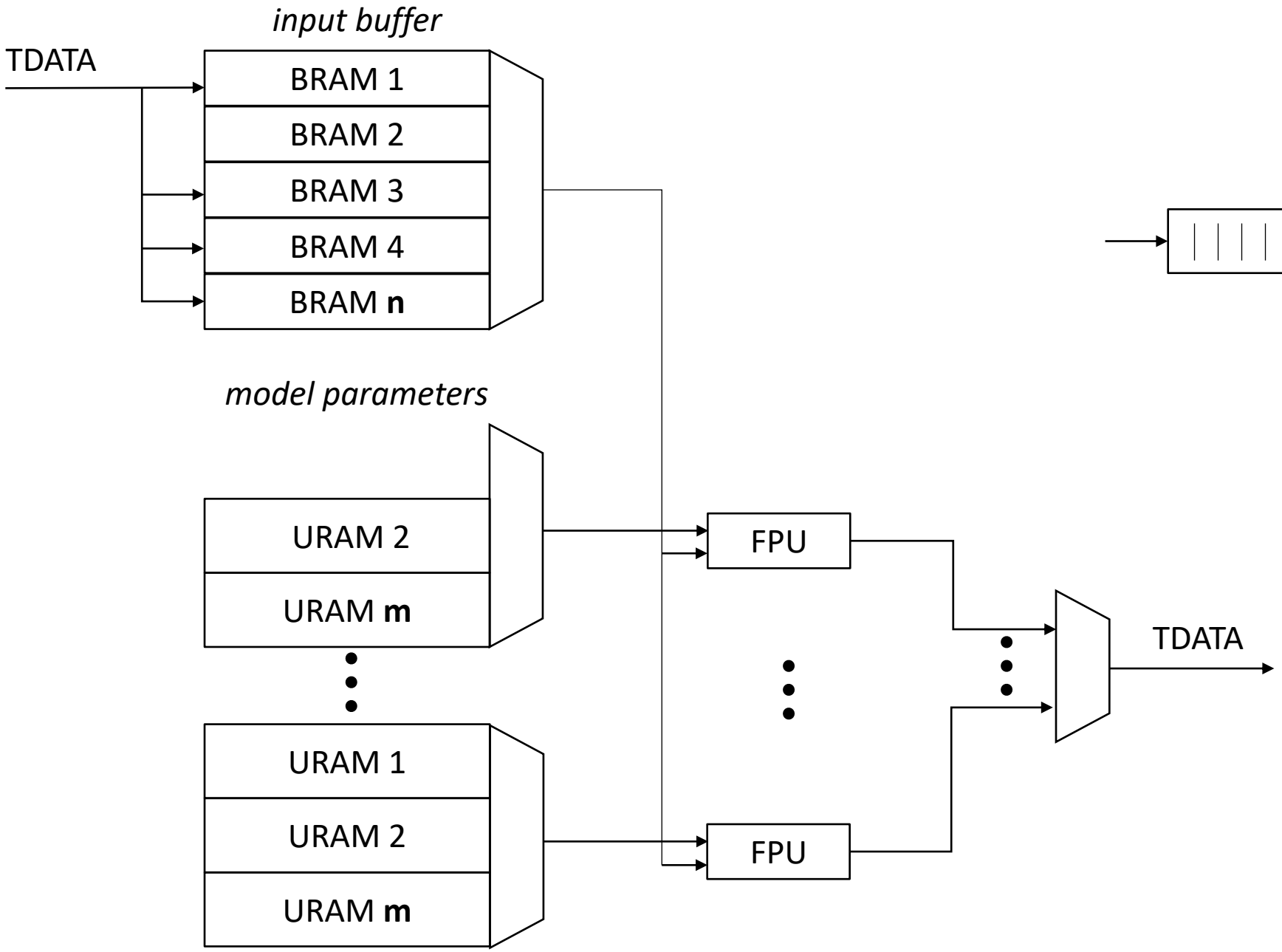

flowgraph tile
URAM
32
model
AXI-S
shape @rate
32
AXI-S
[2160,3840,64]
@30Hz
32
AXI-S
32
circular buffer
32
32
FPU
dot product
32
[24,32,1]
AXI-S


flowgraph tile
URAM
32
model
AXI-S
shape @rate
32
AXI-S
[24,32,1] @30Hz
32
32
FPU
32
[24,32,1] @30Hz
AXI-S
dot product
AXI-S
32
circular buffer

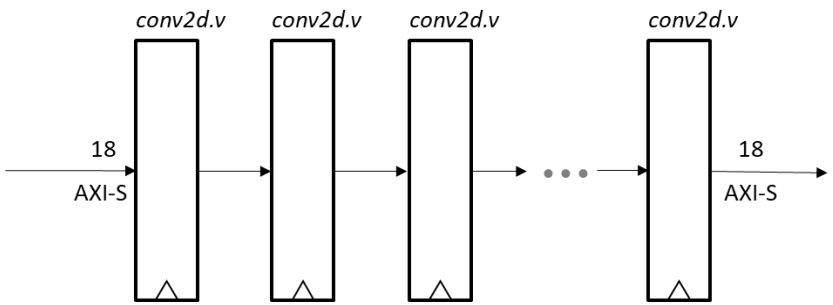

flowgraph tile
URAM
32
model
AXI-S
shape @rate
32
AXI-S
[24,32,1] @30Hz
32
AXI-S
32
fifo
32
FPU
32
dot product
32
32
32
AXI-S
AXI-S
AXI-S

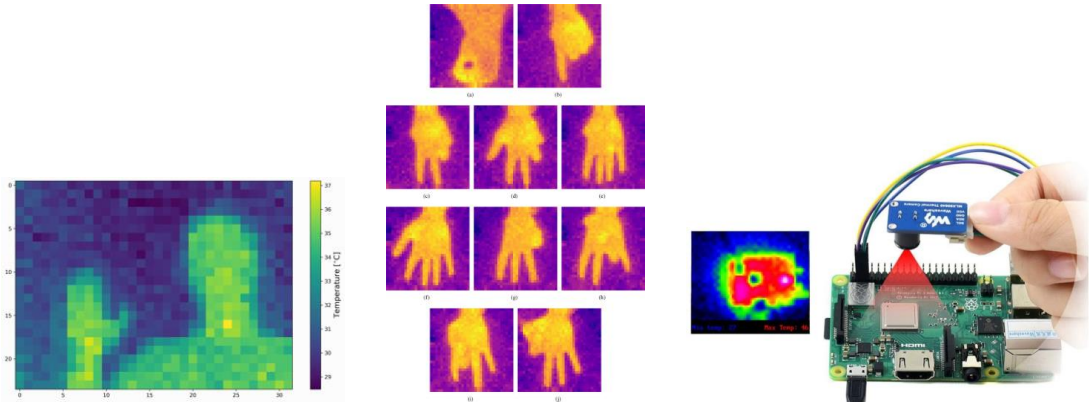

flowgraph tile
URAM
32
model
AXI-S
32
AXI-S
32
AXI-S
32
fifo
32
FPU
32
dot product
AXI-S
CLB


AI tile
URAM
32
model
AXI-S0
32
AXI-SI
32
AXI-S2
32
32
deinterleave
FPU
32
dot product
AXI-S
CLB
fifo


input buffer
TDATA
BRAM 1
BRAM 2
BRAM 3
BRAM 4
BRAM n
model parameters
URAM 2
URAM m
URAM 1
URAM 2
URAM m
FPU
FPU
TDATA

- conv2d.v is a Tensorflow compatible CNN layer implementation
- Each layer in the model is evaluated in parallel using pipelining
- Weights are stored in on-chip BRAM for high performance
- Can be used stand alone or as a front end to a larger model



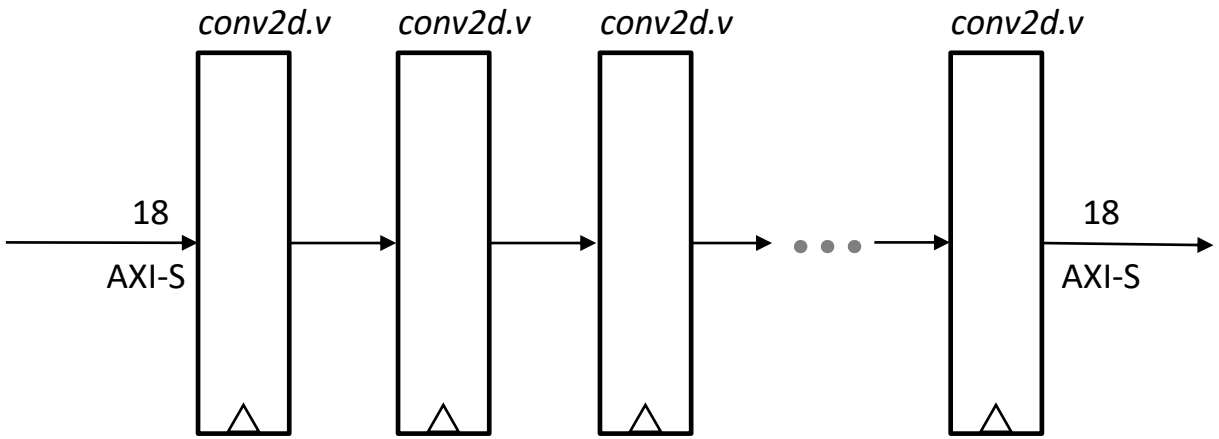


Real time classification demo for Gemini
- Code complete, runs at 200MHz using Artix-7/Vivado
- Next step is functional verification using MNIST

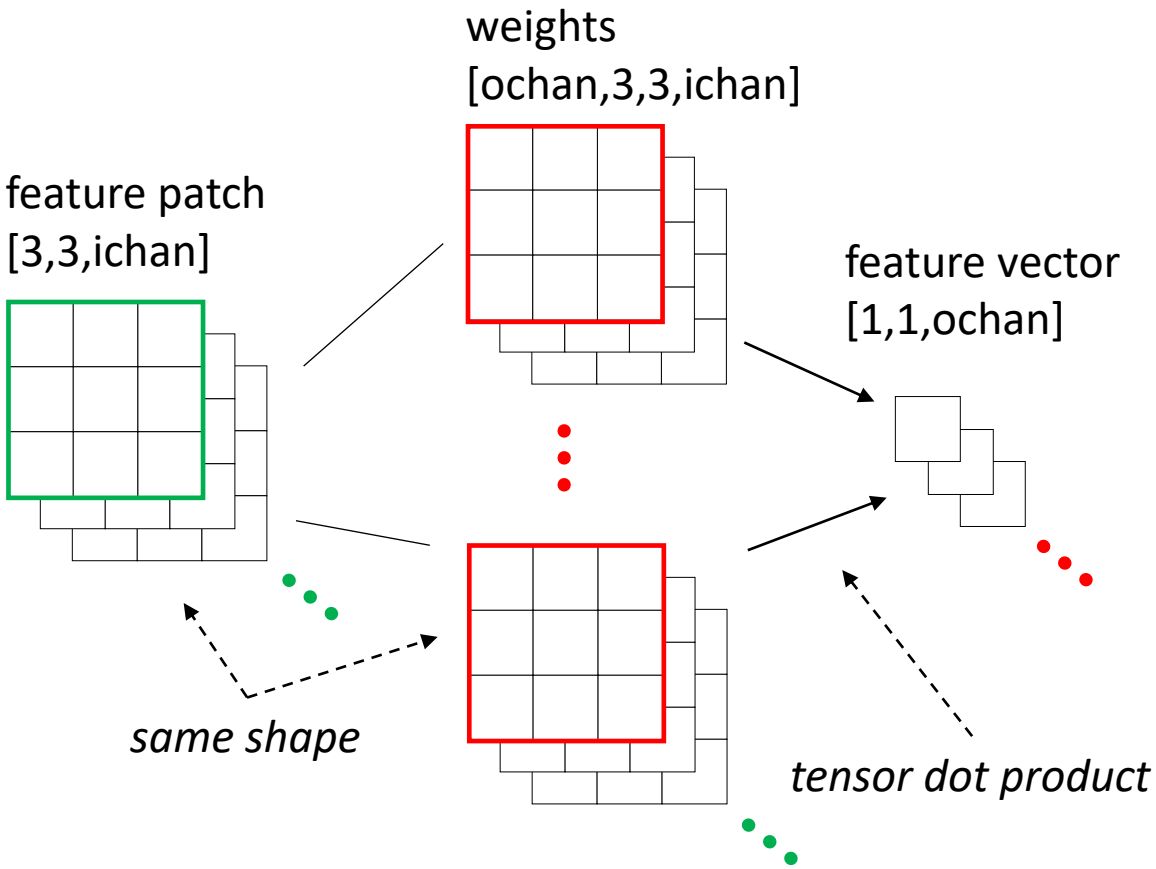
conv2d.v

Objectives

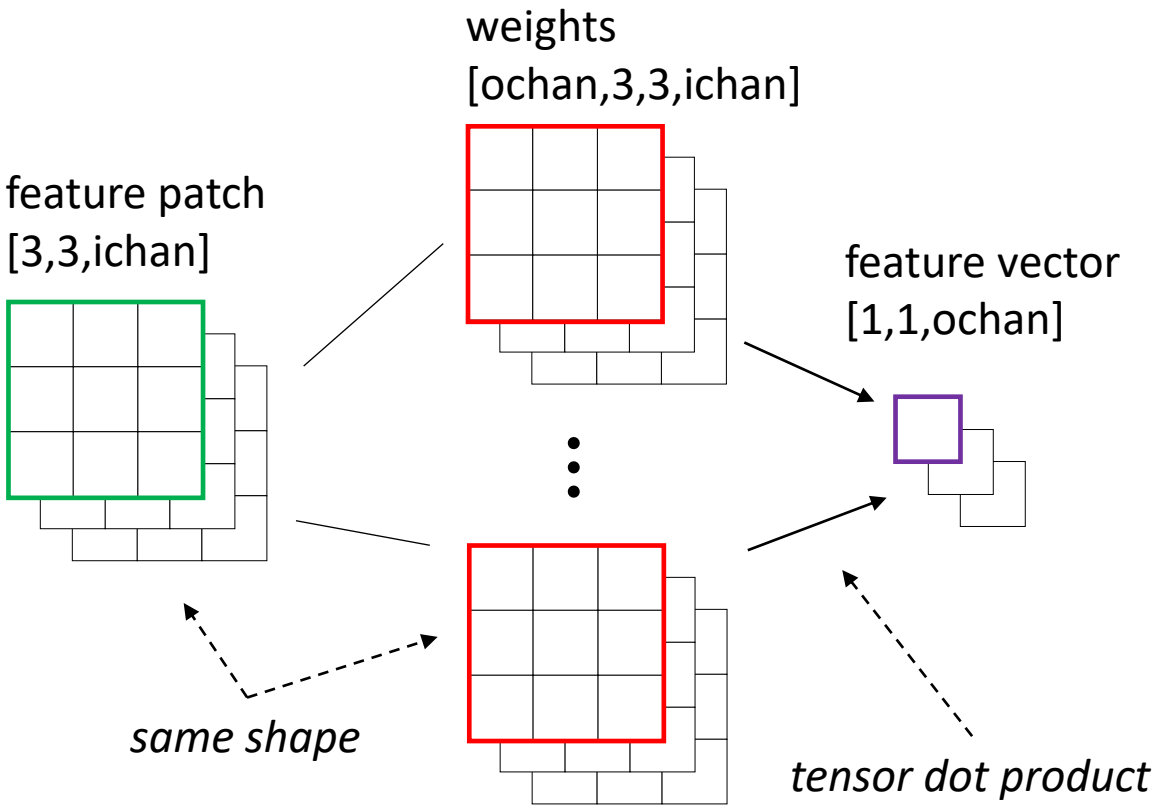
- initially target Gemini fabric
- high throughput, low latency deep model evaluation
- efficiently utilize FPGA BRAM and DSP resources
- package as a parameterized, reusable IP



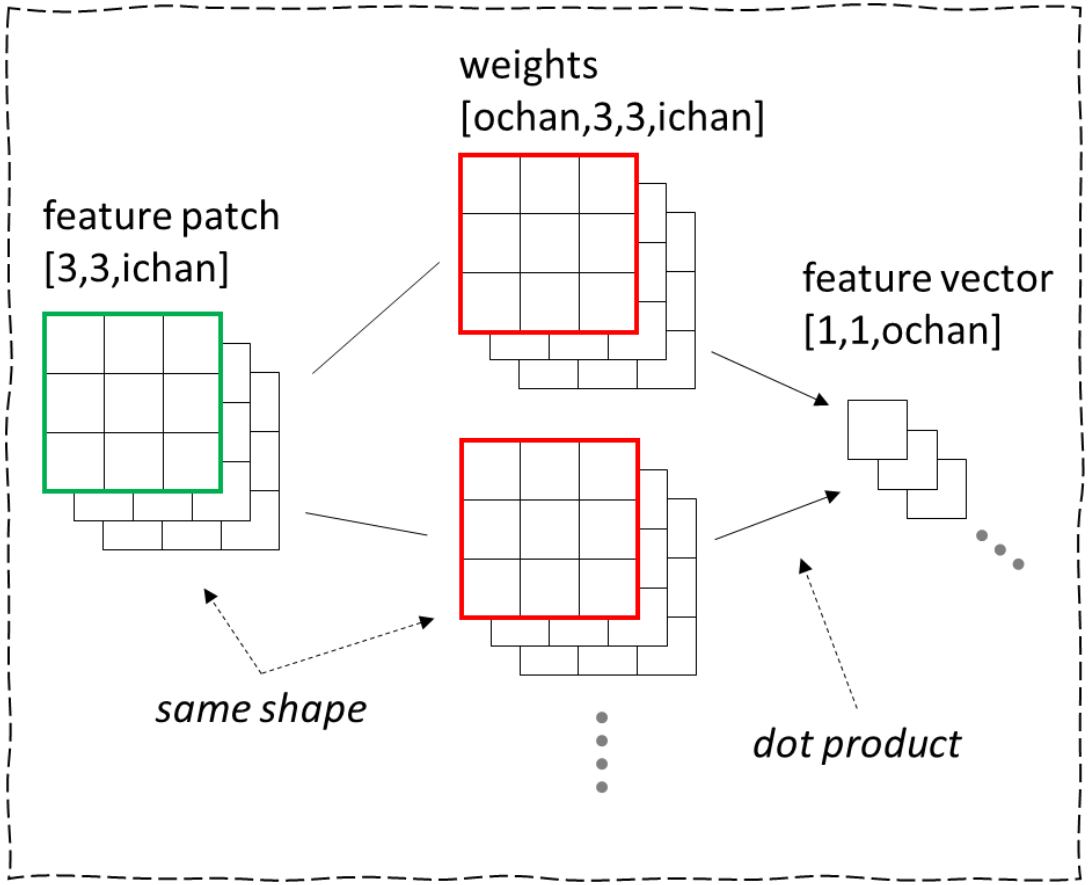
CNN primitive operation

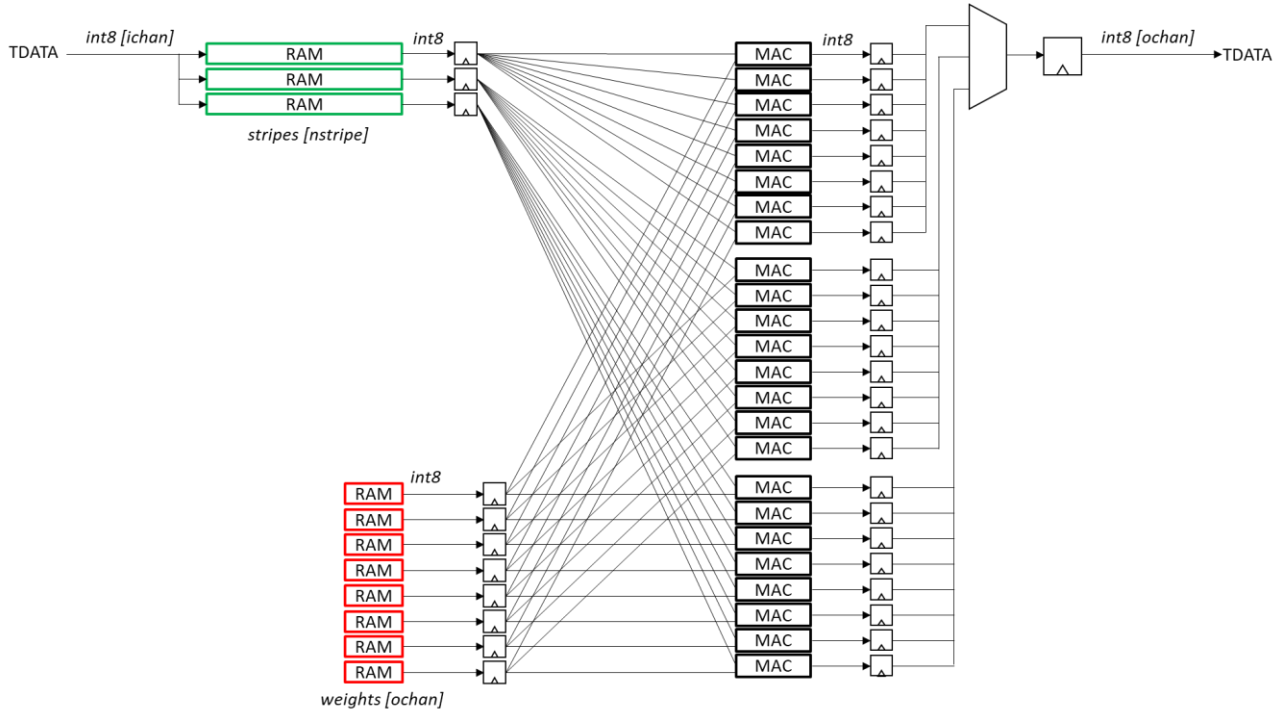


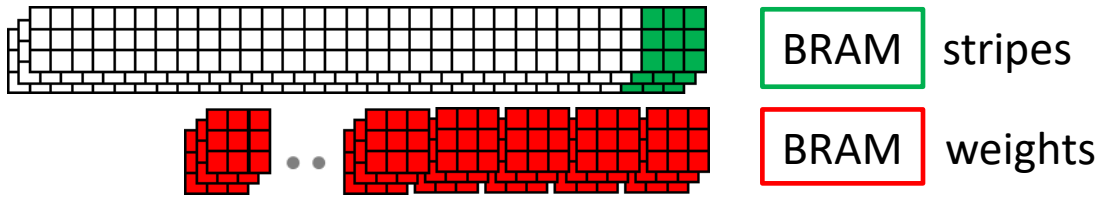
CNN primitive operation



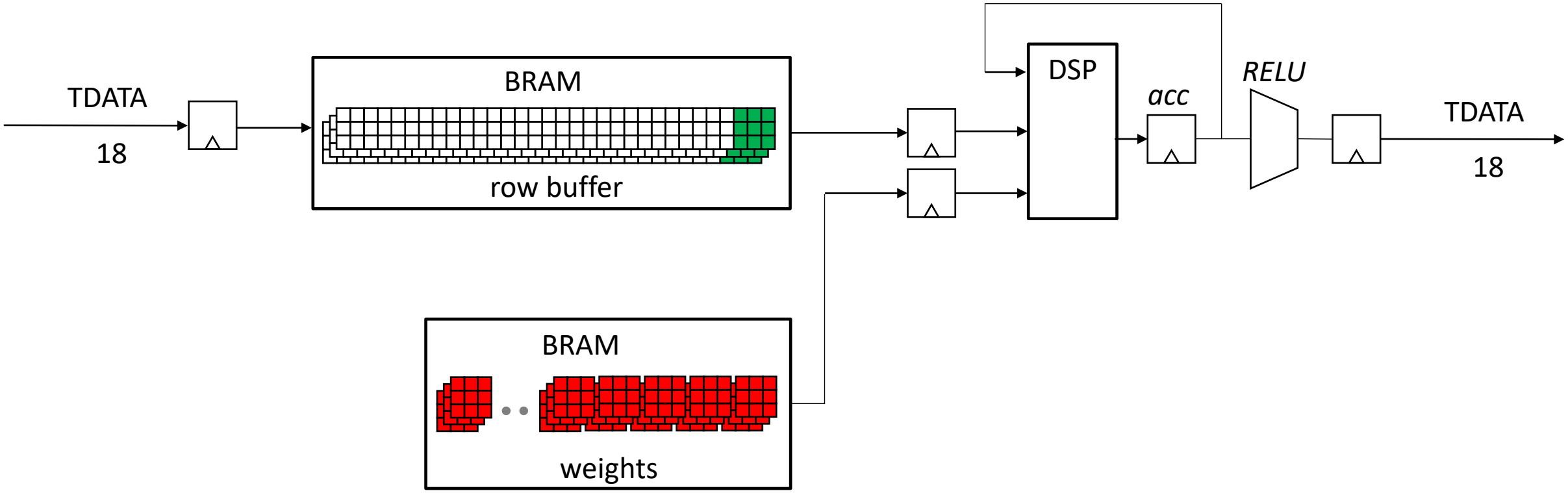
CNN dot product using FPGA



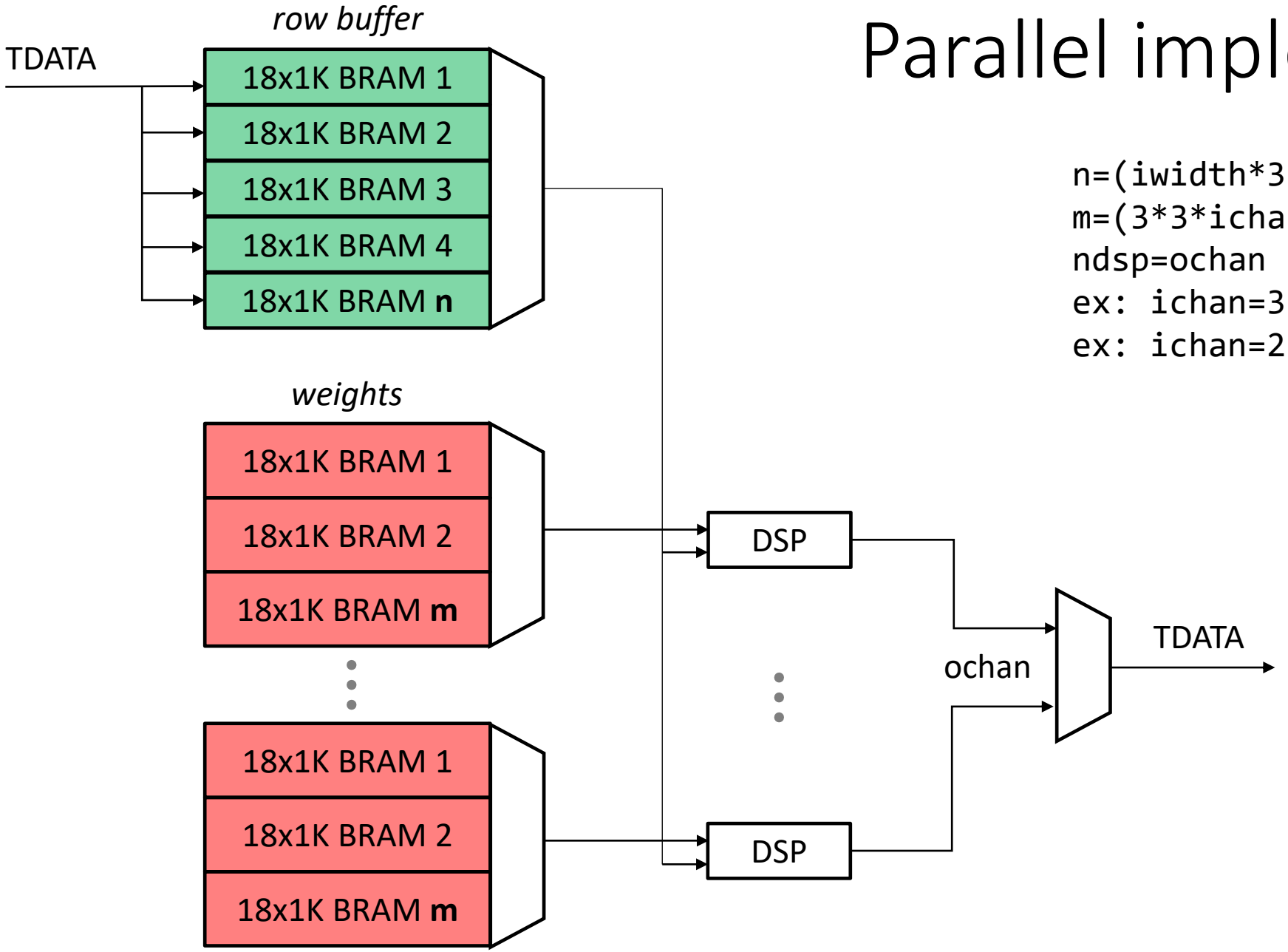




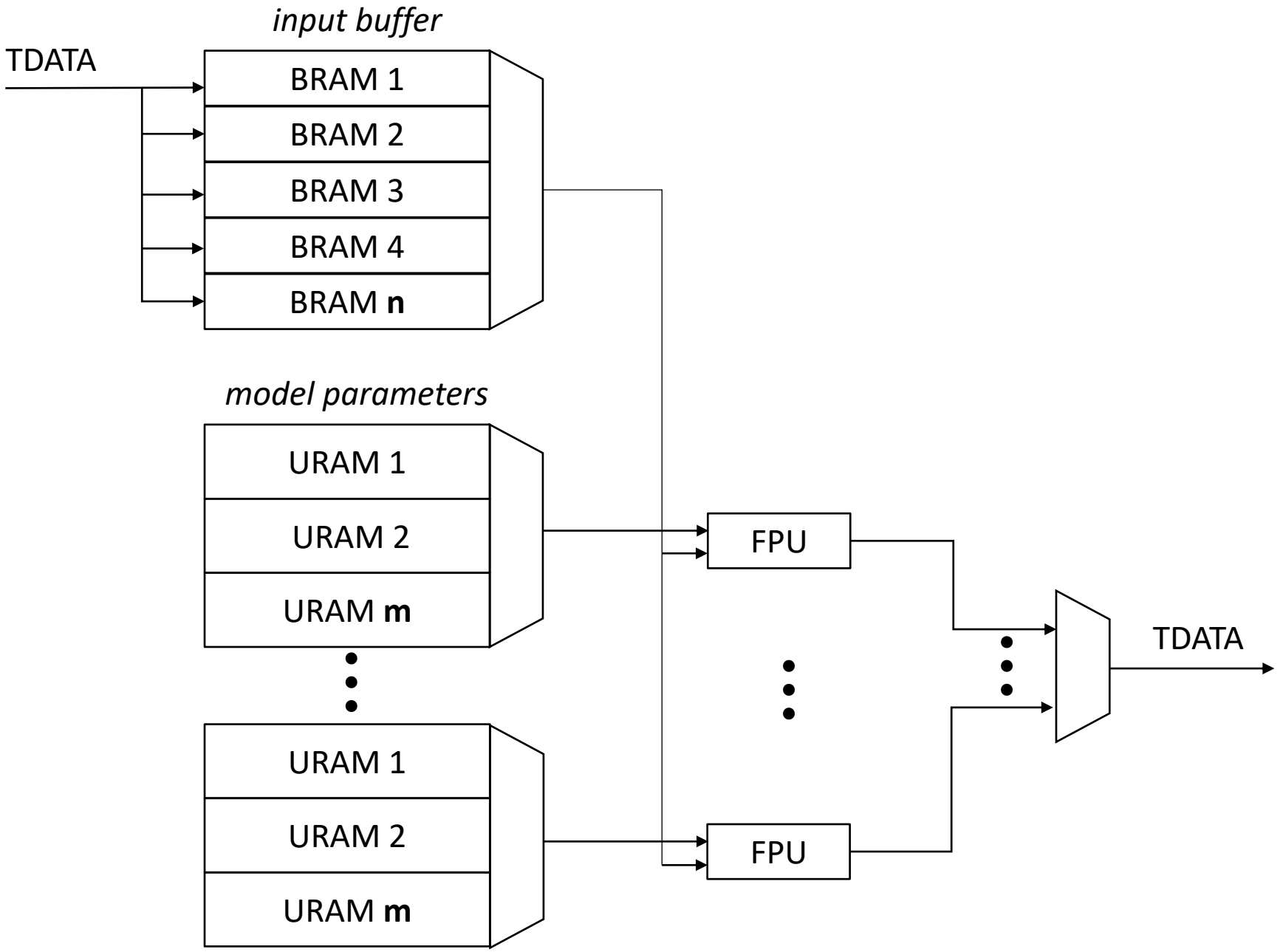
Sequential implementation



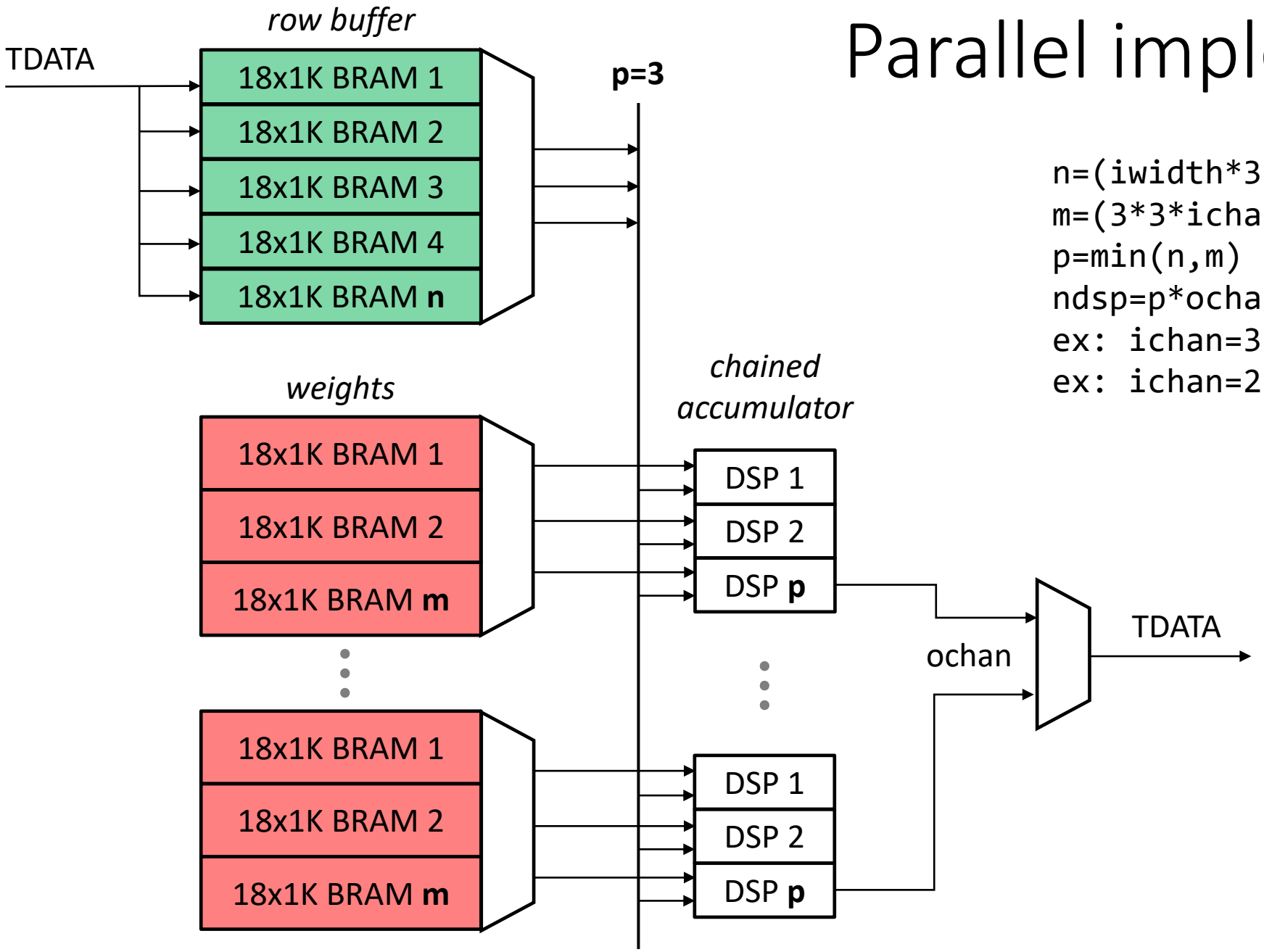
Parallel implementation




```
n=(iwidth*3*ichan)/1K
m=(3*3*ichan)/1K
ndsp=ochan
ex: ichan=3,ochan=16,iwidth=768
ex: ichan=256,ochan=256,iwidth=192
```

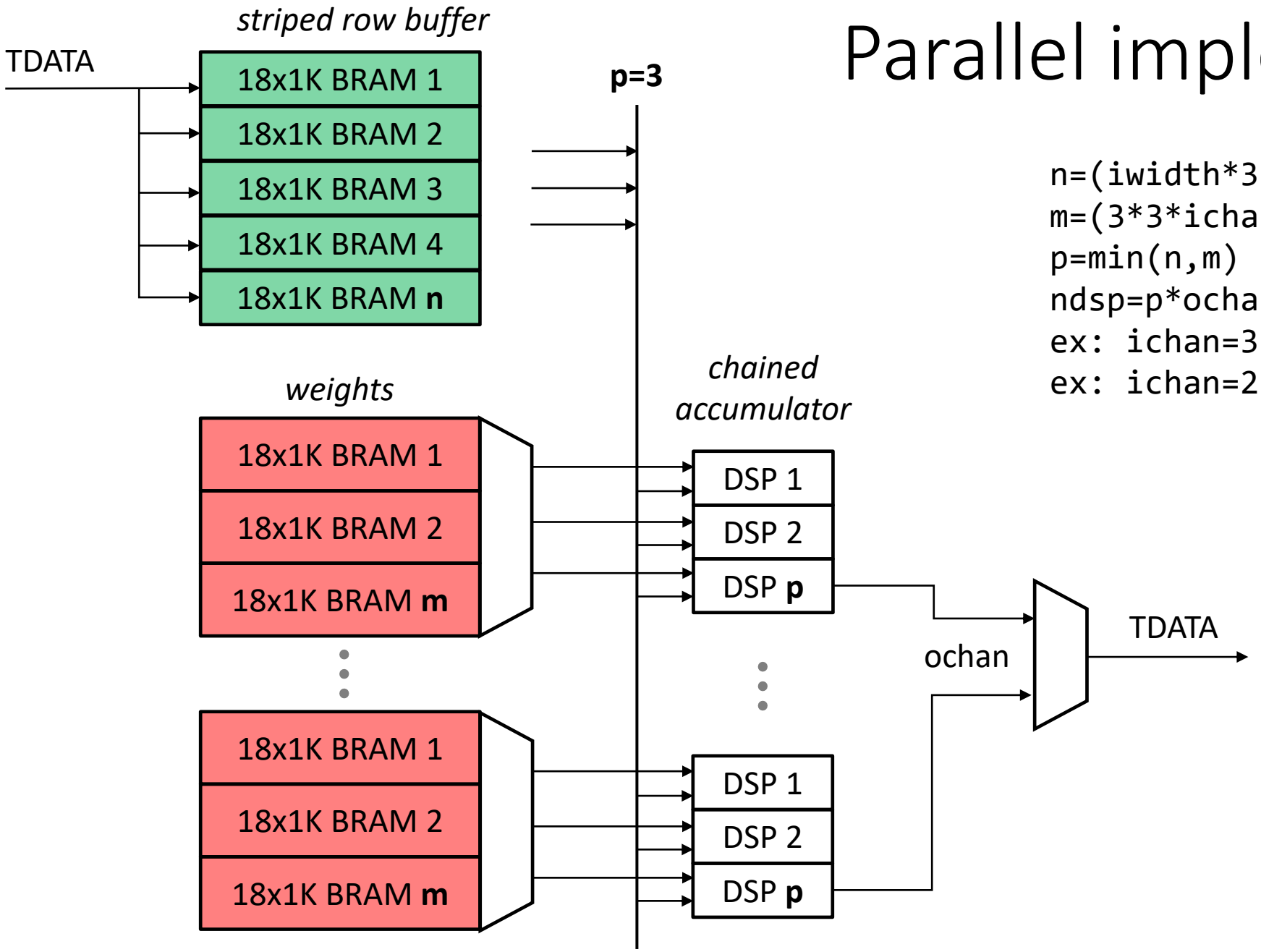

input buffer
TDATA
BRAM 1
BRAM 2
BRAM 3
BRAM 4
BRAM n
model parameters
URAM 1
URAM 2
URAM m
URAM 1
URAM 2
URAM m
FPU
FPU
TDATA

Parallel implementation

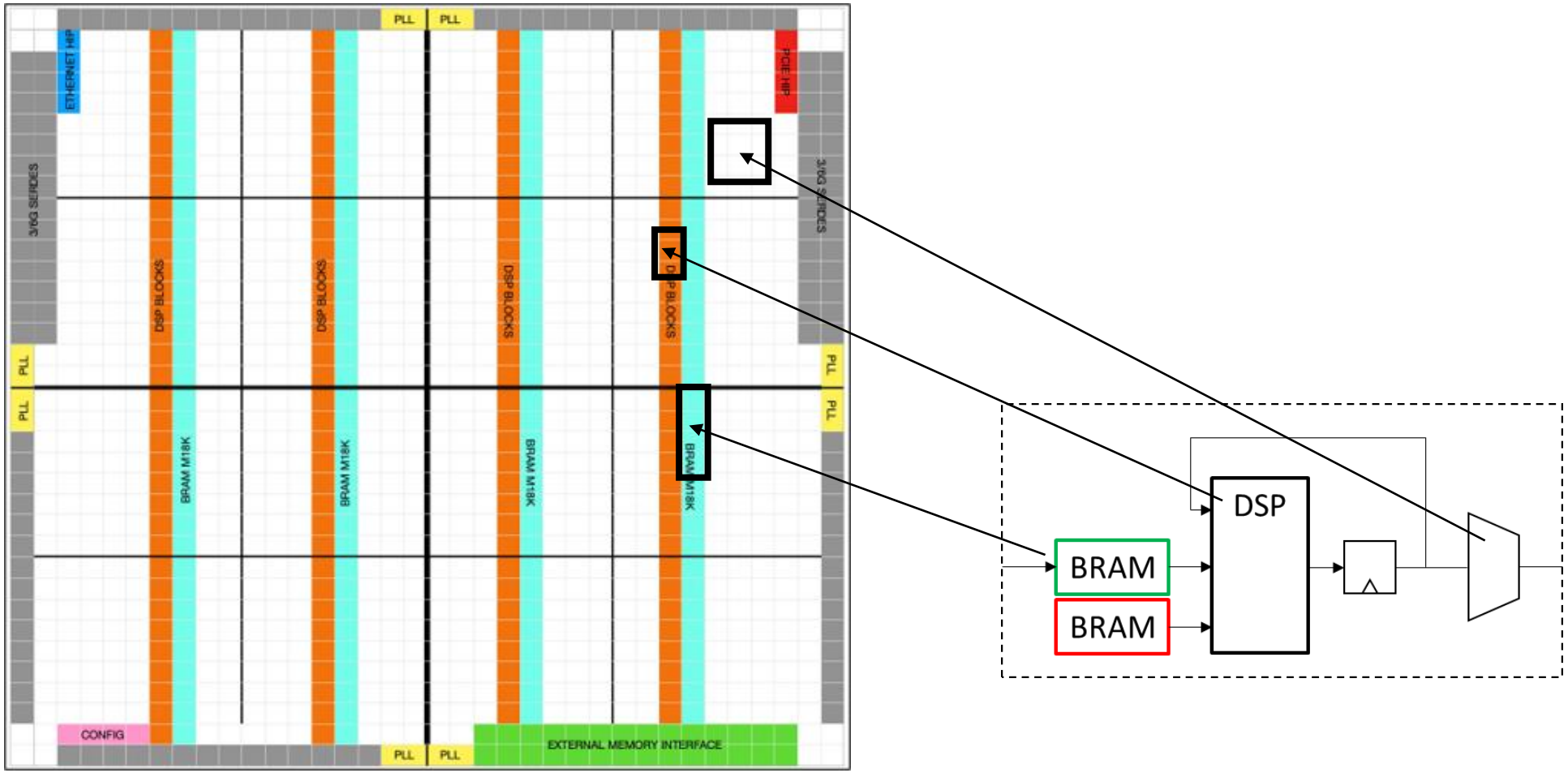



```
n=(iwidth*3*ichan)/1K
m=(3*3*ichan)/1K
p=min(n,m)
ndsp=p*ochan
ex: ichan=3,ochan=16,iwidth=768
ex: ichan=256,ochan=256,iwidth=192
```

Parallel implementation




```
n=(iwidth*3*ichan)/1K
m=(3*3*ichan)/1K
p=min(n,m)
ndsp=p*ochan
ex: ichan=3,ochan=16,iwidth=768
ex: ichan=256,ochan=256,iwidth=192
```


PLL
PLL
ETHERNET HIP
PCIE HIP
3/6G SERDES
DSP BLOCKS
BRAM M18K
CONFIG
EXTERNAL MEMORY INTERFACE
DSP
BRAM
BRAM

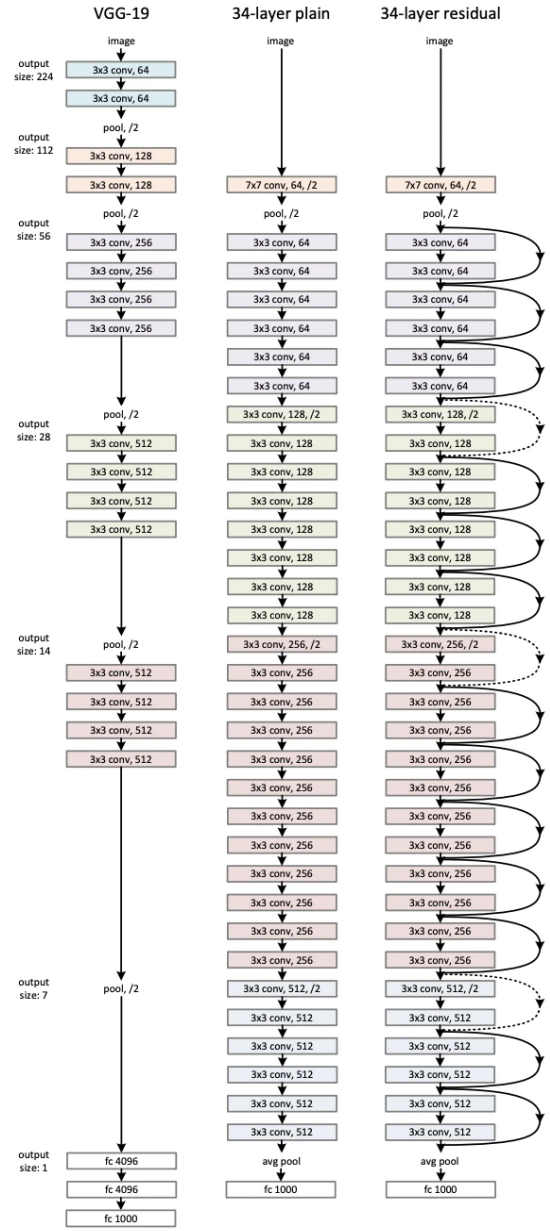
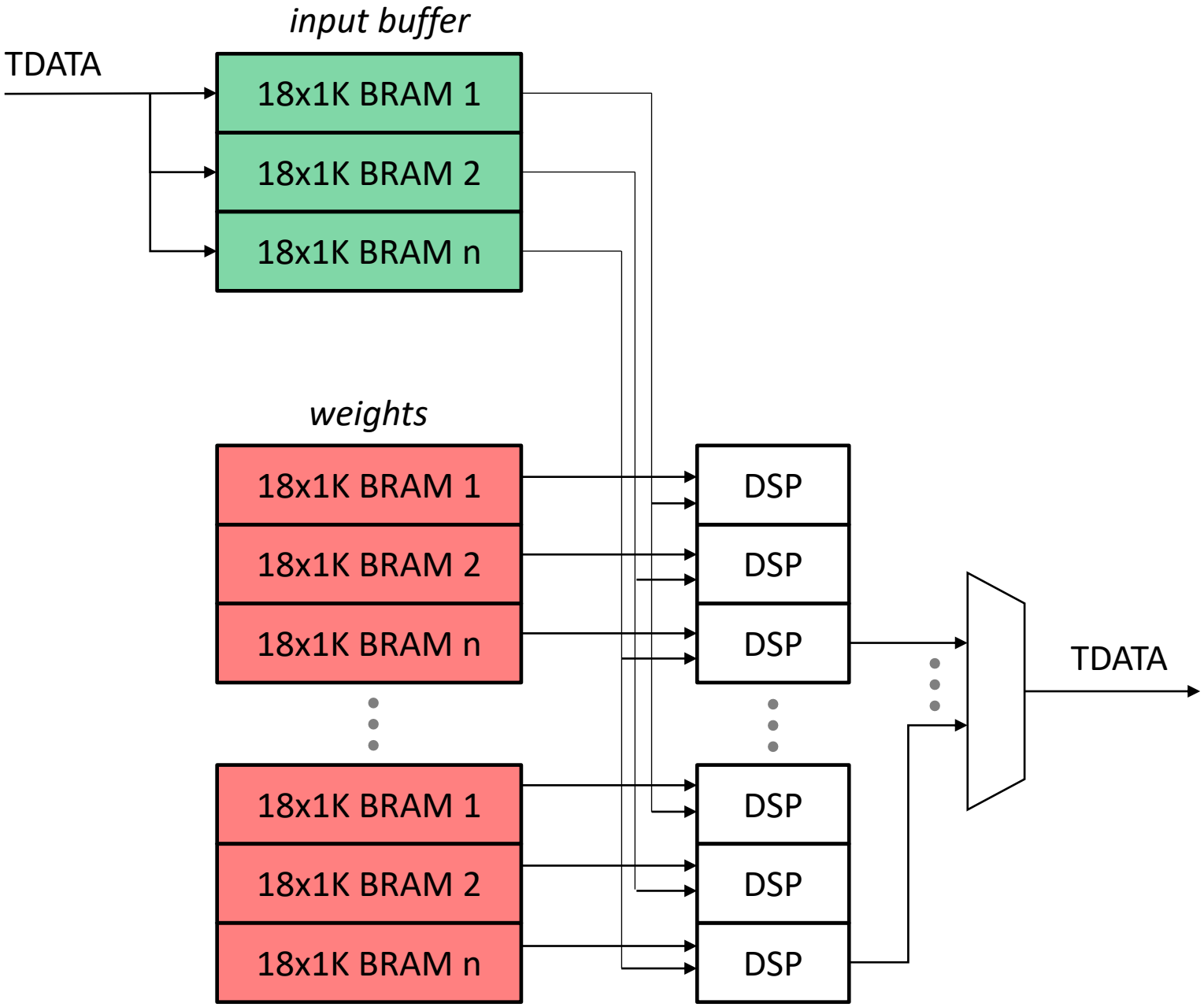
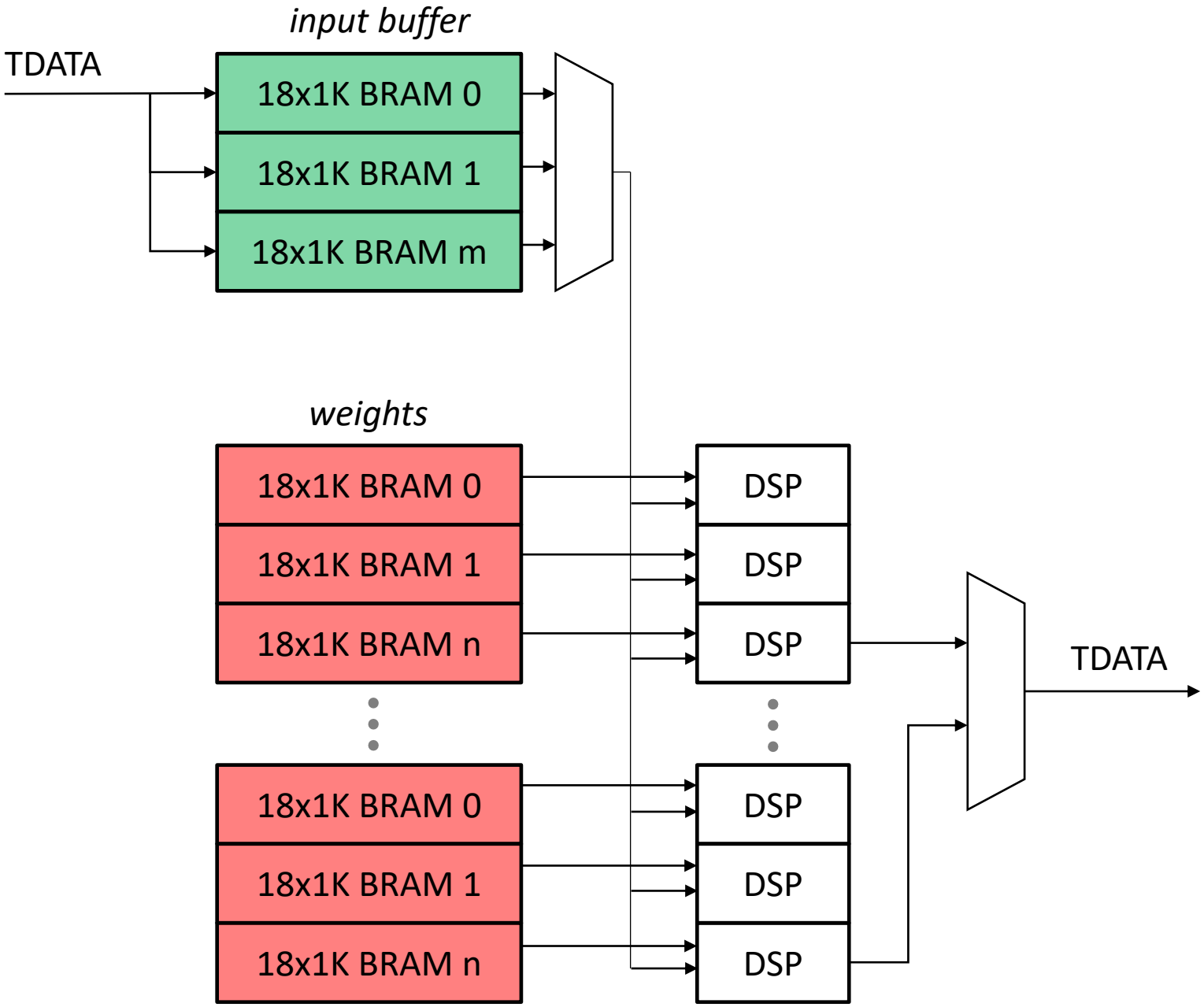
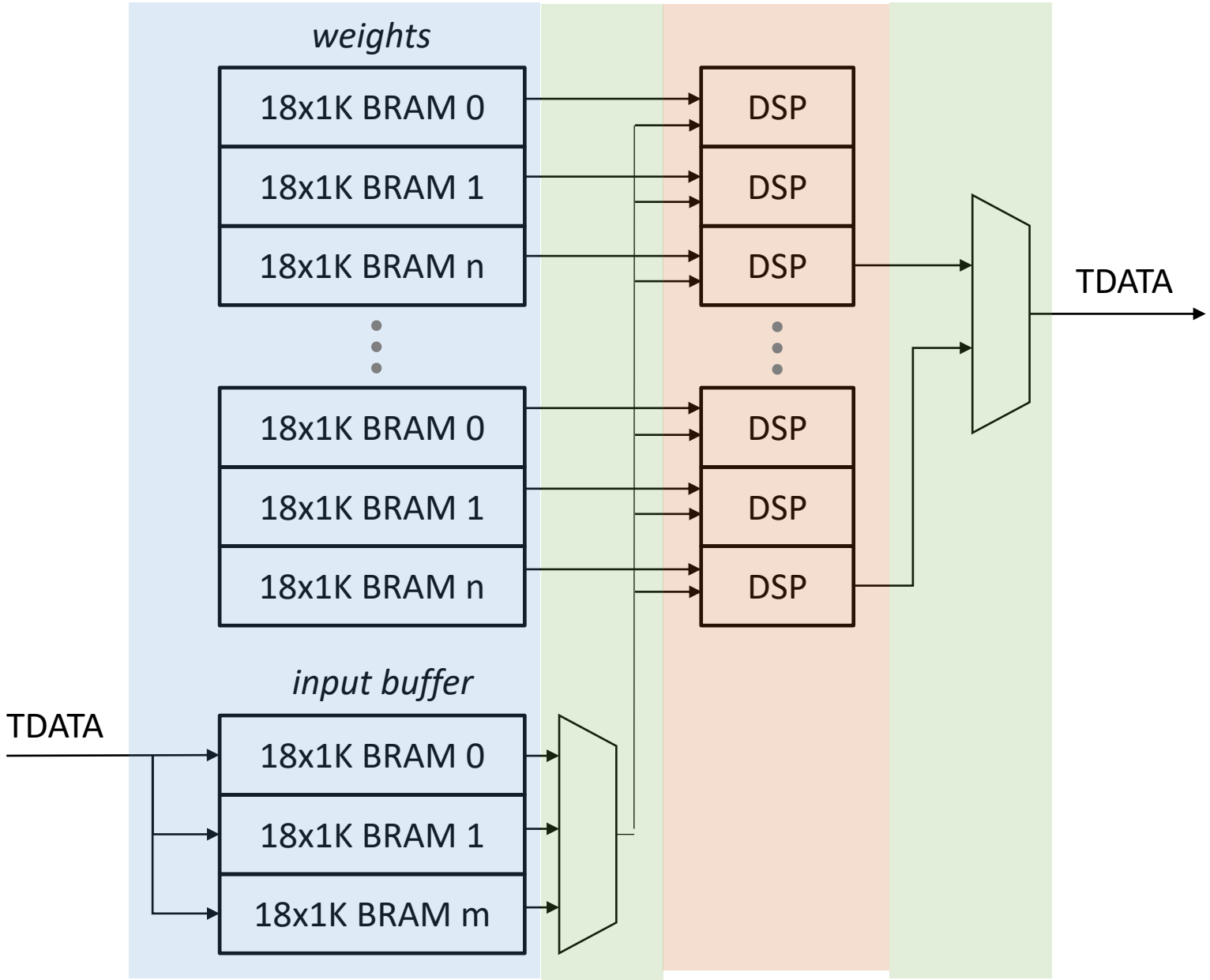


Figure 3. Example network architectures for ImageNet. **Left**: the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

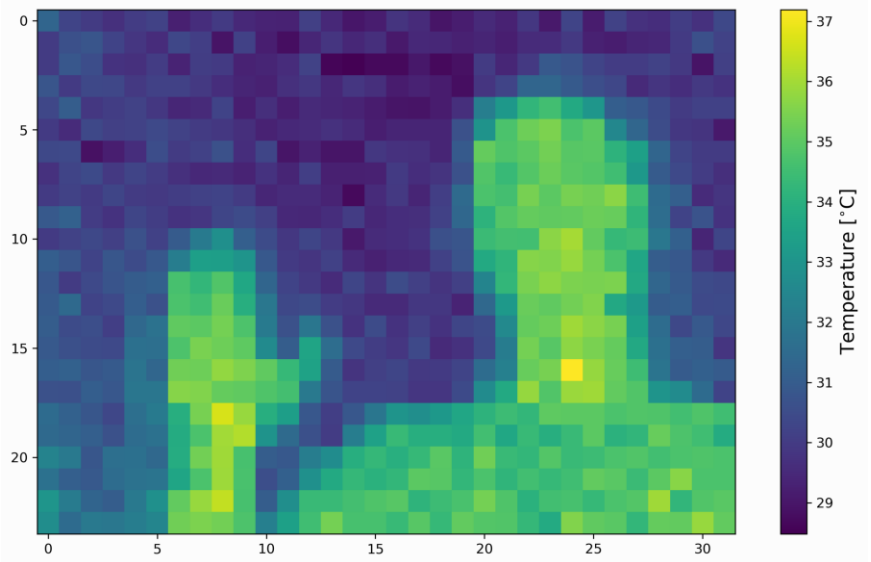

input buffer
TDATA
18x1K BRAM 1
18x1K BRAM 2
18x1K BRAM n
weights
18x1K BRAM 1
18x1K BRAM 2
18x1K BRAM n
18x1K BRAM 1
18x1K BRAM 2
18x1K BRAM n
DSP
DSP
DSP
DSP
DSP
DSP
TDATA


input buffer
TDATA
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM m
weights
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM n
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM n
DSP
DSP
DSP
DSP
DSP
DSP
TDATA


weights
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM n
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM n
DSP
DSP
DSP
DSP
DSP
DSP
TDATA
input buffer
TDATA
18x1K BRAM 0
18x1K BRAM 1
18x1K BRAM m

AI application concept – thermal imaging CNN

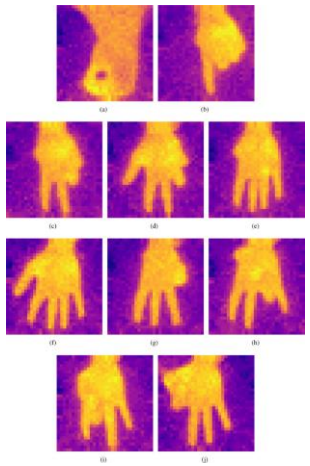
MLX90640 IR Array Thermal Imaging Camera
Module 32×24 Pixels 55° Field of View Camera
with I2C Interface for Raspberry Pi/Arduino



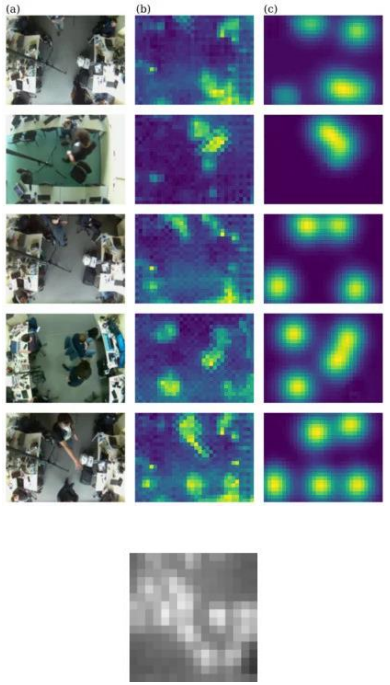
32x24 calibrated FIR pixels, 14 bit precision, 32 fps
High precision non-contact temperature measurements
Intrusion / Movement detection
Presence detection / Person localization
Temperature sensing element for intelligent HVAC
Thermal Comfort sensor in automotive air conditioning control systems
Microwave ovens
Industrial temperature control of moving parts
Visual IR thermometers

Example CNN tasks and training data

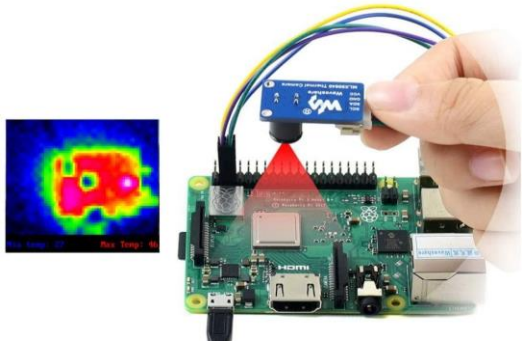
Gesture Recognition
Sign language

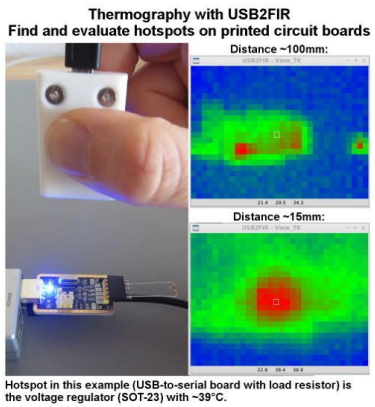


Occupancy Counting
Privacy preserving
Pose detection

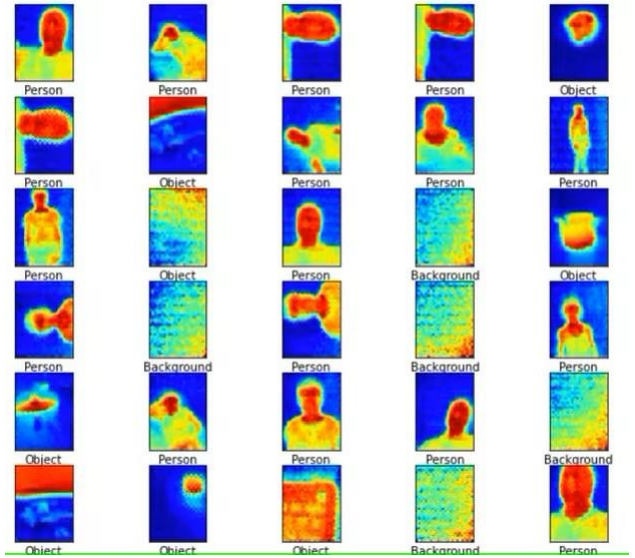


Industrial Inspection
Anomaly Detection
PCB hot spot analysis
Animal health screening

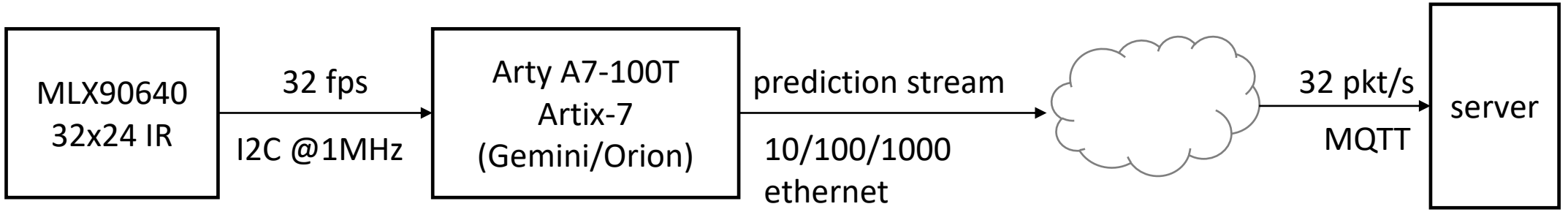


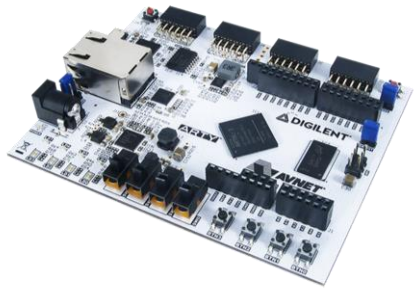


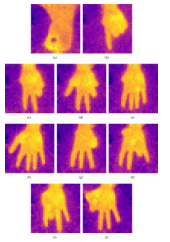
Action Recognition
Human fall detection



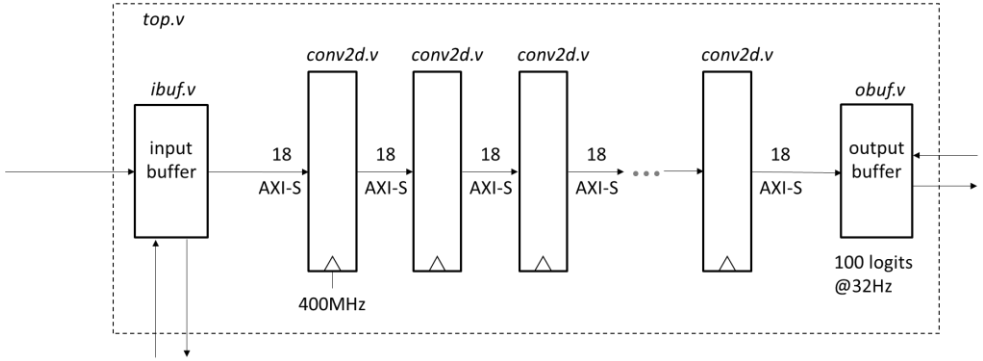
AI application prototype

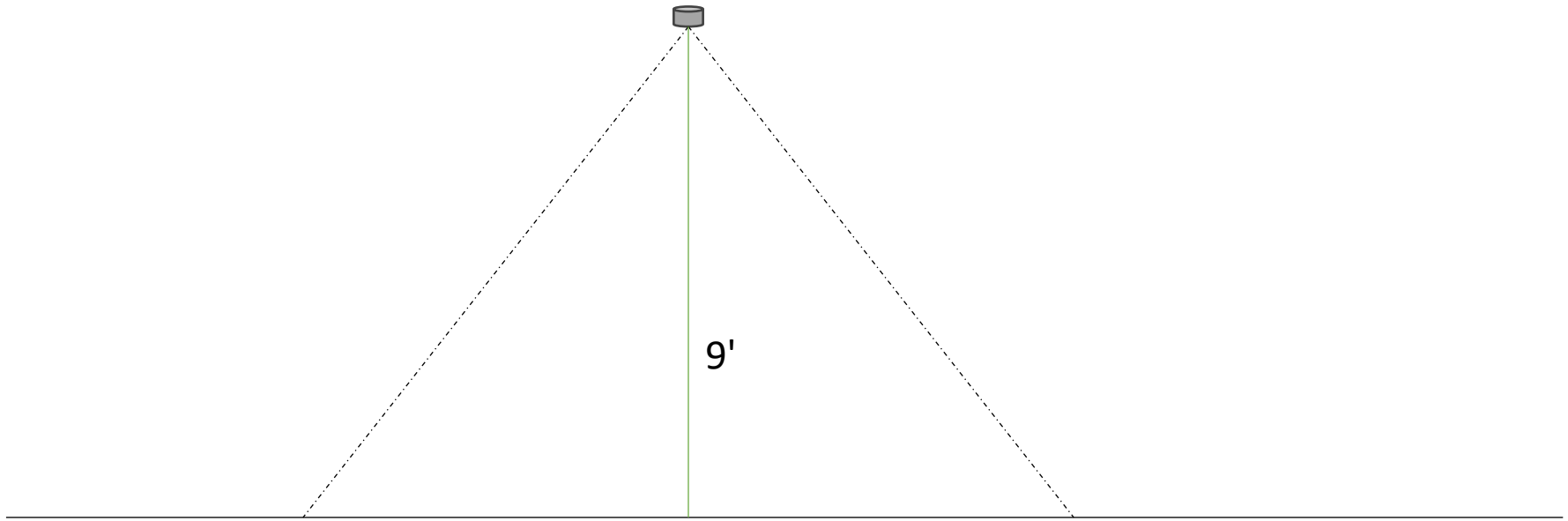






"accelerator" vs. hard real time
All parameters on chip using BRAM
Model evaluated entirely in FPGA fabric
Real time streaming pipeline





110 deg : 24' diameter
55 deg : 10' diameter